

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU VÀ XÂY DỰNG CÁC GIẢI PHÁP ƯỚC
LƯỢNG THỜI GIAN ĐẾN ĐÍCH VÀ TÌM ĐƯỜNG HIỆU QUẢ

Ngành : KHOA HỌC MÁY TÍNH

HỘI ĐỒNG: 10L KHOA HỌC MÁY TÍNH

GVHD: PGS. TS. TRẦN MINH QUANG

GVPB: TS. PHAN TRỌNG NHÂN

---o0o---

SVTH 1: NGUYỄN HỮU HUY THỊNH – 2213291

SVTH 2: NGUYỄN ANH KHOA – 2211612

TP. HỒ CHÍ MINH, 06/2026

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

KHOA: KH & KT Máy tính
BỘ MÔN: Hệ Thống Thông Tin

NHIỆM VỤ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP
Chú ý: Sinh viên phải dán tờ này vào trang nhất của bản thuyết trình

HỌ VÀ TÊN: Nguyễn Hữu Huy Thịnh
HỌ VÀ TÊN: Nguyễn Anh Khoa

MSSV: 2213291

MSSV: 2211612

NGÀNH: Khoa Học Máy Tính

LỚP:

1. Đầu đề luận văn/ đồ án tốt nghiệp:

Nghiên cứu xây dựng các giải pháp ước lượng thời gian đến đích và tìm đường hiệu quả.

2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):

- Giai đoạn 1 (đồ án chuyên ngành):

- Khảo sát các nghiên cứu và các giải pháp hiện có về ước lượng thời gian đến đích và các giải pháp tìm đường đi, ví dụ DeepTTE, GCN, ...
- Chọn lựa giải pháp phù hợp thực tiễn, đề xuất các cải tiến. Các giải pháp phải có khả năng ước lượng và đề xuất các đường đi hiệu quả với độ chính xác cao và tốc độ xử lý nhanh so với các giải pháp hiện có.
- Xây dựng demo để đánh giá các giải pháp trên dữ liệu thật ở TP.HCM.

- Giai đoạn 2 (đồ án tốt nghiệp):

- Xây dựng mô hình huấn luyện/suy luận (inference) thời gian thực, đảm bảo độ trễ thấp.
- Hiện thực hóa giải thuật Multilevel Dijkstra (MLD) để thay thế cho các giải thuật tìm đường cơ bản, đảm bảo khả năng đáp ứng truy vấn trên mạng lưới đường bộ quy mô lớn (TP.HCM).
- Thiết lập pipeline dữ liệu tự động hóa: từ dữ liệu OSM/PoI đến các mô hình dự báo thời gian (TTE) và đẩy kết quả vào bộ nhớ đệm (cache) của hệ thống tìm đường.
- Phát triển lớp Backend có khả năng cân bằng tải, phân mảnh các tác vụ nặng (Map Matching, tính toán trọng số cạnh) để đảm bảo hệ thống không bị treo khi có nhiều người dùng đồng thời.
- Đánh giá thực nghiệm tốc độ của MLD trong điều kiện mạng lưới giao thông biến động và kiểm tra tính chính xác của dự báo TTE so với thực tế.

3. Ngày giao nhiệm vụ: 05/01/2026

4. Ngày hoàn thành nhiệm vụ: 11/05/2026

5. Họ tên giảng viên hướng dẫn:

Phản hướng dẫn:

- 1) PGS. Trần Minh Quang
- 2) ThS. Đỗ Thanh Thái

Nội dung và yêu cầu LVTN/ ĐATN đã được thông qua Bộ môn.

Ngày tháng 05 năm 2026

CHỦ NHIỆM BỘ MÔN

(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN CHÍNH

(Ký và ghi rõ họ tên)



PGS. TS. Trần Minh Quang

PHẦN DÀNH CHO KHOA, BỘ MÔN:

Người duyệt (chấm sơ bộ):

Đơn vị:

Ngày bảo vệ:

Điểm tổng kết:

Nơi lưu trữ LVTN/ĐATN:

PHIẾU CHẤM BẢO VỆ ĐỒ ÁN/ LUẬN VĂN TỐT NGHIỆP

(Dành cho người hướng dẫn)

1. Họ và tên SV: **NGUYỄN ANH KHOA**

MSSV: 2211612

Ngành (chuyên ngành): Khoa học Máy tính

Họ và tên SV: **NGUYỄN HỮU HUY THỊNH**

MSSV: 2213291

Ngành (chuyên ngành): Khoa học Máy tính

2. Đề tài: Nghiên cứu xây dựng các giải pháp ước lượng thời gian đến đích và tìm đường hiệu quả

3. Họ tên người hướng dẫn: Trần Minh Quang

4. Tổng quát về bản thuyết minh:

Số trang: 114

Số chương: 8

Số bảng số liệu:

Số hình vẽ:

Số tài liệu tham khảo: 35

Phần mềm tính toán:

Hiện vật (sản phẩm):

5. Tổng quát về các bản vẽ:

- Số bản vẽ: Bản A1:

Bản A2:

Khổ khác:

- Số bản vẽ vẽ tay:

Số bản vẽ trên máy tính:

6. Những ưu điểm chính của LVTN:

- Mô hình PoI-CL-TTE tích hợp thông tin từ các Điểm quan tâm (PoI) xung quanh đoạn đường, khắc phục được điểm mù của các kiến trúc trước đó (như MulT-TTE) vốn chỉ tập trung vào dữ liệu trên chính tuyến đường mà bỏ qua áp lực giao thông lan truyền từ các khu vực lân cận.

- Bằng cách sử dụng kiến trúc MoCo và cặp âm mềm (soft negatives) dựa trên khoảng cách thời gian di chuyển, mô hình học được các biểu diễn bền vững trước những sai số, nhiễu dữ liệu đặc trưng của nền tảng bản đồ cộng đồng OpenStreetMap (OSM).

- PoI-CL-TTE đạt độ chính xác cao (sai số MAPE chỉ 1.96% trên tập dữ liệu TP.HCM) nhưng chỉ sử dụng khoảng 1,3 triệu tham số, tương đương 25% kích thước của mô hình MulT-TTE, giúp tiết kiệm đáng kể bộ nhớ và có thể triển khai trên các hạ tầng phần cứng hạn chế.

- Đồ án áp dụng thành công giải thuật Multilevel Dijkstra (MLD) phân tách hoàn toàn cấu trúc hình học và trọng số giao thông, cho phép cập nhật tình trạng ùn tắc nhanh gấp 60 lần so với giải thuật Contraction Hierarchies (CH) truyền thống.

- Ứng dụng triển khai kiến trúc Client-Server với API Gateway sử dụng thuật toán cân bằng tải Round-Robin và cơ chế xử lý bất đồng bộ, cho phép chia tách và thực thi song song các tác vụ nặng (Map Matching, suy luận AI) mà không gây sập hệ thống khi nhiều người dùng truy cập cùng lúc.

7. Những thiếu sót chính của LVTN:

- Do hạn chế về nguồn lực, nhóm sử dụng thời gian di chuyển từ API của TomTom làm dữ liệu huấn luyện thay vì quỹ đạo GPS thô thực tế; điều này đồng nghĩa với việc mô hình đang học cách xấp xỉ lại thuật toán dự báo của TomTom.

- Kỹ thuật dùng vùng đệm tĩnh bán kính 50 mét và mã hóa nhị phân (có/không) theo nhóm danh mục đã vô tình đánh đồng sức hút giao thông của các cơ sở có quy mô hoàn toàn khác nhau (ví dụ: một đại siêu thị và một cửa hàng nhỏ).

- Phạm vi nghiên cứu chủ ý loại trừ các tác nhân gây biến động mạnh đến thời gian di chuyển trong đô thị như hành vi lái xe cá nhân, sự cố tai nạn ngẫu nhiên, và thời gian chờ tại các cụm đèn tín hiệu giao thông.

- Trong các bài kiểm thử chịu tải (Stress Test), tỷ lệ truy vấn bị giới hạn bằng thông bởi Public OSRM lên tới 10-24%, buộc hệ thống phải chuyển về tính toán nội bộ, cho thấy sự mong manh khi phụ thuộc vào API mạng bên thứ ba.

- Dù có tốc độ truy vấn trung bình cực nhanh, lần chạy đầu tiên của MLD chưa có bộ nhớ đệm (cache) ghi nhận độ trễ khởi động khá lớn (lên tới 232.22 ms tại mạng lưới quy mô siêu lớn như Việt Nam).

8. Đề nghị: Được bảo vệ ☒ Bỏ sung thêm để bảo vệ ☐ Không được bảo vệ ☐

9. 3 câu hỏi SV phải trả lời trước Hội đồng:

a. Làm thế nào để chứng minh mô hình PoI-CL-TTE không chỉ “bắt chước” thuật toán của TomTom mà còn phản ánh đúng quy luật giao thông thực tế?

b. Có thể áp dụng các phương pháp định lượng như Gravity Model hoặc Kernel Density Estimation để mô tả trọng số và mật độ PoI chính xác hơn không?

c. Hệ thống sẽ thiết kế feedback loop như thế nào để cập nhật liên tục dự báo từ PoI-CL-TTE vào đồ thị MLD theo thời gian thực?

10. Đánh giá chung (bằng chữ: giỏi, khá, TB): Giỏi

Điểm : 9 /10

NGƯỜI VIẾT PHIẾU CHẤM

(ký và ghi rõ họ tên)



Trần Minh Quang

TP. HCM, ngày 15 tháng 6 năm 2026

PHIẾU CHẤM BẢO VỆ ĐỒ ÁN/ LUẬN VĂN TỐT NGHIỆP

(Dành cho người phản biện)

1. Họ và tên SV: **NGUYỄN ANH KHOA**

MSSV: 2211612

Ngành (chuyên ngành): Khoa học Máy tính

Họ và tên SV: **NGUYỄN HỮU HUY THỊNH**

MSSV: 2213291

Ngành (chuyên ngành): Khoa học Máy tính

2. Đề tài: Nghiên cứu xây dựng các giải pháp ước lượng thời gian đến đích và tìm đường hiệu quả

3. Họ tên người phản biện: Phan Trọng Nhân

4. Tổng quát về bản thuyết minh:

Số trang:

Số chương:

Số bảng số liệu:

Số hình vẽ:

Số tài liệu tham khảo:

Phần mềm tính toán:

Hiện vật (sản phẩm):

5. Tổng quát về các bản vẽ:

- Số bản vẽ: Bản A1:

Bản A2:

Khô khác:

- Số bản vẽ vẽ tay:

Số bản vẽ trên máy tính:

6. Những ưu điểm chính của LVTN:

- Nhóm sinh viên ứng dụng thuật giải Multi-level Dijkstra để tìm đường đi ngắn nhất dựa trên khoảng cách, sau đó đề xuất phương pháp POI-CL-TTE để ước lượng thời gian di chuyển trên đoạn đường này. Các phân đoạn đường được trích xuất đặc trưng (độ dài đoạn đường, độ dài tích lũy, loại đường, tốc độ tối đa, số làn xe, thông tin POI lân cận). Từ đó, nhóm sinh viên đề xuất 2 luồng xử lý độc lập: luồng học tương phản và luồng dự đoán thông qua cơ chế FiLM. Các dữ liệu bản đồ và di chuyển được thu thập và tiền xử lý với sự quan sát từ tập dữ liệu. Kết quả từ 2 luồng xử lý này được đưa vào mạng GRU, mô đun "Attention Pooling" và mô đun Hồi quy dự đoán để dự đoán thời gian di chuyển.

- Nhóm sinh viên đã thực hiện các thí nghiệm so sánh và đánh giá. Kết quả cho thấy thời gian dự đoán từ phương pháp đề xuất có độ sai số tốt hơn so với phương pháp DeepTTE và Mult-TTE trên hai tập Porto và TP.HCM.

- Nhóm sinh viên đã sử dụng các công nghệ hiện đại phát triển ứng dụng web để trực quan hóa kết quả của phương pháp.

7. Những thiếu sót chính của LVTN:

- Động lực để thực hiện tăng cường dữ liệu chưa đủ thuyết phục trong khi đặc trưng phân bố dữ liệu chưa được khảo sát. Hơn nữa, các phương pháp tăng cường được sử dụng chưa có cơ sở lý thuyết hoặc thực nghiệm chứng minh lợi ích của các phương pháp này.

- Nhóm sinh viên cần thực hiện nhiều thí nghiệm hơn, bao gồm cả thí nghiệm suy giảm để đánh giá phương pháp đề xuất.

- Báo cáo Đồ án tốt nghiệp cần được xem lại về cấu trúc chương 2 và 3 cho hợp lý, các mục con đơn lẻ, thứ tự nội dung (5.1, 5.2, và 5.3) và cách trích dẫn tham khảo cho đầy đủ.

8. Đề nghị: Được bảo vệ ☒ Bổ sung thêm để bảo vệ ☐ Không được bảo vệ ☐

9. 3 câu hỏi SV phải trả lời trước Hội đồng:

1. Sinh viên hãy nêu vai trò của FiLM trong giải pháp.

2. Sinh viên hãy nêu sự cần thiết của tăng cường dữ liệu trong giải pháp đề xuất.

10. Đánh giá chung (bằng chữ: giỏi, khá, TB): Giỏi

Điểm : 9.4 /10

NGƯỜI VIẾT PHIẾU CHẤM

(ký và ghi rõ họ tên)



Phan Trọng Nhân



LỜI CAM ĐOAN

Chúng em xin tuyên bố rằng đồ án tốt nghiệp là công trình do chính chúng em thực hiện, trừ những phần đã được trích dẫn hoặc ghi nhận nguồn rõ ràng. Nội dung của đồ án là kết quả của quá trình nghiên cứu và nỗ lực cá nhân của chúng em, và chúng em đã tuân thủ đầy đủ các quy định đạo đức và tiêu chuẩn học thuật. Chúng em nhận thức rõ hậu quả của hành vi gian lận học thuật và hiểu rằng bất kỳ vi phạm nào đối với chuẩn mực đạo đức trong đồ án này đều có thể dẫn đến các hình thức kỷ luật theo quy định của Trường Đại học Bách Khoa – Đại học Quốc gia TP. Hồ Chí Minh.

Sinh viên thực hiện

Nguyễn Hữu Huy Thịnh

Sinh viên thực hiện

Nguyễn Anh Khoa



LỜI CẢM ƠN

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến PGS.TS. Trần Minh Quang và ThS. Đỗ Thanh Thái vì sự hỗ trợ và hướng dẫn vô giá. Nghiên cứu của chúng em đã được hưởng lợi rất nhiều từ kiến thức sâu rộng, những lời phê bình sâu sắc, và sự hỗ trợ liên tục của các thầy. Ngoài ra, chúng em cũng xin gửi lời cảm ơn đến gia đình. Niềm tin vững chắc của mọi người vào khả năng và sự động viên không ngừng nghỉ của mọi người chính là chỗ dựa vững chắc cho chúng em. Niềm tin của gia đình vào tiềm năng của chúng em chính là nguồn động lực và sức mạnh bền bỉ không ngừng nghỉ. Chúng em mãi mãi biết ơn tình yêu thương và sự hỗ trợ của tất cả mọi người.

TÓM TẮT

Đề án này tập trung nghiên cứu và khảo sát các giải pháp cho bài toán ước lượng thời gian di chuyển tối ưu nhất hiện nay, đồng thời đề xuất một mô hình học sâu tốt hơn mang tên PoI-CL-TTE. Mô hình được thiết kế nhằm tối ưu hóa và có khả năng tái sử dụng linh hoạt trên mạng lưới giao thông của các thành phố lớn trên thế giới. Nhằm cải thiện độ chính xác, phương pháp đề xuất thay thế các đặc trưng hình học thuần túy bằng việc khai thác thông tin ngữ nghĩa từ các điểm quan tâm, kết hợp cùng cơ chế học tương phản nhằm nâng cao khả năng tổng quát hóa và học được các biểu diễn bền vững trước nhiễu dữ liệu. Mô hình dự báo này được huấn luyện và kiểm thử dựa trên nguồn dữ liệu thực tế thu thập từ TomTom và OpenStreetMap.

Song song với bài toán dự đoán thời gian di chuyển, đề án tiến hành nghiên cứu, khảo sát và tái sử dụng giải thuật Dijkstra đa cấp để giải quyết bài toán tìm đường tối ưu. Đây được đánh giá là một trong những giải thuật tìm đường đi ngắn nhất tiên tiến và tối ưu nhất hiện nay. Nhờ khả năng nén dữ liệu và phân tách giữa cấu trúc mạng lưới với trọng số giao thông, giải thuật mang lại hiệu năng truy vấn tốc độ cao trên các bản đồ quy mô lớn.

Mục tiêu cuối cùng của đề án là tích hợp trực tiếp mô hình dự báo thời gian và giải thuật tìm đường vào một ứng dụng bản đồ giao thông trực tuyến nhằm hỗ trợ người dùng tìm đường và cung cấp các ước lượng giao thông thực tế một cách chính xác và hiệu quả.



Mục lục

LỜI CAM ĐOAN	1
LỜI CẢM ƠN	2
TÓM TẮT	3
Danh sách hình ảnh	8
Danh sách bảng	10
1 Giới thiệu	11
1.1 Giới thiệu	11
1.2 Mục tiêu nghiên cứu	12
1.3 Phạm vi nghiên cứu	13
1.4 Đối tượng nghiên cứu	13
2 Cơ sở lý thuyết	15
2.1 Học giám sát	15
2.2 Học tự giám sát	15
2.3 Học tương phản (Contrastive Learning)	16
2.4 Mạng nơ-ron nhiều lớp (Multi-Layer Perceptron)	16
2.5 Cơ chế tự chú ý (Self-Attention)	17
2.6 Cơ chế tự chú ý đa đầu (Multi-head Self-attention)	17
2.7 Bộ mã hóa Transformer (Transformer Encoder)	18
2.8 Mạng nơ-ron hồi quy (Recurrent Neural Network)	18
2.9 Gated Recurrent Unit	20
2.10 BERT	21
2.11 Mạng tích chập	23
2.12 Hàm mất mát InfoNCE	24
2.12.1 Công thức gốc	24
2.12.2 Vai trò của nhiệt độ τ	24
2.13 Hàm mất mát SmoothL1	24
2.14 Độ đo đánh giá	24
2.14.1 Sai số tuyệt đối trung bình (Mean Absolute Error - MAE)	25
2.14.2 Sai số tuyệt đối tương đối trung bình (Relative Mean Absolute Error - RMAE)	25

2.14.3	Căn bậc hai sai số bình phương trung bình (Root Mean Squared Error - RMSE)	25
2.15	Định nghĩa bài toán dự đoán thời gian di chuyển	25
2.16	Định nghĩa bài toán tìm đường tối ưu	27
3	Công trình nghiên cứu liên quan	28
3.1	Tổng quan về dự đoán thời gian di chuyển	28
3.1.1	Hướng tiếp cận cho bài toán dự đoán thời gian di chuyển . .	28
3.1.2	Tổng quan về phương pháp dự đoán thời gian di chuyển dựa trên lộ trình	29
3.2	Trích xuất đặc trưng	30
3.3	Điểm quan tâm (Points of Interest - PoI)	31
3.4	DeepTTE: Mô hình ước lượng thời gian dựa trên mạng nơ-ron sâu .	31
3.5	MulT-TTE: Phương pháp học biểu diễn lộ trình đa phương thức . .	35
3.6	Tổng quan về bài toán tìm đường tối ưu	41
3.6.1	Hướng tiếp cận cho bài toán tìm đường tối ưu	41
3.6.2	Tổng quan về các giải thuật định tuyến dựa trên cấu trúc phân hoạch	42
3.7	Giải thuật tìm đường Dijkstra	42
3.7.1	Nguyên lý hoạt động	43
3.7.2	Mã giả thuật toán	43
3.7.3	Đánh giá độ phức tạp và Hạn chế	44
3.8	Giải thuật Contraction Hierarchies (CH)	45
3.8.1	Giai đoạn tiền xử lý: Co đỉnh và Thêm cạnh tắt	45
3.8.2	Giai đoạn truy vấn: Tìm kiếm hai chiều trên đồ thị phân cấp	46
3.8.3	Đánh giá và Hạn chế trên môi trường thực tế	47
3.9	Kiến trúc định tuyến Customizable Route Planning (CRP)	47
3.9.1	Giai đoạn 1: Tiền xử lý cấu trúc	48
3.9.2	Giai đoạn 2: Tùy chỉnh trọng số	48
3.9.3	Giai đoạn 3: Truy vấn lộ trình	49
4	Mô hình đề xuất	51
4.1	Mô-đun trích xuất đặc trưng đoạn đường (Segment Encoder)	53
4.2	Mô-đun điều chế đặc trưng theo thời gian bắt đầu (FiLM)	54
4.2.1	Mã hóa thời gian bắt đầu	54
4.2.2	Bộ mã hóa FiLM	56
4.3	Mô-đun học tương phản (Contrastive Encoder)	57
4.3.1	Tăng cường dữ liệu	58
4.3.2	Học tương phản	59
4.4	Mô-đun dự đoán thời gian di chuyển	61
4.4.1	Mã hóa chuỗi theo bước đoạn đường	62

4.4.2	Gộp chú ý (Attention Pooling)	62
4.4.3	Hồi quy dự đoán	62
4.5	Giải thuật tìm đường Multilevel Dijkstra	63
5	Chuẩn bị dữ liệu	67
5.1	Thu thập dữ liệu di chuyển tại khu vực TP. Hồ Chí Minh	67
5.1.1	Cơ sở lựa chọn phương pháp thu thập dữ liệu	67
5.1.2	Quy trình thu thập dữ liệu	68
5.1.3	Dữ liệu thô thu thập	71
5.1.4	Tiền xử lý dữ liệu	72
5.1.5	Khảo sát chất lượng dữ liệu	73
5.1.6	Xây dựng tập dataset	74
5.2	Trích xuất đặc trưng	75
5.2.1	Đặc trưng hình học	75
5.2.2	Đặc trưng thuộc tính đường	76
5.2.3	Đặc trưng ngữ cảnh không gian từ PoI	77
5.3	Đặc trưng đầu vào	78
6	Luồng huấn luyện	79
6.1	Hàm mất mát biểu diễn	79
6.2	Hàm mất mát hồi quy	79
6.3	Cơ chế tối ưu hóa đa mục tiêu	80
6.4	Chiến lược điều chỉnh tốc độ học	80
6.5	Các kỹ thuật ổn định gradient và hiệu năng	81
6.6	Thuật toán huấn luyện tổng quát	81
6.7	Các công thức đánh giá (Evaluation Metrics)	83
6.7.1	Root Mean Squared Error (RMSE)	83
6.7.2	Mean Absolute Error (MAE)	83
6.7.3	Mean Absolute Percentage Error (MAPE)	83
7	Kết quả thực nghiệm	85
7.1	Môi trường thực nghiệm	85
7.2	Tập dữ liệu kiểm nghiệm	85
7.3	Các mô hình thử nghiệm	86
7.4	Kết quả thực nghiệm	86
7.5	So sánh giải thuật định tuyến	88
7.5.1	Môi trường thực nghiệm và thiết lập	88
7.5.2	Đánh giá thời gian và không gian tiền xử lý	88
7.5.3	Đánh giá thời gian truy vấn và Kết luận chung	89

8	Triển khai ứng dụng	91
8.1	Phân tích yêu cầu	91
8.1.1	Tổng quan về ứng dụng	91
8.1.2	Yêu cầu chức năng và phi chức năng	91
8.2	Kiến trúc và Lựa chọn công nghệ	93
8.3	Thiết kế hệ thống	93
8.3.1	Sơ đồ Use Case	93
8.3.2	Sơ đồ lớp (Class Diagram)	95
8.3.3	Sơ đồ hoạt động (Activity Diagram)	97
8.3.4	Sơ đồ tuần tự (Sequence Diagram)	102
8.3.5	Sơ đồ triển khai (Deployment Diagram)	103
8.4	Hiện thực hệ thống	104
8.4.1	Giao diện tổng quan và Tương tác bản đồ	104
8.4.2	Xác định lộ trình thông qua GPS	105
8.4.3	Kết quả dự báo và Phân tích sai số	105
8.5	Kiểm thử hệ thống	106
8.5.1	Kiểm thử chức năng (Functional Testing)	106
8.5.2	Kiểm thử hiệu năng và Độ tin cậy	107
9	Tổng kết	109
	Tài liệu tham khảo	110

Danh sách hình ảnh

1	Kiến trúc Mạng nơ-ron hồi quy [1]	18
2	Kiến trúc GRU [2, 3]	20
3	Kiến trúc Bert [4, 5]	21
4	Kiến trúc mạng tích chập [1]	23
5	Ví dụ của mạng lưới đường bộ [6, 7]	26
6	Cấu trúc của mô hình DeepTTE [6]	32
7	Quy trình của thuật toán Fast Map Matching gồm hai giai đoạn: Tính toán trước (Precomputing) và So khớp (Map matching).[8] . .	35
8	Kiến trúc tổng thể của mô hình MulT-TTE với ba module trích xuất đặc trưng chính: Thuộc tính, Không gian và Ngữ nghĩa nhận thức thời gian. [9]	37
9	Quá trình nén dữ liệu từ đồ thị gốc sang đồ thị lớp phủ	49
10	Kiến trúc tổng quan của mô hình PoI-CL-TTE. Các thành phần được tô màu xanh bao gồm chiến lược huấn luyện, bộ mã hóa đoạn đường, mô-đun FiLM và bộ mã hóa tương phản là các thành phần đóng góp chính của nghiên cứu.	52
11	Biến động thời gian di chuyển trung bình theo giờ trong ngày (trái), ngày trong tuần (giữa), và ngày trong năm (phải) trên tập dữ liệu huấn luyện.	55
12	Kiến trúc Bộ mã hóa FiLM (trái) và kiến trúc FiLM (phải) [10] . .	56
13	Minh họa giải thuật MLD: Quá trình duyệt trên đồ thị tìm kiếm $\mathcal{G}^*$ [11]	64
14	Thiết kế công cụ thu thập dữ liệu	69
15	Phạm vi khu vực TP. Hồ Chí Minh được sử dụng để thu thập dữ liệu	70
16	Mẫu dữ liệu thô thu thập được lưu trữ dưới dạng bảng (CSV) . . .	71
17	Phân bố quãng đường di chuyển	73
18	Mật độ dữ liệu theo Giờ và Quãng đường	74
19	Sơ đồ Use Case của hệ thống.	94
20	Sơ đồ lớp (Class Diagram) mô tả cấu trúc dữ liệu và cơ chế Workpool của hệ thống.	96
21	Sơ đồ hoạt động của API Gateway (Phần 1: Xác thực dữ liệu đầu vào).	99
22	Sơ đồ hoạt động của API Gateway (Phần 2: Cơ chế định tuyến OSRM).	100
23	Sơ đồ hoạt động của API Gateway (Phần 3: Khớp nối bản đồ và Suy luận song song).	101



24	Sơ đồ tuần tự minh họa luồng xử lý dự báo thời gian di chuyển. . .	102
25	Sơ đồ triển khai hệ thống dự báo thời gian di chuyển.	103
26	Giao diện bản đồ tổng quan của ứng dụng.	105
27	Giao diện chọn điểm đi và điểm đến bằng tương tác chuột.	105
28	Hiển thị kết quả dự báo và các chỉ số sai số MAE, MAPE.	106



Danh sách bảng

1	So sánh đặc tính và độ phức tạp của các giải thuật định tuyến . . .	66
2	So sánh hiệu năng giữa mô hình đề xuất và các baseline trên tập kiểm thử của Porto và TP.HCM.	87
3	So sánh thời gian và không gian tiền xử lý giữa giải thuật CH và MLD	88
4	Thời gian truy vấn trung bình các giải thuật (mili giây).	89
5	Đặc tả các Use Case của hệ thống.	95
6	Các trường hợp kiểm thử chức năng chính.	107
7	Đánh giá và so sánh hiệu năng tiêu thụ tài nguyên của các mô hình	107
8	Kết quả kiểm thử chịu đựng của hệ thống Backend	108

1

Giới thiệu

Chương này giới thiệu tổng quan về đề tài. Nội dung bao gồm bối cảnh thực hiện đề tài, mục tiêu nghiên cứu, phạm vi nghiên cứu, đối tượng nghiên cứu.

1.1 Giới thiệu

Hệ thống Giao thông Thông minh (Intelligent Transportation Systems - ITS) hiện đại được xây dựng dựa trên hai bài toán nền tảng: Ước lượng thời gian đến đích (Estimated Time of Arrival - ETA) và Tìm đường tối ưu.

Về mặt định nghĩa, Ước lượng thời gian đến đích (ETA) là quá trình dự báo chính xác khoảng thời gian cần thiết để một phương tiện di chuyển từ điểm xuất phát đến điểm đích thông qua một lộ trình cụ thể. Bài toán này đòi hỏi việc mô hình hóa phức tạp các yếu tố không gian như cấu trúc mạng lưới đường bộ, các đặc trưng ngữ nghĩa của khu vực xung quanh và các yếu tố thời gian động như mật độ phương tiện hay tình trạng ùn tắc tại thời điểm khởi hành. Mục tiêu cốt lõi là xây dựng một hàm ánh xạ từ các dữ liệu đầu vào thành một giá trị thời gian di chuyển sát với thực tế nhất.

Trong khi đó, Tìm đường tối ưu được định nghĩa là bài toán xác định một lộ trình, bao gồm một chuỗi các đoạn đường kết nối nhau giữa hai điểm nút trên đồ thị mạng lưới giao thông, sao cho cực tiểu hóa một hàm chi phí xác định. Hàm chi phí này có thể được thiết lập dựa trên nhiều tiêu chí khác nhau như khoảng cách vật lý ngắn nhất, mức tiêu thụ nhiên liệu thấp nhất hoặc phổ biến nhất là thời gian di chuyển nhanh nhất. Sự kết hợp đồng bộ giữa khả năng định hướng lộ trình và khả năng dự báo thời gian chính xác là yếu tố quyết định đến hiệu quả của các ứng dụng bản đồ, dịch vụ vận tải và logistics hiện nay.

Tuy nhiên, khi triển khai trên quy mô đô thị lớn, bài toán ETA đối mặt với những rào cản kỹ thuật cực kỳ phức tạp. Các phương pháp ước lượng truyền thống dựa trên vận tốc trung bình thường bộc lộ sai số lớn do không thể nắm bắt được tính bất định của dòng chảy giao thông và bỏ qua sự chi phối từ ngữ cảnh không gian xung quanh. Song song đó, yêu cầu về tốc độ phản hồi thời gian thực trên các bản đồ quy mô lớn cũng đặt ra rào cản lớn đối với các thuật toán tìm đường cơ bản, khi chúng phải duyệt qua một không gian tìm kiếm khổng lồ dẫn đến tiêu tốn tài nguyên và thời gian.

Để giải quyết vấn đề của bài toán ETA, các nghiên cứu hiện đại đã chuyển dịch mạnh mẽ sang kỹ thuật Học sâu. Các mô hình tiêu biểu như DeepTTE [6] hay các mô hình dựa trên đồ thị (GCN) đã chứng minh hiệu quả rõ rệt trong việc

nắm bắt các phụ thuộc không gian - thời gian. Đặc biệt, xu hướng gần đây là sự ra đời của các kiến trúc đa phương thức như MulT-TTE [9], cho phép tổng hợp thông tin từ nhiều nguồn dữ liệu để nâng cao đáng kể độ tin cậy của dự báo. Mặc dù vậy, việc áp dụng các mô hình tiên tiến này vào những đô thị lớn với điều kiện giao thông phức tạp và linh hoạt vẫn gặp nhiều thách thức. Phần lớn các kiến trúc này vẫn tập trung xử lý dữ liệu quỹ đạo GPS thuần túy, chưa khai thác triệt để các thông tin ngữ nghĩa từ môi trường, khiến chúng khó đạt hiệu suất tối ưu khi phân phối giao thông thay đổi. Nhằm khắc phục điểm nghẽn này, việc tích hợp đặc trưng từ các Điểm quan tâm (PoI) kết hợp cùng cơ chế Học tương phản đang mở ra một hướng đi mới, giúp mô hình hiểu sâu hơn về áp lực giao thông tiềm ẩn tại từng khu vực, từ đó tăng cường khả năng tổng quát hóa. Bên cạnh đó, để giải quyết nút thắt về hiệu năng của bài toán tìm đường, giải thuật Dijkstra đa cấp (Multilevel Dijkstra - MLD) [11] nổi lên như một giải pháp xuất sắc nhờ khả năng nén dữ liệu và phân tách cấu trúc đồ thị vượt trội.

Dựa trên thực tế đó, đồ án này đặt trọng tâm vào việc nghiên cứu chuyên sâu và đề xuất mô hình học sâu PoI-CL-TTE nhằm giải quyết bài toán dự báo thời gian di chuyển thông qua các biểu diễn ngữ nghĩa mạng lưới. Cùng với đó, để hoàn thiện vòng lặp hệ thống, đồ án tiến hành khảo sát và tích hợp giải thuật MLD nhằm xử lý tối ưu tác vụ tìm đường. Mục tiêu cuối cùng là triển khai đồng bộ cả mô hình dự báo PoI-CL-TTE và giải thuật MLD vào một ứng dụng bản đồ giao thông trực tuyến do nhóm tác giả tự phát triển, mang lại một kiến trúc dẫn đường toàn diện, chính xác và có khả năng tái sử dụng linh hoạt trên mạng lưới giao thông của nhiều thành phố lớn trên thế giới.

1.2 Mục tiêu nghiên cứu

Mục tiêu của đồ án là nghiên cứu và xây dựng giải pháp đồng bộ cho bài toán ước lượng thời gian đến đích và tìm đường hiệu quả, đáp ứng yêu cầu về độ chính xác và tốc độ phản hồi trong thực tế. Để đạt được điều này, đồ án tập trung khảo sát các mô hình học sâu hiện đại và nghiên cứu đề xuất một giải pháp dự báo tiên tiến nhằm nâng cao khả năng khai thác dữ liệu ngữ nghĩa và độ bền vững của mô hình trên các mạng lưới giao thông khác nhau. Đồng thời, đồ án tiến hành khảo sát và ứng dụng giải thuật tối ưu hóa lộ trình nhằm đảm bảo hiệu năng truy vấn và khả năng tiết kiệm tài nguyên trên các bản đồ quy mô lớn. Kết quả nghiên cứu sẽ được hiện thực hóa thông qua việc xây dựng một ứng dụng bản đồ giao thông trực tuyến, cho phép tích hợp và kiểm chứng các giải pháp đã đề xuất trong môi trường thực tế.

1.3 Phạm vi nghiên cứu

Đồ án này tập trung vào việc nghiên cứu và phát triển các giải pháp ước lượng thời gian đến đích và tìm đường có khả năng ứng dụng linh hoạt trên mạng lưới giao thông của các thành phố lớn. Nghiên cứu được tiến hành trong các giới hạn sau:

- **Phạm vi dữ liệu và không gian:** Nghiên cứu được thực hiện, huấn luyện và đánh giá trên các tập dữ liệu giao thông quy mô lớn trên thế giới nhằm đảm bảo tính khách quan, tính đa dạng và có thể đối chiếu trực tiếp với các công bố khoa học hiện hành.
- **Phạm vi mô hình và thuật toán:** Bài toán dự báo thời gian tập trung vào các kiến trúc Học sâu có khả năng xử lý dữ liệu không gian - thời gian, đồng thời khảo sát và đề xuất các kỹ thuật học biểu diễn tiên tiến nhằm khai thác hiệu quả thông tin ngữ nghĩa từ môi trường và tăng cường độ bền vững của mô hình. Đối với bài toán tìm đường, nghiên cứu giới hạn ở việc khảo sát và ứng dụng giải thuật định tuyến hiện đại trên đồ thị mạng lưới quy mô lớn.
- **Các yếu tố ảnh hưởng:** Đề tài khai thác và tích hợp các thông tin cốt lõi cấp đoạn đường bao gồm thuộc tính vật lý (độ dài, loại đường), cấu trúc đồ thị mạng lưới, thông tin thời gian khởi hành và thông tin ngữ nghĩa từ các điểm quan tâm lân cận. Các yếu tố mang tính vi mô và ngẫu nhiên như hành vi của cá nhân người lái xe hay sự cố kỹ thuật phương tiện nằm ngoài phạm vi xem xét của nghiên cứu.
- **Môi trường triển khai:** Các mô hình và giải thuật đề xuất được thiết kế, tối ưu hóa nhằm mục tiêu tích hợp đồng bộ vào một hệ thống bản đồ giao thông trực tuyến do nhóm tác giả tự phát triển, phục vụ nhu cầu dẫn đường và cung cấp ước lượng giao thông trong thực tế.

1.4 Đối tượng nghiên cứu

Dựa trên phạm vi và mục tiêu đề ra, luận văn tập trung nghiên cứu sâu vào các đối tượng chính sau:

- Khảo sát cơ chế hoạt động và phân tích hiệu quả của các mô hình hiện đại trong việc nắm bắt các phụ thuộc phức tạp của mạng lưới giao thông. Đối tượng nghiên cứu bao gồm các kiến trúc dựa trên cơ chế attention và các mạng học tuần tự, hướng tới việc chuyển dịch từ biểu diễn tọa độ hình học thuần túy sang các biểu diễn giàu ngữ nghĩa nhằm nâng cao chất lượng dự báo.

- Phân tích các kỹ thuật trích xuất và kết hợp thông tin từ nhiều nguồn dữ liệu khác nhau như thuộc tính vật lý của mạng lưới đường bộ, đặc trưng thời gian và thông tin ngữ cảnh xung quanh. Nghiên cứu cách thức các yếu tố này tác động đến mô hình nhằm xây dựng các không gian biểu diễn đặc trưng bền vững và có khả năng tổng quát hóa cao trên nhiều khu vực khác nhau.
- Nghiên cứu và khảo sát các thuật toán định tuyến tiên tiến có khả năng xử lý hiệu quả trên mạng lưới đường bộ thực tế. Trọng tâm đặt vào các phương pháp phân hoạch đồ thị và kỹ thuật tối ưu hóa không gian tìm kiếm nhằm đảm bảo sự cân bằng giữa độ chính xác của lộ trình và tốc độ phản hồi của hệ thống trong môi trường thời gian thực.
- Nghiên cứu các phương pháp thực nghiệm để đánh giá hiệu quả của các giải pháp đề xuất trên nguồn dữ liệu thực tế. Đồng thời, nghiên cứu kiến trúc tích hợp đồng bộ giữa mô hình dự báo và giải thuật tìm đường vào một ứng dụng bản đồ giao thông trực tuyến để kiểm chứng khả năng ứng dụng thực tiễn của đề tài.

2

Cơ sở lý thuyết

2.1 Học giám sát

Ngành học máy đã chứng kiến sự tăng trưởng mạnh mẽ với sự ra đời của học có giám sát, một hình thức học cho phép máy tính đưa ra những dự đoán hay lựa chọn dựa trên dữ liệu lịch sử [1]. Phương thức học này được áp dụng rộng rãi trong nhiều lĩnh vực khác nhau như nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên, dự báo chuỗi thời gian và các hệ thống khuyến nghị.

Phương pháp cơ bản của học máy có giám sát xoay quanh ba giai đoạn chính: huấn luyện, kiểm định và kiểm tra [1].

Trong giai đoạn huấn luyện, mô hình được cung cấp dữ liệu có nhãn, trong đó các đặc trưng liên quan được trích xuất và một thuật toán học được sử dụng để xây dựng mô hình dự đoán. Giai đoạn này cho phép mô hình học được các mẫu và mối quan hệ tiềm ẩn trong dữ liệu.

Tiếp theo, giai đoạn kiểm định được sử dụng để đánh giá mô hình trong quá trình huấn luyện, điều chỉnh các siêu tham số và giảm thiểu hiện tượng overfitting.

Cuối cùng, trong giai đoạn kiểm tra, mô hình đã được huấn luyện sẽ được sử dụng để đưa ra dự đoán trên dữ liệu mới chưa từng xuất hiện trước đó. Các kết quả dự đoán này phản ánh đầu ra cuối cùng của mô hình và cho phép đánh giá khả năng tổng quát hóa của phương pháp học có giám sát.

Ví dụ, mô hình học giám sát khi được áp dụng cho bài toán dự đoán thời gian di chuyển, các mô hình học giám sát có thể được thiết kế theo hướng hồi quy nhằm dự đoán giá trị số liên tục, từ đó cung cấp các ước lượng chính xác và có giá trị ứng dụng cao cho các hệ thống giao thông thông minh.

2.2 Học tự giám sát

Học tự giám sát (Self-supervised Learning) là một hướng tiếp cận trong đó tín hiệu giám sát được tự động sinh ra từ chính dữ liệu thô, mà không cần nhãn do con người cung cấp [1]. Mô hình được huấn luyện trên các nhiệm vụ phụ trợ (pretext tasks) được thiết kế để buộc mô hình học các biểu diễn có ý nghĩa từ cấu trúc nội tại của dữ liệu.

Các chiến lược học tự giám sát phổ biến bao gồm:

- **Mô hình ngôn ngữ có che** (Masked Language Modelling): một phần các phần tử trong chuỗi đầu vào bị che ngẫu nhiên, và mô hình phải dự đoán lại

các giá trị bị che dựa trên ngữ cảnh xung quanh. Chiến lược này được sử dụng trong BERT và các biến thể của nó [4].

- **Học tương phản** (Contrastive Learning): mô hình được huấn luyện để kéo gần biểu diễn của các mẫu tương đồng (cặp dương) và đẩy xa biểu diễn của các mẫu khác biệt (cặp âm) trong không gian véc-tơ. Chi tiết về học tương phản và hàm mất mát InfoNCE được trình bày tại Mục 2.12.

Học tự giám sát đặc biệt hiệu quả trong các tình huống dữ liệu có nhãn khan hiếm, khi mô hình có thể được tiền huấn luyện trên lượng lớn dữ liệu không nhãn trước khi tinh chỉnh (fine-tuning) trên tập nhãn nhỏ.

2.3 Học tương phản (Contrastive Learning)

Học tương phản (Contrastive Learning) là một kỹ thuật trong học tự giám sát (self-supervised learning), tập trung vào việc học các biểu diễn đặc trưng bằng cách so sánh các cặp dữ liệu [12]. Mục tiêu cốt lõi là tối ưu hóa không gian nhúng sao cho các mẫu tương đồng (*positive pairs*) nằm gần nhau và các mẫu khác biệt (*negative pairs*) bị đẩy ra xa.

Một mô hình học tương phản điển hình gồm ba thành phần chính:

Bộ mã hóa (Encoder): Một mạng nơ-ron $f(\cdot)$ (như ResNet, Transformer hoặc GRU) thực hiện ánh xạ dữ liệu đầu vào x sang vector biểu diễn $h = f(x)$.

Đầu chiếu (Projection Head): Một mạng MLP $g(\cdot)$ ánh xạ h sang không gian $z = g(h)$, nơi hàm mất mát được tính toán. Việc này giúp loại bỏ các thông tin nhiễu không cần thiết cho việc phân biệt.

Hàm mất mát tương phản: Phổ biến nhất là hàm **InfoNCE** (Information Noise-Contrastive Estimation), được định nghĩa dựa trên độ tương đồng Cosine.

2.4 Mạng nơ-ron nhiều lớp (Multi-Layer Perceptron)

Mạng nơ-ron nhiều lớp là một trong những kiến trúc mạng nơ-ron nhân tạo cơ bản nhất, thuộc nhóm mạng truyền thẳng (feedforward network), trong đó thông tin lan truyền theo một chiều từ lớp đầu vào đến lớp đầu ra thông qua một hoặc nhiều lớp ẩn [1].

Mỗi nơ-ron trong một lớp được kết nối đầy đủ với tất cả nơ-ron của lớp tiếp theo thông qua các trọng số có thể học. Đầu ra của một nơ-ron được tính bằng cách áp dụng hàm kích hoạt phi tuyến lên tổ hợp tuyến tính có trọng số của các đầu vào:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad (1)$$

trong đó $\mathbf{W}^{(l)}$ và $\mathbf{b}^{(l)}$ lần lượt là ma trận trọng số và véc-tơ độ chệch của lớp thứ l , và $\sigma(\cdot)$ là hàm kích hoạt (ReLU, GELU, sigmoid, v.v.). Nhờ các hàm kích hoạt phi tuyến, MLP có khả năng xấp xỉ bất kỳ hàm liên tục nào với độ chính xác tùy ý (định lý xấp xỉ toàn cục) [1].

2.5 Cơ chế tự chú ý (Self-Attention)

Cơ chế tự chú ý cho phép mỗi phần tử trong một chuỗi trực tiếp tương tác với tất cả các phần tử còn lại, qua đó nắm bắt các phụ thuộc dài hạn mà các mô hình tuần tự truyền thống gặp khó khăn [13].

Với chuỗi đầu vào $X = [\mathbf{x}_1, \dots, \mathbf{x}_n] \in \mathbb{R}^{n \times d}$, mỗi phần tử được chiếu tuyến tính thành ba véc-tơ truy vấn, khóa và giá trị:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (2)$$

trong đó W^Q, W^K, W^V là các ma trận trọng số có thể học. Đầu ra của cơ chế tự chú ý được tính bằng:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (3)$$

trong đó d_k là số chiều của véc-tơ khóa, đóng vai trò hệ số tỉ lệ nhằm ổn định gradient khi d_k lớn.

2.6 Cơ chế tự chú ý đa đầu (Multi-head Self-attention)

Cơ chế tự chú ý đa đầu mở rộng tự chú ý đơn lẻ bằng cách thực hiện đồng thời h phép chú ý độc lập trên các không gian con khác nhau, cho phép mô hình nắm bắt đồng thời nhiều loại quan hệ phụ thuộc giữa các phần tử [13].

Mỗi đầu i sử dụng bộ trọng số riêng $\{W_i^Q, W_i^K, W_i^V\}$ để chiếu đầu vào vào một không gian con có số chiều thấp hơn, thực hiện phép chú ý độc lập, rồi các kết quả được nối ghép và chiếu lại:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (4)$$

trong đó $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ và W^O là ma trận trọng số đầu ra. So với tự chú ý đơn đầu, cơ chế đa đầu tránh được hiện tượng biểu diễn của một phần tử bị chi phối quá mức bởi một mối quan hệ duy nhất.

2.7 Bộ mã hóa Transformer (Transformer Encoder)

Bộ mã hóa Transformer là kiến trúc xếp chồng nhiều lớp, mỗi lớp gồm hai thành phần chính được kết nối qua cơ chế kết nối tắt (residual connection) và chuẩn hóa lớp (Layer Normalization) [13]:

- **Tự chú ý đa đầu:** nắm bắt các phụ thuộc giữa các vị trí trong chuỗi.
- **Mạng truyền thẳng theo vị trí** (Position-wise FFN): áp dụng độc lập tại mỗi vị trí, tăng khả năng biểu diễn phi tuyến:

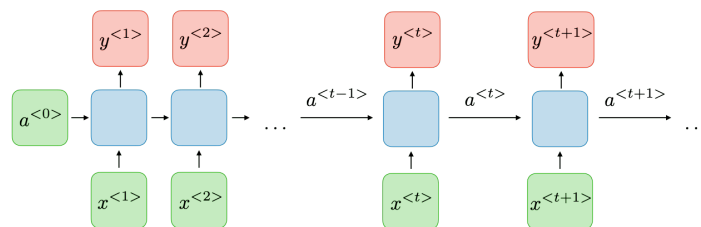
$$\text{FFN}(\mathbf{x}) = \sigma(\mathbf{x}W_1 + b_1)W_2 + b_2 \quad (5)$$

Đầu ra của mỗi thành phần con tuân theo công thức:

$$\mathbf{x}' = \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x})) \quad (6)$$

Kết nối tắt giúp duy trì dòng chảy gradient qua nhiều lớp, trong khi chuẩn hóa lớp ổn định phân phối của các kích hoạt trung gian. Do không có cấu trúc hồi quy, Transformer Encoder xử lý toàn bộ chuỗi song song, mang lại hiệu quả tính toán cao hơn so với các mô hình tuần tự.

2.8 Mạng nơ-ron hồi quy (Recurrent Neural Network)



Hình 1: Kiến trúc Mạng nơ-ron hồi quy [1]

Mạng nơ-ron hồi quy (Recurrent Neural Network - RNN) là một lớp mô hình mạng nơ-ron được thiết kế chuyên biệt để xử lý dữ liệu dạng chuỗi [1]. RNN cho phép mô hình duy trì một hidden state nhằm lưu trữ thông tin từ các bước thời gian trước đó, qua đó khai thác được và biểu diễn được các mối quan hệ phụ thuộc trong dữ liệu dạng chuỗi.

Từ Hình 1, ta có thể thấy ở cốt lõi của mô hình RNN là lớp hồi quy trạng thái ẩn, trong đó trạng thái ẩn tại mỗi bước thời gian không chỉ phụ thuộc vào dữ liệu đầu vào hiện tại mà còn kế thừa thông tin từ trạng thái ẩn ở bước thời gian

trước đó. Cấu trúc hồi quy này cho phép mô hình nắm bắt được sự thay đổi động của hệ thống giao thông theo thời gian, yếu tố đóng vai trò quan trọng trong việc dự đoán chính xác thời gian di chuyển.

Trạng thái ẩn ở một bước thời gian bất kỳ, gọi là h_t , được tính dựa trên trạng thái ẩn ở bước thời gian trước đó, h_{t-1} , đầu vào hiện tại x_t cùng với các ma trận trọng số và độ chệch tương ứng. Quá trình cập nhật hồi quy này giúp mô hình duy trì thông tin lịch sử và tích hợp nó vào các dự đoán hiện tại, theo công thức:

$$h_t = g_1(W_{hh}h_{t-1} + W_{hx}x_t + b_h) \quad (7)$$

trong đó W_{hh} , W_{hx} lần lượt là các ma trận trọng số kết nối giữa trạng thái ẩn - trạng thái ẩn và đầu vào - trạng thái ẩn; b_h là vectơ độ chệch; $g_1(.)$ là hàm kích hoạt.

Đầu ra của RNN tại thời điểm t , ký hiệu là y_t , được suy ra thông qua một phép biến đổi tuyến tính từ trạng thái ẩn, và có thể được kết hợp thêm một hàm kích hoạt phù hợp với từng bài toán cụ thể:

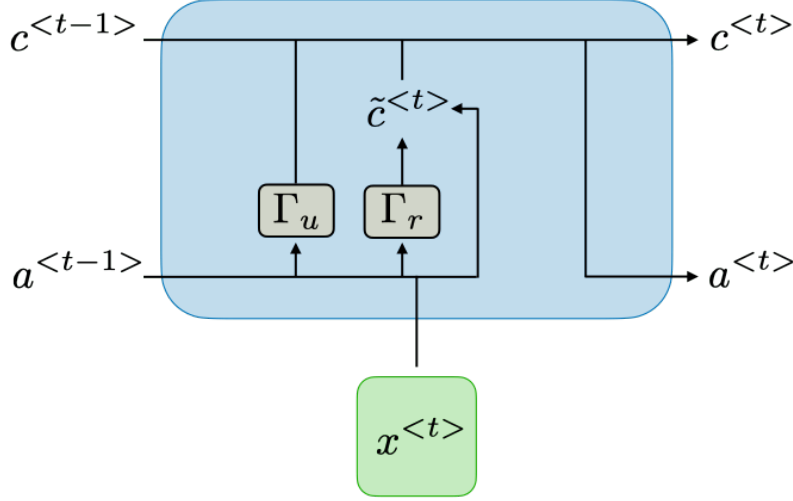
$$y_t = g_2(W_{hy}h_t + b_y) \quad (8)$$

trong đó, W_{hy} là ma trận trọng số kết nối giữa trạng thái ẩn - đầu ra; b_y là vectơ độ chệch, và $g_2(.)$ là hàm kích hoạt đầu ra.

Nhờ khả năng mô hình hóa dữ liệu theo chuỗi, RNN hỗ trợ xây dựng các mô hình dự đoán có thể nắm bắt hiệu quả sự phụ thuộc theo thời gian trong mạng lưới giao thông, từ đó nâng cao độ chính xác và tin cậy của bài toán ước lượng thời gian di chuyển.

Tuy nhiên, RNN cơ bản vẫn tồn tại một số hạn chế đáng kể khi xử lý các chuỗi dữ liệu dài. Cụ thể, trong quá trình huấn luyện bằng thuật toán lan truyền ngược theo thời gian, RNN thường gặp phải hiện tượng tiêu biến hoặc bùng nổ gradient, khiến mô hình khó học được các mối quan hệ phụ thuộc dài hạn. Điều này làm suy giảm hiệu quả dự đoán trong các kịch bản giao thông phức tạp, nơi mà trạng thái hiện tại có thể chịu ảnh hưởng mạnh từ các sự kiện xảy ra trong quá khứ xa.

2.9 Gated Recurrent Unit



Hình 2: Kiến trúc GRU [2, 3]

Đơn vị Hồi quy Có Cổng (Gated Recurrent Unit - GRU) là một biến thể cải tiến của Mạng nơ-ron hồi quy được đề xuất nhằm khắc phục các hạn chế của RNN cơ bản, đặc biệt là hiện tượng tiêu biến gradient khi xử lý các chuỗi dữ liệu dài [2]. GRU vẫn giữ được khả năng mô hình hóa dữ liệu chuỗi theo thời gian, đồng thời sử dụng các cơ chế cổng (gates) để kiểm soát dòng chảy thông tin, giúp mô hình học hiệu quả hơn các mối quan hệ phụ thuộc dài hạn. Cấu trúc chi tiết của GRU được mô tả tại hình 2.

Khác với RNN truyền thống, GRU không sử dụng một trạng thái bộ nhớ riêng biệt mà tích hợp trực tiếp khả năng ghi nhớ vào trạng thái ẩn thông qua hai cổng chính: cổng cập nhật (update gate) và cổng đặt lại (reset gate) [2]. Các cổng này cho phép mô hình quyết định lượng thông tin trong quá khứ cần được giữ lại hoặc loại bỏ tại mỗi bước thời gian.

Cụ thể, với đầu vào tại thời điểm t là x_t và trạng thái ẩn ở bước thời gian trước đó là h_{t-1} , cổng cập nhật z_t và cổng đặt lại r_t được xác định như sau:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad (9)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \quad (10)$$

trong đó W_z, W_r và U_z, U_r là các ma trận trọng số có thể học được, b_z, b_r là các vectơ độ chệch, và $\sigma(\cdot)$ là hàm sigmoid.

Trạng thái ẩn ứng viên $\tilde{h}_t$ được tính bằng cách kết hợp đầu vào hiện tại với trạng thái ẩn trước đó đã được điều chỉnh bởi cổng đặt lại:

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h), \quad (11)$$

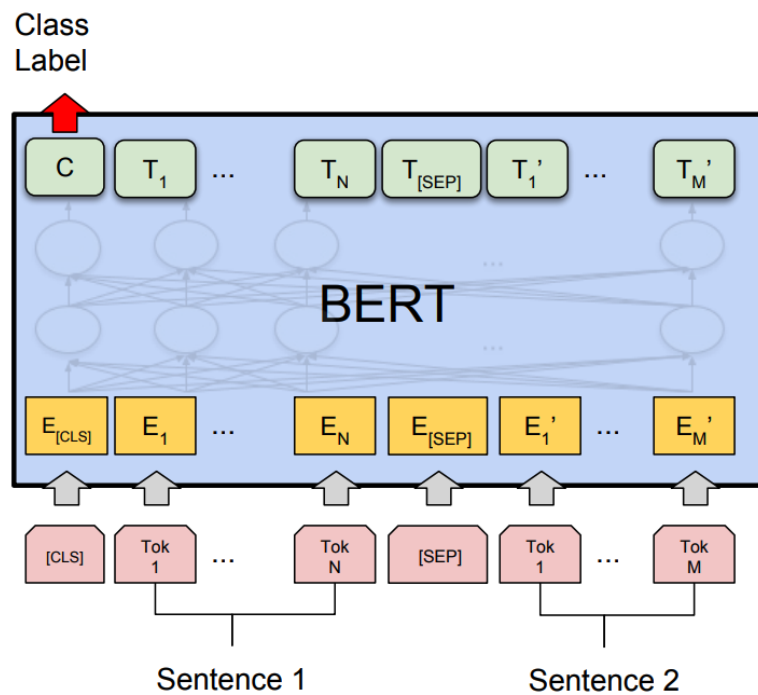
trong đó $\odot$ biểu thị phép nhân theo từng phần tử.

Cuối cùng, trạng thái ẩn tại thời điểm t được cập nhật bằng cách kết hợp trạng thái ẩn cũ và trạng thái ẩn ứng viên thông qua cổng cập nhật:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (12)$$

Nhờ cơ chế cổng cập nhật, GRU cho phép mô hình duy trì các thông tin quan trọng từ quá khứ trong khi vẫn loại bỏ được các thông tin không cần thiết. So với Long Short-Term Memory (LSTM), GRU có cấu trúc đơn giản hơn và số lượng tham số ít hơn, từ đó giúp giảm chi phí tính toán và tăng tốc độ hội tụ trong quá trình huấn luyện.

2.10 BERT



Hình 3: Kiến trúc Bert [4, 5]

Biểu diễn Mã hóa Hai Chiều từ Transformers (Bidirectional Encoder Representations from Transformers - BERT) là một mô hình ngôn ngữ tiền huấn luyện dựa trên kiến trúc Transformer, được thiết kế nhằm học các biểu diễn ngữ nghĩa giàu ngữ cảnh từ dữ liệu chuỗi. Kiến trúc của mô hình BERT được thể hiện trong Hình 3. Khác với các mô hình ngôn ngữ truyền thống chỉ khai thác ngữ cảnh theo

một chiều, BERT cho phép mô hình hóa đồng thời cả ngữ cảnh trái và phải của một phần tử trong chuỗi thông qua cơ chế tự chú ý hai chiều [4].

Về mặt kiến trúc, BERT bao gồm nhiều lớp mã hóa Transformer xếp chồng lên nhau. Mỗi lớp mã hóa gồm hai thành phần chính: cơ chế tự chú ý đa đầu và mạng truyền thẳng theo vị trí, kết hợp với các kỹ thuật chuẩn hóa và kết nối tắt.

Giả sử đầu vào của mô hình là một chuỗi đã được ánh xạ thành các vectơ biểu diễn nhúng:

$$X = [x_1, x_2, \dots, x_n],$$

trong đó mỗi x_i là tổng của biểu diễn nhúng từ vựng, biểu diễn nhúng vị trí và biểu diễn nhúng phân đoạn. Đối với mỗi lớp mã hóa, cơ chế tự chú ý được tính theo công thức:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (13)$$

với $Q = XW^Q$, $K = XW^K$, $V = XW^V$, và W^Q , W^K , W^V là các ma trận trọng số có thể học được.

Trong kiến trúc tự chú ý nhiều đầu, cơ chế chú ý được thực hiện song song trên nhiều không gian con khác nhau:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad (14)$$

trong đó mỗi head_i được tính bằng một phép chú ý độc lập, và W^O là ma trận trọng số đầu ra.

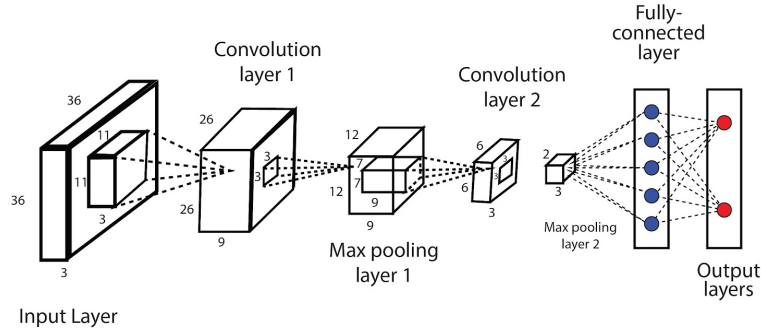
Đầu ra của tự chú ý sau đó được truyền qua mạng truyền thẳng theo vị trí:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2, \quad (15)$$

giúp tăng khả năng biểu diễn phi tuyến của mô hình.

BERT được huấn luyện trước trên tập dữ liệu lớn thông qua các nhiệm vụ tự giám sát, tiêu biểu là mô hình ngôn ngữ bị che (Masked Language Modelling - MLM), trong đó một phần các phần tử trong chuỗi bị che (mask) và mô hình phải dự đoán lại giá trị ban đầu, cùng với nhiệm vụ Dự đoán câu tiếp theo (Next Sentence Prediction - NSP) nhằm học mối quan hệ giữa các chuỗi. Quá trình tiền huấn luyện này giúp BERT học được các đặc trưng ngữ nghĩa và ngữ cảnh tổng quát, có khả năng chuyển giao tốt sang các bài toán hạ nguồn.

2.11 Mạng tích chập



Hình 4: Kiến trúc mạng tích chập [1]

Mạng tích chập (Convolution Neural Network - CNN) là một kiến trúc mạng nơ-ron đặc biệt, được thiết kế để xử lý dữ liệu có cấu trúc lưới, chẳng hạn như hình ảnh hoặc ma trận đặc trưng [1]. Điểm nổi bật của CNN là khả năng học các bộ lọc có thể chia sẻ trên toàn bộ đầu vào, giúp trích xuất các đặc trưng cục bộ hiệu quả và giảm số lượng tham số cần học. Cấu trúc cơ bản của mạng tích chập được minh họa ở Hình 4.

CNN được cấu thành từ ba loại lớp chính [1]:

- **Lớp tích chập:** Thực hiện phép tích chập giữa đầu vào và các bộ lọc để tạo ra các bản đồ đặc trưng, giúp phát hiện các mẫu cục bộ như cạnh, góc hoặc các đặc trưng phức tạp hơn ở các lớp sâu hơn.
- **Lớp gộp:** Giảm kích thước không gian của bản đồ đặc trưng, giúp giảm tính toán và tăng khả năng khái quát hóa của mô hình. Các kỹ thuật phổ biến gồm gộp cực đại (max pooling) và gộp trung bình (average pooling).
- **Lớp kết nối đầy đủ:** Thường được sử dụng ở các lớp cuối cùng, tổng hợp các đặc trưng đã trích xuất để đưa ra dự đoán cuối cùng.

Với đầu vào là một ma trận đặc trưng $X \in \mathbb{R}^{H \times W \times C}$, một lớp tích chập với bộ lọc $K \in \mathbb{R}^{k_H \times k_W \times C}$ sẽ tạo ra bản đồ đặc trưng $F \in \mathbb{R}^{H' \times W'}$ theo công thức:

$$F_{i,j} = \sigma \left(\sum_{m=1}^{k_H} \sum_{n=1}^{k_W} \sum_{c=1}^C K_{m,n,c} \cdot X_{i+m-1,j+n-1,c} + b \right), \quad (16)$$

trong đó $\sigma(\cdot)$ là hàm kích hoạt phi tuyến (ReLU, sigmoid, v.v.), và b là độ chệch của bộ lọc.

2.12 Hàm mất mát InfoNCE

2.12.1 Công thức gốc

InfoNCE là hàm mất mát tương phản được thiết kế nhằm tối đa hóa cận dưới của thông tin tương hỗ (mutual information) giữa các cặp mẫu dương [12]. Với một mẫu neo $\mathbf{z}$, một mẫu dương $\mathbf{z}^+$ và K mẫu âm $\{\mathbf{z}_k^-\}_{k=1}^K$, hàm mất mát InfoNCE chuẩn được định nghĩa là:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(s(\mathbf{z}, \mathbf{z}^+)/\tau)}{\exp(s(\mathbf{z}, \mathbf{z}^+)/\tau) + \sum_{k=1}^K \exp(s(\mathbf{z}, \mathbf{z}_k^-)/\tau)} \quad (17)$$

trong đó $s(\cdot, \cdot)$ là hàm đo độ tương đồng (cosine similarity) và τ là siêu tham số nhiệt độ kiểm soát độ sắc nét của phân phối.

2.12.2 Vai trò của nhiệt độ τ

Siêu tham số nhiệt độ τ kiểm soát mức độ tập trung của gradient vào các mẫu âm khó (hard negatives), trong đó giá trị τ nhỏ khiến mô hình chú trọng hơn vào các mẫu âm có độ tương đồng cao với mẫu neo, tăng cường khả năng phân biệt tinh tế nhưng cũng dễ gây bất ổn định trong huấn luyện.

2.13 Hàm mất mát SmoothL1

Hàm mất mát SmoothL1 (còn được gọi là Huber loss) là sự kết hợp giữa sai số tuyệt đối (L1) và sai số bình phương (L2), giúp mô hình ít nhạy cảm hơn với các điểm ngoại lệ (outliers) đồng thời tránh được hiện tượng bùng nổ gradient trong quá trình huấn luyện [14]. Công thức của SmoothL1 được định nghĩa như sau:

$$\text{Smooth}_{L1}(x) = \begin{cases} 0.5x^2 & \text{nếu } |x| < 1 \\ |x| - 0.5 & \text{ngược lại} \end{cases} \quad (18)$$

trong đó x là sự chênh lệch giữa giá trị dự đoán và giá trị thực tế.

2.14 Độ đo đánh giá

Trong các bài toán hồi quy, chất lượng của mô hình dự đoán được đánh giá thông qua các độ đo định lượng sai lệch giữa giá trị dự đoán $\tilde{y}_i$ và giá trị thực y_i trên tập kiểm tra gồm N mẫu [1].

2.14.1 Sai số tuyệt đối trung bình (Mean Absolute Error - MAE)

MAE đo lường trung bình độ lệch tuyệt đối giữa dự đoán và giá trị thực:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i| \quad (19)$$

MAE có ưu điểm dễ diễn giải vì kết quả mang cùng đơn vị với biến mục tiêu, và không bị chi phối quá mức bởi các điểm ngoại lệ do không có phép bình phương.

2.14.2 Sai số tuyệt đối tương đối trung bình (Relative Mean Absolute Error - RMAE)

RMAE chuẩn hóa sai số tuyệt đối theo giá trị thực, biểu diễn sai lệch dưới dạng tỉ lệ phần trăm so với giá trị quan sát:

$$\text{RMAE} = \frac{1}{N} \sum_{i=1}^N \frac{|\hat{y}_i - y_i|}{y_i} \quad (20)$$

Nhờ chuẩn hóa theo giá trị thực, RMAE cho phép so sánh hiệu năng giữa các mẫu có biên độ rất khác nhau, khắc phục hạn chế của MAE khi phân phối của biến mục tiêu có độ trải rộng lớn.

2.14.3 Căn bậc hai sai số bình phương trung bình (Root Mean Squared Error - RMSE)

RMSE tính căn bậc hai của trung bình bình phương sai số:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad (21)$$

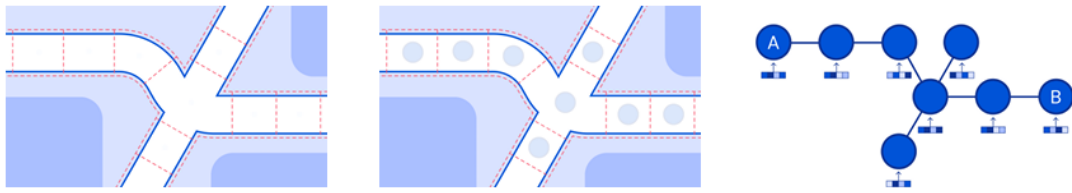
Do phép bình phương, RMSE phạt nặng hơn các sai số lớn so với MAE, khiến độ đo này nhạy cảm hơn với các điểm ngoại lệ. RMSE mang cùng đơn vị với biến mục tiêu, thuận tiện cho việc so sánh trực tiếp với MAE để đánh giá mức độ ảnh hưởng của các ngoại lệ lên hiệu năng tổng thể của mô hình.

2.15 Định nghĩa bài toán dự đoán thời gian di chuyển

Để xây dựng được một mô hình dự báo thời gian di chuyển có độ ổn định và chính xác cao, yêu cầu tiên quyết là phải nắm vững cấu trúc mạng lưới giao thông cũng như các yếu tố tác động đến thời gian di chuyển. Theo đó, mục này sẽ trình

bày khung cơ sở lý thuyết, đóng vai trò làm nền tảng cho phương pháp mô hình hóa được áp dụng trong nghiên cứu.

Để dễ hình dung, Hình 5 minh họa một ví dụ trực quan về mạng lưới đường bộ trên bản đồ địa lý thực tế. Trên bản đồ này, các giao lộ hoặc vòng xuyên giao thông được trừu tượng hóa thành các điểm nút, trong khi các đoạn đường nối liền giữa chúng đóng vai trò là các cạnh kết nối. Việc chuyển đổi một bản đồ phức tạp thành một cấu trúc toán học chặt chẽ là bước đi bắt buộc để máy tính có thể hiểu, từ đó áp dụng hiệu quả các thuật toán tìm đường và mô hình học máy.



Hình 5: Ví dụ của mạng lưới đường bộ [6, 7]

Định nghĩa 1 (Mạng lưới đường bộ). Mạng lưới đường bộ được mô hình hóa dưới dạng một đồ thị $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ như trên hình 5. Trong đó:

- $\mathcal{V}$ là tập hợp các nút giao thông.
- $\mathcal{E}$ là tập hợp các đoạn đường. Mỗi phần tử $s \in \mathcal{E}$ đại diện cho một đoạn đường nối giữa hai nút giao thông.

Định nghĩa 2 (Quỹ đạo và Lộ trình). Một quỹ đạo thô hay một lộ trình được định nghĩa là một chuỗi các điểm GPS $T = \{p_1, \dots, p_N\}$ biểu diễn vị trí của phương tiện, trong đó mỗi điểm $p_i = (lat_i, lon_i)$ bao gồm vĩ độ và kinh độ. Tương ứng với mỗi điểm vị trí, ta có một nhãn thời gian ghi nhận, tạo thành tập hợp $X^{Time} = \{\tau_1, \dots, \tau_N\}$, trong đó τ_i là thời gian ghi nhận vị trí p_i với thời gian xuất phát là $\tau_1 = 0$.

Vì mỗi điểm GPS luôn thuộc về một đoạn đường xác định, từ chuỗi T , ta ánh xạ từng điểm GPS với các đoạn đường tương ứng để thu được chuỗi ngữ nghĩa $X^S = \{s_1, \dots, s_N\}$, trong đó $s_i \in \mathcal{E}$ là ID đại diện cho đoạn đường.

Định nghĩa 3 (Tập dữ liệu). Một tập dữ liệu $\mathcal{D}$ phục vụ cho bài toán ước lượng thời gian là tập hợp của nhiều lộ trình, được biểu diễn như sau:

$$\mathcal{D} = \{(q^{(i)}, y^{(i)})\}_{i=1}^K$$

Trong đó:

- K là tổng số lượng mẫu dữ liệu (số lượng lộ trình).

- $q^{(i)}$ là truy vấn thứ i , bao gồm: chuỗi đoạn đường đi $X^{S(i)}$, quỹ đạo GPS gốc $T^{(i)}$ và chuỗi thời gian ghi nhận tương ứng $X^{Time(i)}$.
- $y^{(i)}$ là thời gian di chuyển thực tế của toàn bộ lộ trình đang xét hay nhân của một mẫu dữ liệu.

Với mỗi cặp $(q^{(i)}, y^{(i)}) \in \mathcal{D}$, tổng thời gian di chuyển thực tế thỏa mãn:

$$y^{(i)} = \tau_N - \tau_1$$

Bài toán. Cho một truy vấn q bao gồm chuỗi đoạn đường X^S và thông tin thời gian X^{Time} trên mạng lưới $\mathcal{G}$, mục tiêu của nghiên cứu là xây dựng một hàm ánh xạ F để dự đoán thời gian di chuyển $\hat{y}$:

$$\hat{y} = F(X^S, X^{Time}, \mathcal{G})$$

Trong đó $F(\cdot)$ là mô hình học máy được huấn luyện để cực tiểu hóa sai số giữa $\hat{y}$ và y trên tập dữ liệu $\mathcal{D}$.

2.16 Định nghĩa bài toán tìm đường tối ưu

Dựa trên cấu trúc mạng lưới đường bộ đã được mô tả tại Mục 2.15, bài toán tìm đường được mô hình hóa thông qua lý thuyết đồ thị với các định nghĩa bổ sung sau:

Định nghĩa 4 (Trọng số và Chi phí di chuyển). Trên đồ thị mạng lưới $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, mỗi đoạn đường $e \in \mathcal{E}$ được gán một hàm trọng số $w : \mathcal{E} \rightarrow \mathbb{R}^+$. Trọng số này đại diện cho chi phí để phương tiện di chuyển qua đoạn đường đó, có thể được thiết lập dựa trên khoảng cách vật lý, thời gian di chuyển thực tế hoặc mức tiêu thụ nhiên liệu.

Định nghĩa 5 (Đường đi ngắn nhất). Một đường đi (path) P từ nút xuất phát v_s đến nút đích v_t là một chuỗi các đỉnh liên tiếp $P = (v_s, v_1, \dots, v_t)$ sao cho mọi cặp $(v_i, v_{i+1}) \in \mathcal{E}$. Chi phí của đường đi P , ký hiệu là $C(P)$, được tính bằng tổng trọng số của các cạnh tạo nên đường đi đó:

$$C(P) = \sum_{(v_i, v_{i+1}) \in P} w(v_i, v_{i+1})$$

Bài toán. Cho đồ thị $\mathcal{G}$ và một truy vấn tìm đường $q = (v_s, v_t)$, mục tiêu của nghiên cứu là xác định một lộ trình P^* sao cho chi phí di chuyển là nhỏ nhất trong tập hợp tất cả các lộ trình khả thi kết nối từ v_s đến v_t :

$$P^* = \arg \min_P C(P)$$

3

Công trình nghiên cứu liên quan

Chương này sẽ trình bày nghiên cứu về các công trình liên quan tới đề tài.

3.1 Tổng quan về dự đoán thời gian di chuyển

3.1.1 Hướng tiếp cận cho bài toán dự đoán thời gian di chuyển

Nhìn chung, có hai phương pháp chính để giải quyết bài toán dự đoán thời gian trong các hệ thống giao thông: phương pháp dựa trên lộ trình và phương pháp dựa trên điểm xuất phát - điểm đích (Origin-Destination based, hay OD-based) [15, 16].

Phương pháp dự đoán thời gian di chuyển dựa trên lộ trình trong giao thông đường bộ đòi hỏi thông tin về lộ trình cụ thể hoặc chuỗi các đoạn đường mà người tham gia giao thông sẽ đi qua. Phương pháp này mô hình hóa thời gian di chuyển bằng cách tổng hợp thời gian dự đoán cho từng đoạn riêng lẻ dọc theo lộ trình. Cách tiếp cận này thường phân tích các đặc điểm về cấu trúc đường bộ, các tình huống lái xe và thời gian di chuyển dự đoán của các đoạn này, sau đó tổng hợp chúng lại để ước tính tổng ETA (thời gian di chuyển ước lượng) cho toàn bộ hành trình. Ưu điểm chính của phương pháp dựa trên đoạn đường là sự đơn giản, vì nó chia nhỏ bài toán ETA thành các bài toán con thông qua việc mô hình hóa thời gian di chuyển trên từng đoạn đường riêng lẻ. Phương pháp này rất hiệu quả về mặt tính toán và có khả năng mở rộng tốt, do thời gian di chuyển của các đoạn đường có thể được ước lượng song song.

Mặt khác, phương pháp dựa trên điểm đầu - điểm cuối không phụ thuộc vào thông tin lộ trình cụ thể. Thay vào đó, nó tập trung vào việc dự đoán thời gian di chuyển giữa điểm xuất phát và điểm đích, sử dụng một tập hợp các đặc trưng đầu vào nhỏ hơn như điểm đi, điểm đến và thời gian khởi hành [16]. Tuy nhiên, phương pháp này có một số hạn chế. Thứ nhất, nếu không truy cập được thông tin lộ trình đầy đủ, các thuộc tính đầu vào thô có sẵn cho việc dự đoán trực tuyến chỉ giới hạn ở điểm đi, điểm đến và thời gian khởi hành. Tập hợp các đặc trưng hạn chế này có thể gây khó khăn trong việc nắm bắt các sắc thái và sự phức tạp của lộ trình di chuyển. Thứ hai, việc sử dụng các đặc điểm tự nhiên, chẳng hạn như khoảng cách hoặc độ tương đồng giữa hai chuyến đi, như một phần của mô hình dự đoán là không đơn giản. Việc chỉ dựa vào kinh độ và vĩ độ của điểm đầu và điểm cuối là không đủ để ước lượng chính xác khoảng cách thực tế hoặc sự tương đồng giữa các lộ trình di chuyển khác nhau. Các đặc điểm khác liên quan

đến mạng lưới đường bộ, mô hình giao thông và các yếu tố ngữ cảnh khác vốn có ảnh hưởng đáng kể đến thời gian di chuyển thường bị bỏ qua.

Tại Thành phố Hồ Chí Minh, đô thị lớn nhất Việt Nam với số lượng lớn các giao lộ, vòng xoay và đèn tín hiệu giao thông, phương pháp dựa trên điểm xuất phát - điểm đích gặp phải khó khăn rất lớn. Bên cạnh đó, tình trạng tắc nghẽn giao thông cao tại thành phố, vốn có thể thay đổi đáng kể qua các đoạn đường và khung thời gian khác nhau, đòi hỏi một sự hiểu biết chi tiết hơn về các mô hình giao thông không gian - thời gian. Điều này tạo thành một trường hợp sử dụng đặc biệt thuyết phục cho phương pháp dự đoán ETA dựa trên lộ trình thay vì các phương pháp dựa trên điểm xuất phát - điểm đích. Do đó, trong luận văn này, chúng tôi sẽ tập trung vào phương pháp dự đoán ETA dựa trên lộ trình.

3.1.2 Tổng quan về phương pháp dự đoán thời gian di chuyển dựa trên lộ trình

Để rõ ràng hơn, phương pháp ETA dựa trên lộ trình được chia thành hai nhóm nhỏ: phương pháp dựa trên đoạn đường và phương pháp dựa trên đường đi.

Phương pháp dựa trên đoạn đường: Một cách tiếp cận đơn giản để ước lượng thời gian di chuyển là dự đoán thời gian trên từng đoạn đường riêng lẻ và ước lượng thời gian tại các giao lộ, sau đó tổng hợp các dự đoán cấp đoạn này để thu được tổng thời gian di chuyển của lộ trình.

Phương pháp này cũng đối mặt với một số hạn chế như sau:

- **Thiếu nhận thức về ngữ cảnh:** Phương pháp dựa trên đoạn đường không xem xét rõ ràng các yếu tố ngữ cảnh khác nhau có thể ảnh hưởng đến thời gian di chuyển, chẳng hạn như điều kiện giao thông, thời tiết, sự kiện hoặc thời gian trong ngày.
- **Khó khăn trong việc xử lý các giao lộ và rẽ:** Ngoài ra, thời gian tiêu tốn tại các giao lộ, khi rẽ hoặc điều hướng qua các mạng lưới đường phức tạp thường bị bỏ qua.
- **Tích lũy sai số ước lượng:** Việc chia nhỏ lộ trình có thể dẫn đến sự tích lũy sai số qua từng đoạn.

Phương pháp dựa trên đường đi hay End-to-end: Để khắc phục các hạn chế trên, các phương pháp dựa trên đường đi nhận toàn bộ lộ trình làm đầu vào và ước tính thời gian di chuyển tổng thể. Cách tiếp cận này yêu cầu thông tin tuần tự của toàn bộ đường đi, kết hợp dữ liệu không gian - thời gian [17]. Các mô hình tiêu biểu như DeepTTE [6] sử dụng lớp tích chập địa lý để nắm bắt tương quan không gian và LSTM cho mô hình hóa thời gian, hay mô hình Wide-Deep-Recurrent [15] xem ETA là bài toán hồi quy phức tạp.

Đặc biệt, sự phát triển gần đây của nhóm phương pháp này đã dẫn đến xu hướng **Đa phương thức**. Các mô hình này mở rộng khả năng của phương pháp dựa trên đường đi bằng cách tích hợp thêm các nguồn dữ liệu ngoại lai (như hình ảnh vệ tinh, thông tin thời tiết, văn bản mạng xã hội) nhằm nắm bắt sâu hơn các yếu tố ngữ cảnh ảnh hưởng đến dòng chảy giao thông mà dữ liệu GPS đơn thuần không thể hiện được.

Mặc dù phương pháp dựa trên đường đi và đa phương thức mang lại độ chính xác cao hơn, chúng vẫn tồn tại những thách thức cần giải quyết:

- **Độ phức tạp tính toán:** Việc mô hình hóa toàn bộ lộ trình và xử lý dữ liệu đa phương thức làm tăng đáng kể chi phí tính toán, đặc biệt với các mạng lưới quy mô lớn.
- **Yêu cầu về dữ liệu:** Để nắm bắt hiệu quả các mẫu phụ thuộc phức tạp, phương pháp này đòi hỏi tập dữ liệu lớn, toàn diện và đồng bộ từ nhiều nguồn khác nhau.
- **Khả năng thích ứng:** Phương pháp này khó thích ứng nhanh với các thay đổi đột ngột trong cấu trúc mạng lưới giao thông (như sửa đường, chặn đường) so với phương pháp dựa trên đoạn đường.

3.2 Trích xuất đặc trưng

Đối với các mô hình dự đoán thời gian di chuyển có hiệu suất cao, việc xác định và trích xuất các đặc trưng liên quan từ tập hợp các nguồn dữ liệu đa dạng là rất cần thiết để nắm bắt được tính chất đa diện của mạng lưới giao thông. Dữ liệu di chuyển có thể được phân loại thành các nhóm chính sau đây [15], đóng vai trò là các đặc trưng đầu vào quan trọng cho việc dự đoán.

- **Thông tin không gian:** Đây là yếu tố then chốt vì thời gian di chuyển có mối tương quan chặt chẽ với đặc điểm cấu trúc của lộ trình. Các đặc trưng này bao gồm thuộc tính của đoạn đường (chiều dài, chiều rộng, cấp độ đường, số làn xe) và sự phân bố của các điểm quan tâm (PoIs) dọc theo lộ trình.
- **Thông tin thời gian:** Thời gian khởi hành (giờ trong ngày, ngày trong tuần, ngày trong năm) và các sự kiện đặc biệt (ngày lễ, mùa) ảnh hưởng lớn đến mật độ giao thông và tốc độ di chuyển.
- **Thông tin giao thông:** Điều kiện thực tế của mạng lưới, được biểu diễn qua các chỉ số như tốc độ trung bình, tốc độ ước tính thời gian thực và tốc độ dòng chảy tự do.

- **Thông tin hành vi:** Hồ sơ và thói quen lái xe của từng tài xế cụ thể, giúp cá nhân hóa dự đoán ETA.
- **Thông tin mở rộng:** Các yếu tố ngoại lai như điều kiện thời tiết, hạn chế giao thông hoặc sửa đường.

3.3 Điểm quan tâm (Points of Interest - PoI)

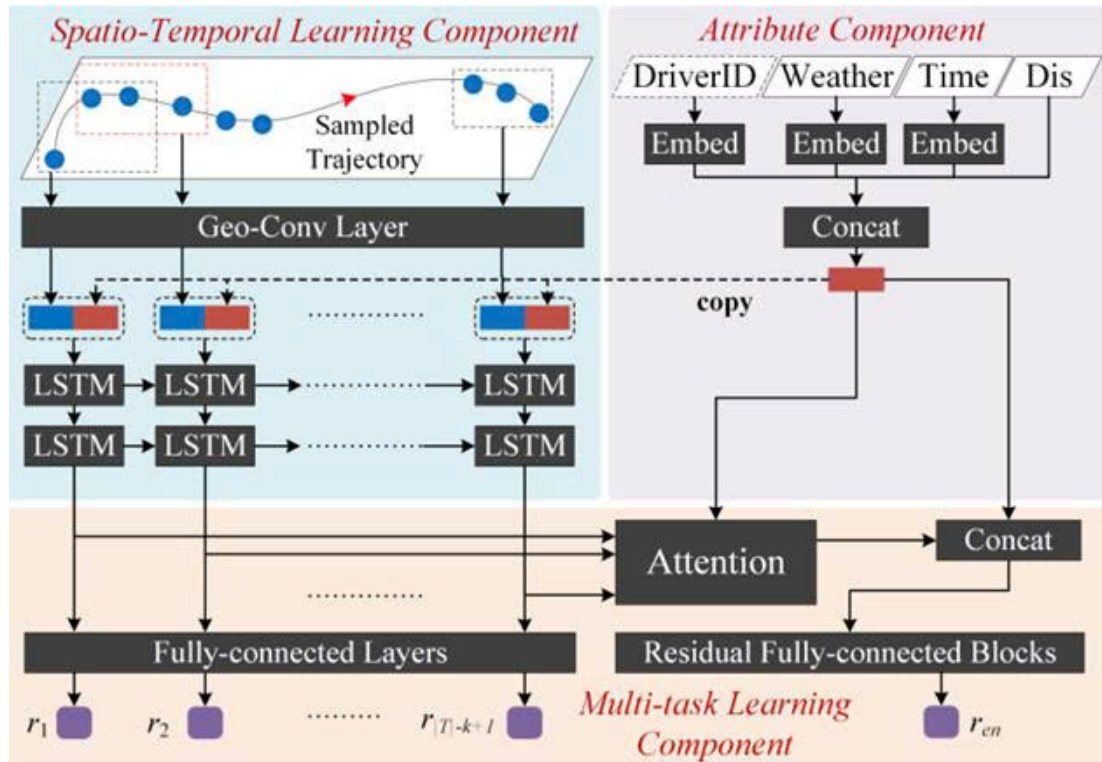
Trong mạng lưới giao thông đô thị, điều kiện lưu thông của một đoạn đường không chỉ phụ thuộc vào các thuộc tính hình học của chính đoạn đường đó mà còn chịu ảnh hưởng của môi trường chức năng xung quanh. Các địa điểm như trường học, trung tâm thương mại, bệnh viện hay khu văn phòng tạo ra các luồng phương tiện đặc trưng, phản ánh hoạt động kinh tế - xã hội của khu vực. Do đó, khái niệm *Điểm quan tâm* được giới thiệu nhằm cung cấp một đơn vị ngữ cảnh không gian bổ sung cho việc mô hình hóa đặc tính giao thông cục bộ.

Định nghĩa 6 (Điểm quan tâm). Một **Điểm quan tâm** (Point of Interest - PoI) là một thực thể địa lý có vị trí xác định trong mặt phẳng địa lý, được đặc trưng bởi tọa độ không gian $\ell \in \mathbb{R}^2$ và một danh mục chức năng $g \in \{1, \dots, G\}$ phản ánh loại hoạt động diễn ra tại đó, với G là tổng số nhóm danh mục.

Tập hợp toàn bộ PoI trong phạm vi nghiên cứu được ký hiệu là $\mathcal{P} = \{p_1, p_2, \dots, p_M\}$. Đối với mỗi đoạn đường s , thông tin PoI lân cận được tổng hợp thành véc-tơ đếm $\mathbf{p}_s \in \mathbb{N}^G$, trong đó phần tử thứ g biểu thị số lượng PoI thuộc nhóm danh mục g nằm trong vùng lân cận của s . Véc-tơ này mã hóa chức năng đô thị của khu vực xung quanh đoạn đường và đóng vai trò đầu vào cho cơ chế mã hóa PoI được trình bày trong Mục 4.1. Chi tiết về cách xây dựng $\mathbf{p}_s$ từ dữ liệu thô được trình bày trong Mục 5.

3.4 DeepTTE: Mô hình ước lượng thời gian dựa trên mạng nơ-ron sâu

DeepTTE (Deep Travel Time Estimation) [6] là một mô hình “end-to-end” tiêu biểu, được thiết kế để ước lượng thời gian di chuyển cho toàn bộ lộ trình thay vì tổng hợp từ các đoạn đường nhỏ lẻ. Kiến trúc tổng quát của mô hình được minh họa như trong Hình 6.



Hình 6: Cấu trúc của mô hình DeepTTE [6]

Như quan sát trong hình, cấu trúc này bao gồm ba thành phần chính phối hợp với nhau để xử lý các loại dữ liệu đầu vào khác nhau:

- **Thành phần Thuộc tính:** Chịu trách nhiệm xử lý các đặc trưng tĩnh như thời tiết, thông tin tài xế.
- **Thành phần Không gian - Thời gian:** Đóng vai trò nòng cốt trong việc trích xuất đặc trưng từ chuỗi quỹ đạo GPS thô.
- **Thành phần Học đa nhiệm vụ:** Thực hiện ước lượng đồng thời thời gian cho từng đoạn con và toàn bộ lộ trình.

Các thành phần đó có cấu trúc chi tiết như sau:

Thành phần Thuộc tính:

Thành phần này chịu trách nhiệm xử lý các đặc trưng tĩnh (ngoại lai) có ảnh hưởng đến thời gian di chuyển, bao gồm: thời tiết, định danh tài xế, tổng chiều dài của lộ trình và thời gian xuất phát (bao gồm ngày trong tuần, ngày trong tháng

và phút trong ngày). Do các thông tin này là dữ liệu dạng phân loại (categorical data) và rời rạc, mô hình sử dụng kỹ thuật **biểu diễn vector (embedding)** để chuyển đổi chúng thành các giá trị số thực có thể tính toán được. Cụ thể, mỗi giá trị phân loại $v \in [V]$ sẽ được ánh xạ sang một không gian vector $\mathbb{R}^E$ thông qua ma trận trọng số $\mathbf{W} \in \mathbb{R}^{V \times E}$. Cuối cùng, các vector biểu diễn này được nối với thông tin về tổng khoảng cách của lộ trình $\Delta d_{p_1 \rightarrow p_{|T|}}$ để tạo thành vector đặc trưng thuộc tính đầu ra, ký hiệu là **attr**.

Thành phần Không gian - Thời gian:

a) Geo-Conv Layer:

Để nắm bắt sự phụ thuộc không gian từ chuỗi GPS thô, Geo-Conv thực hiện ánh xạ phi tuyến tọa độ địa lý và áp dụng tích chập 1D. Với mỗi điểm GPS p_i , biểu diễn vector được tính như sau:

$$\mathbf{loc}_i = \tanh(\mathbf{W}_{\text{loc}} \cdot [p_i.\text{lat} \circ p_i.\text{lng}]) \quad (22)$$

Sau đó, một phép tích chập với cửa sổ trượt kích thước k được áp dụng để tổng hợp thông tin từ các điểm lân cận (tạo thành đoạn đường cục bộ):

$$\mathbf{loc}_i^{\text{conv}} = \sigma_{\text{cnn}}(\mathbf{W}_{\text{conv}} * \mathbf{loc}_{i:i+k-1} + \mathbf{b}) \quad (23)$$

Cuối cùng, khoảng cách của đoạn cục bộ được nối vào để tạo thành bản đồ đặc trưng $\mathbf{loc}^f$.

b) Recurrent Layer:

Sử dụng kiến trúc LSTM chồng để nắm bắt sự phụ thuộc thời gian của chuỗi đặc trưng không gian. Trạng thái ẩn được cập nhật bằng cách kết hợp cả đặc trưng không gian và vector thuộc tính **attr**:

$$h_i = \sigma_{\text{rnn}}(\mathbf{W}_x \cdot \mathbf{loc}_i^f + \mathbf{W}_h \cdot h_{i-1} + \mathbf{W}_a \cdot \mathbf{attr}) \quad (24)$$

Thành phần Học đa nhiệm vụ:

DeepTTE ước lượng đồng thời thời gian của từng đoạn cục bộ và toàn bộ lộ trình. Đặc biệt, đối với toàn bộ lộ trình, mô hình sử dụng cơ chế attention để tập trung vào các đoạn đường quan trọng như giao lộ hoặc đoạn tắc nghẽn. Trọng số attention α_i được tính toán dựa trên sự tương tác giữa vector thuộc tính và trạng thái ẩn:

$$z_i = \langle \sigma_{\text{att}}(\mathbf{attr}), h_i \rangle \quad (25)$$

$$\alpha_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad (26)$$

Vector biểu diễn cuối cùng của lộ trình là tổng có trọng số: $h_{\text{att}} = \sum \alpha_i h_i$.

Hàm mục tiêu

Để huấn luyện mô hình, tác giả sử dụng Sai số Phần trăm Tuyệt đối Trung bình (MAPE) làm mục tiêu tối ưu hóa. Hàm mất mát được thiết kế bao gồm hai thành phần riêng biệt để đảm bảo độ chính xác cho cả các đoạn con và toàn bộ hành trình:

a) *Mất mát cho các đoạn đường cục bộ (L_{local}):*

Đây là trung bình sai số dự đoán của tất cả các đoạn đường con. Với r_i là thời gian dự đoán cho đoạn thứ i , $p_i.ts$ là nhãn thời gian thực tế, và ϵ là hằng số nhỏ để tránh lỗi mẫu số bằng 0:

$$L_{\text{local}} = \frac{1}{|T| - k + 1} \sum_{i=1}^{|T|-k+1} \left| \frac{r_i - (p_{i+k-1}.ts - p_i.ts)}{p_{i+k-1}.ts - p_i.ts + \epsilon} \right| \quad (27)$$

b) *Mất mát cho toàn bộ lộ trình (L_{en}):*

Đây là sai số tuyệt đối giữa tổng thời gian dự đoán r_{en} và thời gian thực tế của toàn bộ hành trình (từ điểm đầu p_1 đến điểm cuối $p_{|T|}$):

$$L_{\text{en}} = \left| \frac{r_{\text{en}} - (p_{|T|}.ts - p_1.ts)}{p_{|T|}.ts - p_1.ts} \right| \quad (28)$$

c) *Hàm mất mát tổng hợp:*

Cuối cùng, mô hình tối thiểu hóa sự kết hợp giữa hai thành phần trên thông qua hệ số cân bằng λ :

$$\mathcal{L} = \lambda \cdot L_{\text{local}} + (1 - \lambda) \cdot L_{\text{en}} \quad (29)$$

Hạn chế và Khoảng trống nghiên cứu

Mặc dù DeepTTE có khả năng trích xuất tốt các đặc trưng không gian cục bộ từ dữ liệu GPS thô, phương pháp này vẫn tồn tại hạn chế cốt lõi trong việc mô hình hóa mối liên kết cấu trúc giữa các đoạn đường tạo nên lộ trình.

Cụ thể, DeepTTE dựa vào cơ chế hồi quy (RNN/LSTM) để xử lý chuỗi tọa độ theo thời gian. Cách tiếp cận này khiến mô hình tập trung vào sự chuyển tiếp tuần tự giữa các điểm GPS liên kề, nhưng lại không nắm bắt được mối quan hệ ngữ nghĩa toàn cục và sự phụ thuộc lẫn nhau giữa các đoạn đường trong mạng lưới. Trong thực tế, các đoạn đường trên một lộ trình không hoạt động độc lập mà có tính liên kết chặt chẽ (ví dụ: đặc điểm lưu thông của đoạn A có thể tác động đến trạng thái của đoạn C dù chúng không nằm kề nhau). Việc chỉ xem xét chuỗi GPS đơn thuần khiến DeepTTE bỏ qua các thông tin về các mối tương quan ẩn này, dẫn đến hạn chế trong việc hiểu trọn vẹn ngữ cảnh của chuyến đi.

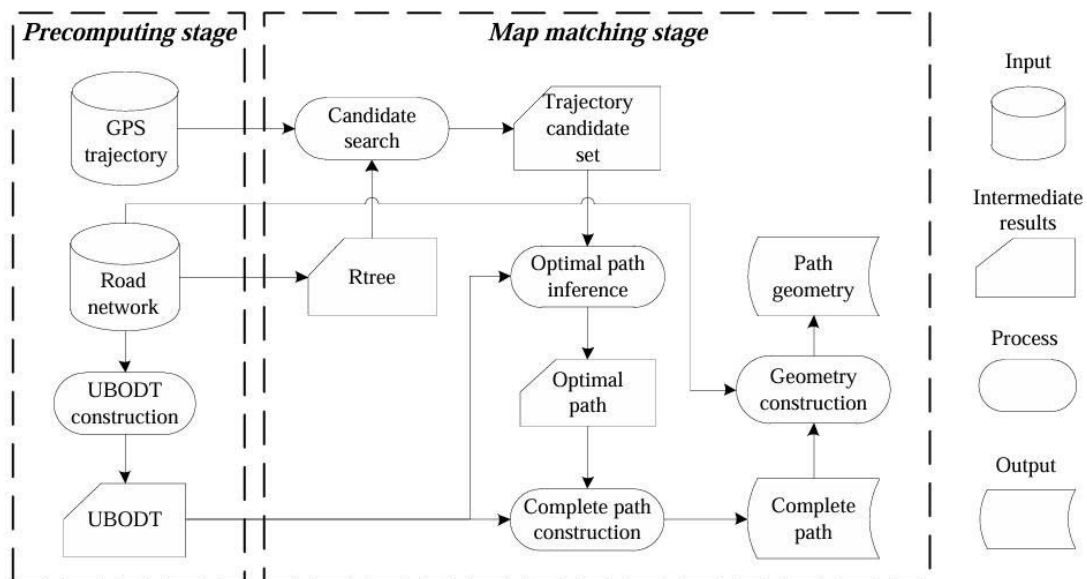
Khoảng trống này đặt ra nhu cầu về một mô hình biểu diễn có khả năng học được ngữ nghĩa của các đoạn đường trong lộ trình và nắm bắt được sự liên kết giữa các đoạn đường để cải thiện độ chính xác dự đoán.

3.5 MulT-TTE: Phương pháp học biểu diễn lộ trình đa phương thức

Để khắc phục hạn chế về khả năng nắm bắt ngữ nghĩa và sự phụ thuộc ngữ cảnh của các phương pháp chỉ dựa trên GPS, MulT-TTE (Multi-Faceted Route Representation Learning) [9] đề xuất một framework đa phương thức. Tuy nhiên, trước khi đi vào mô hình hóa, dữ liệu GPS thô cần được xử lý để chuyển đổi thành các biểu diễn có cấu trúc trên mạng lưới giao thông.

So khớp bản đồ nhanh (Fast Map Matching - FMM)

Thành phần cốt lõi của MulT-TTE là việc học biểu diễn dựa trên các đoạn đường, do đó chúng tôi sử dụng thuật toán **So khớp bản đồ nhanh (Fast Map Matching - FMM)** được đề xuất bởi Yang và Gidófalvi [8] để tiền xử lý dữ liệu. Như được minh họa trong Hình 7, kiến trúc của FMM được chia thành hai giai đoạn chính nhằm tối ưu hóa hiệu năng truy vấn trên tập dữ liệu lớn:



Hình 7: Quy trình của thuật toán Fast Map Matching gồm hai giai đoạn: Tính toán trước (Precomputing) và So khớp (Map matching).[8]

1. **Giai đoạn tính toán trước (Precomputing stage):** Dựa trên mạng lưới đường bộ, thuật toán xây dựng cấu trúc dữ liệu được gọi là *Upper Bounded Origin Destination Table* (UBODT). Đây là bảng lưu trữ trước tất cả các cặp đường đi ngắn nhất giữa các nút trong mạng lưới có khoảng cách nhỏ

hơn một ngưỡng Δ xác định. Việc sử dụng UBODT cho phép thay thế các thuật toán tìm đường lặp lại tốn kém bằng thao tác tra cứu bảng băm tốc độ cao.

2. **Giai đoạn so khớp bản đồ (Map matching stage):** Với đầu vào là quỹ đạo GPS thô $T = \{p_1, p_2, \dots, p_m\}$, quy trình xử lý được thực hiện qua các bước:

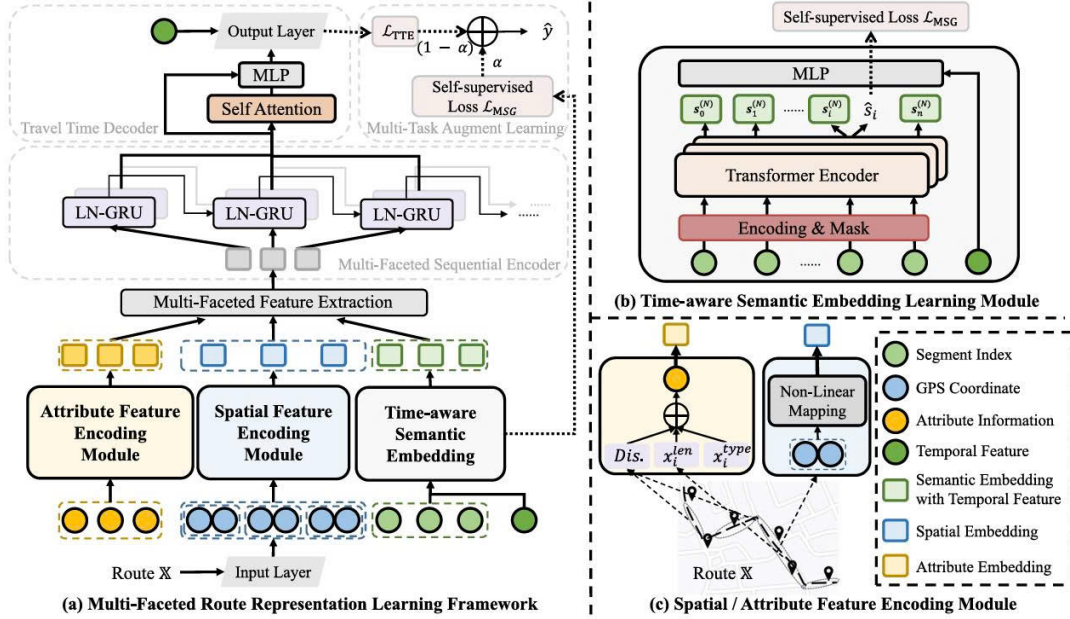
- **Tìm kiếm ứng viên (Candidate Search - CS):** Sử dụng cấu trúc chỉ mục không gian R-tree để tìm kiếm tập hợp các đoạn đường ứng viên (*Candidate set*) cho mỗi điểm GPS quan sát được trong bán kính $[18]r$.
- **Suy luận đường đi tối ưu (Optimal Path Inference - OPI):** Áp dụng mô hình Markov ẩn để tìm đường đi tối ưu (*Optimal path*). Theo Yang và Gidófalvi, hàm mục tiêu được tối đa hóa dựa trên tích của hai loại xác suất:
 - *Xác suất phát xạ (Emission probability):* Mô hình hóa sai số đo đạc GPS theo phân phối chuẩn dựa trên khoảng cách từ điểm quan sát đến đoạn đường ứng viên.
 - *Xác suất chuyển trạng thái (Transition probability):* Được tính toán dựa trên sự chênh lệch giữa khoảng cách Euclide và khoảng cách đường đi ngắn nhất (truy xuất từ UBODT) giữa hai ứng viên liên tiếp.
- **Xây dựng đường đi (Path Construction):** Từ chuỗi các ứng viên tối ưu, thuật toán khôi phục lại hình học chi tiết (*Path geometry*) và hoàn thiện đường đi (*Complete path*) để **đảm bảo tính kết nối liên mạch** trên mạng lưới đường bộ.

Kết quả đầu ra cuối cùng chuyển đổi quỹ đạo GPS thành một chuỗi ngữ nghĩa các ID đoạn đường $X^S = \{s_1, s_2, \dots, s_n\}$, đóng vai trò là đầu vào cho mô hình MulT-TTE.

Trích xuất đặc trưng đa phương thức

Sau khi hoàn tất quá trình **so khớp bản đồ (Map Matching)**, dữ liệu quỹ đạo GPS ban đầu được chuyển đổi thành một chuỗi biểu diễn các đoạn đường nối tiếp nhau. Chuỗi đoạn đường này đóng vai trò là dữ liệu đầu vào chính cho MulT-TTE.

Như được minh họa trong Hình 8, kiến trúc của MulT-TTE được thiết kế để nắm bắt toàn diện thông tin của lộ trình thông qua **3 module xử lý chính**, tương ứng với ba khía cạnh đặc trưng khác nhau:



Hình 8: Kiến trúc tổng thể của mô hình Multi-TTE với ba module trích xuất đặc trưng chính: Thuộc tính, Không gian và Ngữ nghĩa nhận thức thời gian. [9]

- **Đặc trưng Không gian - H^T :** Thay vì sử dụng trực tiếp chuỗi GPS thô (vốn chứa nhiều nhiễu), module này trích xuất thông tin hình học từ chuỗi đoạn đường X^S đã thu được ở bước so khớp bản đồ. Cụ thể, với mỗi đoạn đường s_i , mô hình sử dụng tọa độ địa lý của hai điểm đầu mút (giao lộ) để đại diện cho vị trí không gian của nó.

Giả sử đoạn đường s_i nối hai điểm có tọa độ là $p_i(lat, lon)$ và $p_{i+1}(lat, lon)$. Biểu diễn vector không gian $\mathbf{x}_i^t$ cho đoạn đường này được tính toán thông qua một ánh xạ phi tuyến:

$$\mathbf{x}_i^t = \tanh(\mathbf{W}_{loc}[p_i.lat; p_i.lon; p_{i+1}.lat; p_{i+1}.lon] + \mathbf{b}_{loc}) \quad (30)$$

Trong đó:

- $[p_i.lat; p_i.lon; \dots]$ là phép nối các giá trị vĩ độ và kinh độ của hai điểm đầu mút.
- $\mathbf{W}_{loc}$ và $\mathbf{b}_{loc}$ là các ma trận trọng số và bias học được trong quá trình huấn luyện.

Kết quả đầu ra là tập hợp các vector này, tạo thành chuỗi đặc trưng không gian, ký hiệu là $H^T = \{\mathbf{x}_1^t, \mathbf{x}_2^t, \dots, \mathbf{x}_n^t\}$. Chuỗi H^T này sẽ đại diện cho thông tin vị trí và hình học của toàn bộ lộ trình để đưa vào các lớp xử lý tiếp theo.

- **Đặc trưng Thuộc tính - H^A :** Mỗi đoạn đường chứa các thông tin môi trường như loại đường (highway, residential...) và độ dài. Để máy tính có thể xử lý được các thông tin dạng chữ (như tên loại đường), mô hình thực hiện các bước sau cho một đoạn đường s :

1. **Mã hóa loại đường:** Chuyển đổi loại của đoạn đường s từ dạng phân loại sang một vector số thực ($\mathbf{x}_s^{type}$).
2. **Tính toán ngữ cảnh:** Xác định độ dài của chính đoạn đường đó (s^{len}) và tổng độ dài quãng đường đã đi qua trước đó ($\sum k^{len}$) để mô hình hiểu được vị trí tương đối của đoạn đường trong một lộ trình.
3. **Tổng hợp thông tin:** Tất cả các giá trị trên được ghép lại thành một vector duy nhất $\mathbf{a}_s$ đại diện cho toàn bộ thuộc tính của đoạn đường

$$\mathbf{a}_s = [\mathbf{x}_s^{type}; s^{len}; \sum_{k \in \mathbb{E}'} k^{len}] \quad (31)$$

Tập hợp các vector này tạo thành chuỗi đặc trưng thuộc tính H^A .

- **Đặc trưng Ngữ nghĩa nhận thức thời gian - H^{tS} :** Đây là thành phần cốt lõi của MulT-TTE, giúp mô hình nắm bắt được các mối quan hệ ngữ cảnh phức tạp giữa các đoạn đường. Lấy cảm hứng từ mô hình BERT trong xử lý ngôn ngữ tự nhiên, mô hình coi ID của mỗi đoạn đường như một "từ" và cả lộ trình như một "câu".

Quy trình xử lý gồm hai khía cạnh gắn liền với nhau:

- **Kiến trúc của module:** Sử dụng các lớp mã hóa Transformer đa tầng để trích xuất đặc trưng. Cơ chế Attention bên trong Transformer cho phép mỗi đoạn đường "nhìn" thấy và liên kết với tất cả các đoạn đường khác trong lộ trình để tạo ra chuỗi vector ngữ cảnh $S^{(N)}$.
- **Cơ chế Học tự giám sát (Self-supervised Learning):** Để huấn luyện Transformer Encoder hiểu sâu hơn về cấu trúc mạng lưới, mô hình áp dụng cơ chế Mô hình Ngôn ngữ bị che (Masked Language Model - MLM). Cụ thể, một tỉ lệ ngẫu nhiên các đoạn đường đầu vào sẽ bị "che" đi, và mô hình phải dựa vào ngữ cảnh của các đoạn còn lại để dự đoán chính xác đoạn đường bị che đó.

Sau khi có được biểu diễn ngữ nghĩa $S^{(N)}$ từ Transformer, để tích hợp yếu tố động, thông tin về thời điểm xuất phát được mã hóa thành vector X^t (tạo từ phút trong ngày, ngày trong tuần, ngày trong năm).

Cuối cùng, đặc trưng ngữ nghĩa $S^{(N)}$ và đặc trưng thời gian X^t được hợp nhất thông qua Mạng nơ-ron đa lớp:

$$H^{tS} = \text{ReLU}(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 [S^{(N)}; X^t] + \mathbf{b}_1) + \mathbf{b}_2) + [S^{(N)}; X^t] \quad (32)$$

Trong đó:

- X^t : Vector đại diện cho thời điểm xuất phát của phương tiện.
- $\mathbf{W}_1, \mathbf{W}_2$ và $\mathbf{b}_1, \mathbf{b}_2$: Các tham số trọng số và bias học được.

Kết quả H^{TS} là chuỗi đặc trưng chứa đựng cả thông tin cấu trúc mạng lưới (nhờ Transformer/BERT) lẫn ngữ cảnh thời gian thực tế.

Bộ mã hóa tuần tự và Hợp nhất

Sau khi các module thành phần trích xuất được ba chuỗi đặc trưng riêng biệt: Ngữ nghĩa nhận thức thời gian (H^{TS}), Không gian (H^T) và Thuộc tính (H^A), chúng được hợp nhất để tạo thành biểu diễn đa phương diện thống nhất.

1. Cơ chế Hợp nhất (Fusion):

Tại mỗi bước i (tương ứng với đoạn đường thứ i trong lộ trình), các vector đặc trưng từ ba nguồn được hợp nhất thông qua hàm g_{fuse} , được định nghĩa là phép nối (concatenation) theo chiều đặc trưng:

$$x_i = g_{\text{fuse}}(h_i^{TS}, h_i^T, h_i^A) = [h_i^{TS}; h_i^T; h_i^A] \quad (33)$$

Tập hợp các vector này tạo thành chuỗi đầu vào tổng hợp cho bộ mã hóa tuần tự, ký hiệu là $H_{\mathbb{X}} = \{x_1, x_2, \dots, x_n\}$.

2. Bộ mã hóa LayerNorm-enhanced GRU:

Để xử lý chuỗi $H_{\mathbb{X}}$ và nắm bắt sự phụ thuộc tuần tự về thời gian, MulT-TTE sử dụng mạng GRU được tăng cường Chuẩn hóa lớp (LayerNorm-enhanced GRU) [19]. Việc tích hợp LayerNorm giúp ổn định quá trình huấn luyện và giảm thiểu vấn đề biến thiên hiệp phương sai trên các lộ trình dài.

Với đầu vào là x_t tại bước thời gian t , các cổng và trạng thái ẩn h_t được tính toán như sau:

$$z_t = \sigma(\mathcal{LN}(\mathbf{W}_z x_t) + \mathcal{LN}(\mathbf{U}_z h_{t-1})) \quad (34)$$

$$r_t = \sigma(\mathcal{LN}(\mathbf{W}_r x_t) + \mathcal{LN}(\mathbf{U}_r h_{t-1})) \quad (35)$$

$$\tilde{h}_t = \tanh(\mathcal{LN}(\mathbf{W}_h x_t) + \mathcal{LN}(\mathbf{U}_h (r_t \odot h_{t-1}))) \quad (36)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (37)$$

Trong đó:

- z_t và r_t lần lượt là cổng cập nhật và cổng đặt lại.
- $\tilde{h}_t$ là trạng thái ẩn ứng viên.
- $\mathcal{LN}$ là hàm Chuẩn hóa lớp.

- σ là hàm kích hoạt Sigmoid, $\odot$ là phép nhân từng phần tử.
- $\mathbf{W}_*, \mathbf{U}_*$ là các tham số trọng số học được.

Cuối cùng, trạng thái ẩn tại bước cuối cùng h_n sẽ được sử dụng làm biểu diễn toàn cục của lộ trình để đưa vào lớp Output dự đoán thời gian di chuyển.

Huấn luyện Đa nhiệm vụ và Học tự giám sát

Để tăng cường khả năng hiểu ngữ nghĩa mạng lưới, MulT-TTE áp dụng cơ chế Mô hình Ngôn ngữ bị che (Masked Language Model - MLM) tương tự BERT. Một tỷ lệ phần trăm các đoạn đường trong chuỗi ngữ nghĩa sẽ bị che và mô hình phải dự đoán lại chúng ($\mathcal{L}_{\text{MSG}}$). Hàm mất mát cuối cùng là sự kết hợp giữa nhiệm vụ tự giám sát này và nhiệm vụ dự đoán thời gian chính ($\mathcal{L}_{\text{TTE}}$):

$$\mathcal{L} = (1 - \alpha)\mathcal{L}_{\text{MSG}} + \alpha\mathcal{L}_{\text{TTE}} \quad (38)$$

Đánh giá hiệu quả thực nghiệm

Thực nghiệm trên hai bộ dữ liệu quy mô lớn (Chengdu và Porto) cho thấy MulT-TTE đạt hiệu suất vượt trội so với các phương pháp khác [9]:

- **So với DeepTTE:** MulT-TTE giảm sai số (MAPE) đáng kể (12.59% so với 18.44% trên dữ liệu Chengdu). Điều này chứng minh rằng việc kết hợp biểu diễn ngữ nghĩa (Semantic ID) và cơ chế Attention giúp mô hình hiểu sâu hơn về cấu trúc mạng lưới so với việc chỉ dựa vào tọa độ GPS thô như DeepTTE.
- **So với phương pháp truyền thống (TEMP, ST-NN):** Các phương pháp dựa trên OD hoặc thống kê truyền thống không thể nắm bắt được sự thay đổi phức tạp của các đoạn đường trung gian, dẫn đến sai số cao hơn nhiều (trên 25% MAPE).

Hạn chế và Khoảng trống nghiên cứu

Mặc dù MulT-TTE đạt hiệu suất cao, hạn chế lớn nhất của phương pháp này là chỉ tập trung khai thác thông tin cục bộ nằm trên chính lộ trình di chuyển. Cụ thể, mô hình chỉ giới hạn việc học các đặc trưng nội tại bao gồm: đặc trưng không gian, đặc trưng thời gian, thuộc tính tĩnh của đoạn đường và mối liên kết ngữ nghĩa giữa các đoạn đường. Do đó, mô hình đã bỏ qua hoàn toàn thông tin không gian từ các khu vực lân cận, đặc biệt là sự phân bố của các Điểm quan tâm (Points of Interest - PoI). Trong thực tế, tốc độ di chuyển không chỉ phụ thuộc vào bản thân con đường mà còn chịu tác động lan truyền từ môi trường xung quanh. Ví dụ: Một đoạn đường dù thông thoáng về hạ tầng vẫn có thể bị tắc nghẽn nếu

nằm gần một bệnh viện lớn. Lưu lượng phương tiện ra vào bệnh viện sẽ gây cản trở dòng giao thông (tác động lan truyền), khiến các xe đi ngang qua bị chậm lại dù không có nhu cầu vào bệnh viện. Vì MulT-TTE chỉ xét dữ liệu "trên đường" mà không biết đến sự tồn tại của bệnh viện "cạnh đường", mô hình sẽ không dự báo chính xác được các tình huống này. Đây chính là khoảng trống mà nghiên cứu này tập trung giải quyết.

3.6 Tổng quan về bài toán tìm đường tối ưu

Bên cạnh khả năng dự báo thời gian chính xác, việc xác định lộ trình di chuyển tốt nhất là một thành phần cốt lõi không thể thiếu trong các hệ thống dẫn đường. Mục này trình bày khảo sát các hướng phát triển của bài toán tìm đường trên đồ thị mạng lưới quy mô lớn.

3.6.1 Hướng tiếp cận cho bài toán tìm đường tối ưu

Nhìn chung, các phương pháp giải quyết bài toán tìm đường trên mạng lưới giao thông quy mô lớn được phân chia thành hai hướng tiếp cận chính: nhóm giải thuật tìm kiếm trực tiếp và nhóm kỹ thuật tăng tốc dựa trên tính toán trước (Precomputation-based acceleration).

Kỹ thuật tìm kiếm trực tiếp và định hướng mục tiêu: Các thuật toán cổ điển như Dijkstra [20] hay Bellman-Ford [21, 22] đóng vai trò là nền tảng sơ khai, đảm bảo tìm được lộ trình tối ưu tuyệt đối. Tuy nhiên, các thuật toán này gặp hạn chế lớn về tốc độ khi không gian tìm kiếm mở rộng theo cấp số nhân trên các bản đồ đô thị hiện đại. Để cải thiện, thuật toán A^* [23] đã tích hợp thêm các hàm ước lượng (heuristic) để thu hẹp phạm vi tìm kiếm về phía điểm đích. Các biến thể nâng cao khác như ALT [24] hay Arc Flags [25, 26] thực hiện tiền xử lý nhẹ để loại bỏ sớm các nhánh đường không tiềm năng, giúp giảm bớt không gian duyệt nhưng vẫn chưa giải quyết triệt để vấn đề hiệu suất trên các đồ thị có kích thước siêu lớn.

Kỹ thuật giới hạn bước nhảy và dán nhãn: Nhằm đạt được tốc độ truy vấn ở mức cực nhanh, hướng tiếp cận này thực hiện tiền xử lý một khối lượng dữ liệu khổng lồ để lưu trữ trước khoảng cách giữa các nút giao thông quan trọng. Các giải thuật tiêu biểu như Transit Node Routing (TNR) [27] hay Hub Labels (HL) [28, 29] đưa bài toán tìm đường về dạng tra cứu bảng dữ liệu nhanh chóng. Tuy nhiên, hạn chế lớn nhất của nhóm này là yêu cầu dung lượng lưu trữ rất lớn và phải tính toán lại toàn bộ dữ liệu từ đầu mỗi khi có sự thay đổi về trọng số giao thông, khiến chúng khó thích ứng được với những biến động thực tế của dòng xe.

3.6.2 Tổng quan về các giải thuật định tuyến dựa trên cấu trúc phân hoạch

Đây là hướng tiếp cận quan trọng nhất trong nghiên cứu này, tập trung vào việc khai thác cấu trúc phân cấp và tính chất phân hoạch của mạng lưới đường bộ để cân bằng giữa tốc độ truy vấn và khả năng cập nhật trọng số giao thông.

Kỹ thuật phân cấp mạng lưới (Hierarchical Techniques): Dựa trên quan sát thực tế là các lộ trình dài thường ưu tiên sử dụng các tuyến đường huyết mạch, thuật toán Contraction Hierarchies (CH) [30] thực hiện sắp xếp các đỉnh theo mức độ quan trọng và thiết lập các "cạnh tắt" (shortcuts) để bỏ qua việc phải duyệt qua các con đường nhỏ lẻ. Cơ chế này cho phép thuật toán chỉ tập trung tìm kiếm hướng lên các tầng bậc cao hơn của mạng lưới, giúp rút ngắn thời gian phản hồi hàng nghìn lần trong khi vẫn đảm bảo tính chính xác của lộ trình.

Kỹ thuật phân hoạch tùy chỉnh (Customizable Partition-based): Để ứng phó với tình trạng trọng số giao thông thay đổi liên tục, kiến trúc Customizable Route Planning (CRP) đã ra đời [11] với đặc điểm then chốt là tách biệt hoàn toàn giữa việc xử lý cấu trúc mạng lưới và việc cập nhật trọng số di chuyển. CRP phân hoạch đồ thị thành các ô (cells) lồng nhau và tính toán trước các đường đi ngắn nhất giữa các đỉnh nằm ở biên giới của các ô. Khi tình trạng giao thông thay đổi, hệ thống chỉ cần cập nhật lại thông tin trong các ô bị ảnh hưởng thay vì phải xử lý lại toàn bộ bản đồ, cho phép phản hồi các biến động thực tế chỉ trong thời gian mili giây.

Giải thuật Multilevel Dijkstra (MLD): Là giải pháp mở rộng dựa trên nguyên lý của CRP, MLD sử dụng cấu trúc phân hoạch đa tầng kết hợp với đồ thị lớp phủ (overlay graph). Điểm mạnh của MLD nằm ở khả năng xây dựng các lớp đồ thị rút gọn chỉ bao gồm các đỉnh biên, cho phép thuật toán tìm kiếm trên một không gian dữ liệu đã được nén gọn. Nhờ cơ chế đa tầng lồng nhau và khả năng tùy chỉnh linh hoạt, MLD mang lại hiệu suất vượt trội cả về tốc độ tìm đường lẫn khả năng tích hợp các dữ liệu dự báo thời gian thực, trở thành giải pháp tối ưu cho các hệ thống bản đồ quy mô lớn hiện nay.

3.7 Giải thuật tìm đường Dijkstra

Thuật toán Dijkstra, được đề xuất bởi nhà khoa học máy tính Edsger W. Dijkstra vào năm 1956 [20], là một trong những thuật toán nền tảng và kinh điển nhất trong lý thuyết đồ thị. Thuật toán này được sử dụng để giải quyết bài toán tìm đường đi ngắn nhất từ một đỉnh nguồn đến tất cả các đỉnh còn lại trên một đồ thị, với điều kiện tiên quyết là trọng số của các cạnh không được có giá trị âm.

3.7.1 Nguyên lý hoạt động

Thuật toán Dijkstra hoạt động dựa trên phương pháp tham lam. Tại mỗi bước, thuật toán luôn chọn đỉnh có khoảng cách tạm thời nhỏ nhất chưa được xử lý để duyệt, sau đó tiến hành cập nhật khoảng cách từ đỉnh này đến các đỉnh kề với nó. Quá trình này đảm bảo rằng khi một đỉnh được đưa ra khỏi hàng đợi để xử lý, khoảng cách từ đỉnh nguồn đến nó đã là ngắn nhất và không thể tối ưu thêm.

Giả sử đồ thị mạng lưới $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ với hàm trọng số $w(u, v) \geq 0$ cho mọi cạnh $(u, v) \in \mathcal{E}$. Gọi v_s là đỉnh xuất phát và v_t là đỉnh đích. Thuật toán duy trì hai cấu trúc dữ liệu chính:

- Mảng khoảng cách $d[v]$: Lưu trữ khoảng cách ngắn nhất hiện tại từ v_s đến đỉnh v . Ban đầu, khởi tạo $d[v_s] = 0$ và $d[v] = \infty$ với mọi $v \neq v_s$.
- Hàng đợi ưu tiên Q : Lưu trữ các đỉnh đang chờ được khám phá, được sắp xếp theo giá trị $d[v]$ tăng dần để tối ưu thời gian trích xuất đỉnh có khoảng cách nhỏ nhất.

3.7.2 Mã giả thuật toán

Chi tiết các bước thực thi của thuật toán để tìm đường đi ngắn nhất giữa một cặp điểm cụ thể được trình bày trong Thuật toán 1.

Algorithm 1 Thuật toán Dijkstra cơ bản

Require: Đồ thị $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ với trọng số $w \geq 0$, đỉnh xuất phát v_s , đỉnh đích v_t .

Ensure: Khoảng cách ngắn nhất từ v_s đến v_t .

```
1: Khởi tạo  $d[v] \leftarrow \infty$  cho mọi  $v \in \mathcal{V}$ 
2:  $d[v_s] \leftarrow 0$ 
3: Khởi tạo hàng đợi ưu tiên  $Q$ , chèn  $(v_s, 0)$  vào  $Q$ 
4: while  $Q$  không rỗng do
5:    $u \leftarrow$  đỉnh lấy ra từ  $Q$  có  $d[u]$  nhỏ nhất
6:   if  $u = v_t$  then
7:     break
8:   end if
9:   for all đỉnh kề  $v$  của  $u$  do
10:     $alt \leftarrow d[u] + w(u, v)$ 
11:    if  $alt < d[v]$  then
12:       $d[v] \leftarrow alt$ 
13:      Cập nhật hoặc chèn  $(v, d[v])$  vào hàng đợi ưu tiên  $Q$ 
14:    end if
15:  end for
16: end while
17: return  $d[v_t]$ 
```

3.7.3 Đánh giá độ phức tạp và Hạn chế

Nếu sử dụng cấu trúc dữ liệu Hàng đợi ưu tiên dựa trên Min-Heap (Binary Heap), thuật toán mất thời gian $\mathcal{O}(\log |\mathcal{V}|)$ để trích xuất đỉnh và cập nhật khoảng cách. Do mỗi đỉnh được trích xuất tối đa một lần và mỗi cạnh được duyệt tối đa một lần, tổng độ phức tạp thời gian của thuật toán trong trường hợp xấu nhất là $\mathcal{O}((|\mathcal{V}| + |\mathcal{E}|) \log |\mathcal{V}|)$.

Hạn chế trên mạng lưới thực tế: Mặc dù đảm bảo tính tối ưu tuyệt đối về mặt toán học, thuật toán Dijkstra nguyên thủy mang một nhược điểm lớn khi áp dụng vào các hệ thống bản đồ quy mô lớn. Do bản chất khám phá không gian theo mọi hướng (đẳng hướng - isotropic search) từ điểm xuất phát, thuật toán sẽ lan truyền theo dạng hình tròn đồng tâm và phải duyệt qua một lượng khổng lồ các đỉnh không liên quan trước khi chạm đến điểm đích. Điều này tạo ra nút thắt cổ chai về mặt hiệu suất, khiến thời gian truy vấn trên các đồ thị có hàng triệu nút giao thông trở nên quá chậm so với yêu cầu phản hồi trực tuyến. Hạn chế cốt lõi này chính là động lực để tích hợp các cấu trúc phân cấp đồ thị, từ đó hình thành nên giải thuật Multilevel Dijkstra nhằm thu hẹp triệt để không gian tìm kiếm.

3.8 Giải thuật Contraction Hierarchies (CH)

Như đã phân tích ở Mục 3.7, thuật toán Dijkstra nguyên thủy bộc lộ rõ giới hạn về mặt hiệu năng do bản chất tìm kiếm đẳng hướng, khiến nó phải duyệt qua một không gian cực kỳ lớn trên các đồ thị quy mô thực tế. Để khắc phục bài toán này mà không làm mất đi tính tối ưu của lộ trình, giải thuật Contraction Hierarchies (CH) [30] đã được đề xuất. Đây là một kỹ thuật tăng tốc dựa trên phân cấp (Hierarchical Routing), tận dụng tính chất tự nhiên của mạng lưới đường bộ: các chuyến đi đường dài thường ưu tiên sử dụng các tuyến đường huyết mạch (quan trọng) thay vì các con đường nhỏ lẻ.

Giải thuật CH bao gồm hai giai đoạn tách biệt: Giai đoạn tiền xử lý (Preprocessing Phase) và Giai đoạn truy vấn (Query Phase).

3.8.1 Giai đoạn tiền xử lý: Co đỉnh và Thêm cạnh tắt

Khái niệm cốt lõi của CH là việc gán cho mỗi đỉnh $v \in \mathcal{V}$ trong đồ thị một mức độ quan trọng (Node Ordering). Quá trình đánh giá này tạo ra một hàm song ánh $L : \mathcal{V} \rightarrow \{1, \dots, |\mathcal{V}|\}$, trong đó $L(v)$ biểu diễn thứ bậc ưu tiên của đỉnh v . Các đỉnh nằm ở ngõ cụt hoặc hẻm nhỏ sẽ có mức ưu tiên thấp, trong khi các nút giao thông lớn hoặc trạm thu phí cao tốc sẽ có mức ưu tiên cao.

Sau khi xác định được thứ tự, đồ thị sẽ trải qua quá trình **co đỉnh** (Node Contraction). Thuật toán sẽ lần lượt loại bỏ (co) các đỉnh theo thứ tự ưu tiên từ thấp đến cao. Khi một đỉnh v bị co, để không làm thay đổi khoảng cách ngắn nhất giữa các đỉnh còn lại trong đồ thị, thuật toán sẽ chèn thêm các **cạnh tắt** (Shortcuts).

Cụ thể, giả sử đỉnh v đang bị co, với mọi cặp đỉnh lân cận (u, w) sao cho có cạnh nối từ u đến v và từ v đến w , thuật toán sẽ kiểm tra xem đường đi qua v có phải là đường đi ngắn nhất duy nhất từ u đến w hay không. Nếu đúng, một cạnh tắt (u, w) sẽ được thêm vào đồ thị với trọng số được tính bằng công thức:

$$w(u, w) = w(u, v) + w(v, w)$$

Để quyết định có nên thêm cạnh tắt hay không, CH sử dụng kỹ thuật tìm kiếm cục bộ gọi là **Witness Search** (Tìm kiếm đường đi chứng minh). Đường đi chứng minh là một đường đi từ u đến w không đi qua v mà có tổng trọng số nhỏ hơn hoặc bằng $w(u, v) + w(v, w)$. Nếu tồn tại đường đi chứng minh, việc thêm cạnh tắt là không cần thiết.

Tiêu chí xếp hạng thứ tự đỉnh (Heuristics): Việc tính toán thứ tự ưu tiên là yếu tố quyết định hiệu suất của CH. Thông thường, độ ưu tiên $P(v)$ của một đỉnh v được tổng hợp từ các chỉ số đánh giá (heuristics) sau:

- **Edge Difference (Độ chênh lệch cạnh - ED):** Biểu diễn sự thay đổi về số lượng cạnh của đồ thị nếu v bị co.

$$ED(v) = |\text{Shortcuts}(v)| - |\text{IncidentEdges}(v)|$$

Đỉnh có $ED(v)$ càng nhỏ càng được ưu tiên co trước để tránh làm phình to dung lượng đồ thị.

- **Deleted Neighbors (Số lượng đỉnh kề đã xóa - DN):** Khuyến khích việc co các đỉnh phân bố đều trên toàn đồ thị, tránh việc co tập trung cục bộ tại một khu vực gây mất cân bằng cấu trúc phân cấp.

3.8.2 Giai đoạn truy vấn: Tìm kiếm hai chiều trên đồ thị phân cấp

Sau giai đoạn tiền xử lý, ta thu được một đồ thị mở rộng $\mathcal{G}' = (\mathcal{V}, \mathcal{E} \cup \mathcal{E}_{\text{shortcuts}})$. Đồ thị này được chia làm hai đồ thị có hướng dựa trên bậc phân cấp của các đỉnh:

- **Đồ thị hướng lên (Upward Graph) $\mathcal{G}^\uparrow$:** Chỉ chứa các cạnh nối từ đỉnh có thứ bậc thấp lên đỉnh có thứ bậc cao hơn.

$$\mathcal{G}^\uparrow = (\mathcal{V}, \mathcal{E}^\uparrow) \text{ với } \mathcal{E}^\uparrow = \{(u, v) \in \mathcal{E} \cup \mathcal{E}_{\text{shortcuts}} \mid L(u) < L(v)\}$$

- **Đồ thị hướng xuống (Downward Graph) $\mathcal{G}^\downarrow$:** Chỉ chứa các cạnh nối từ đỉnh có thứ bậc cao xuống đỉnh có thứ bậc thấp hơn.

$$\mathcal{G}^\downarrow = (\mathcal{V}, \mathcal{E}^\downarrow) \text{ với } \mathcal{E}^\downarrow = \{(u, v) \in \mathcal{E} \cup \mathcal{E}_{\text{shortcuts}} \mid L(u) > L(v)\}$$

Giai đoạn truy vấn để tìm đường đi ngắn nhất giữa đỉnh xuất phát v_s và đỉnh đích v_t được thực hiện bằng thuật toán **Dijkstra hai chiều (Bidirectional Dijkstra)**. Điểm đột phá của CH là quá trình tìm kiếm được giới hạn nghiêm ngặt theo hướng của đồ thị phân cấp:

- **Luồng tìm kiếm tiến (Forward Search):** Xuất phát từ v_s , chỉ được phép di chuyển trên đồ thị hướng lên $\mathcal{G}^\uparrow$.
- **Luồng tìm kiếm lùi (Backward Search):** Xuất phát từ v_t , đi ngược chiều các cạnh nhưng cũng chỉ được phép di chuyển trên đồ thị hướng lên $\mathcal{G}^\uparrow$ (nghĩa là duyệt các cạnh trong $\mathcal{G}^\downarrow$ theo chiều ngược lại).

Khoảng cách ngắn nhất cuối cùng là giá trị cực tiểu của tổng khoảng cách từ hai luồng tìm kiếm chạm nhau tại một đỉnh u có thứ bậc cao nhất trên lộ trình:

$$d(v_s, v_t) = \min_{u \in \mathcal{V}} \{d^\uparrow(v_s, u) + d^\downarrow(v_t, u)\}$$

Trong đó, $d^\uparrow(v_s, u)$ là khoảng cách tìm được từ luồng tiến, và $d^\downarrow(v_t, u)$ là khoảng cách tìm được từ luồng lùi.

3.8.3 Đánh giá và Hạn chế trên môi trường thực tế

Ưu điểm: Thuật toán Contraction Hierarchies tạo ra một bước nhảy vọt về hiệu năng. Nhờ việc bỏ qua hàng loạt các con đường nhỏ thông qua các cạnh tắt, không gian duyệt của thuật toán được thu hẹp tối đa. Thực nghiệm cho thấy, thời gian truy vấn của CH trên các bản đồ quy mô châu lục (hàng chục triệu đỉnh) chỉ diễn ra trong vài trăm micro giây, nhanh hơn hàng ngàn lần so với thuật toán Dijkstra nguyên thủy.

Hạn chế và Nguyên nhân dẫn đến sự ra đời của CRP: Bất chấp tốc độ truy vấn vượt trội, kiến trúc của CH bộc lộ nhược điểm chí mạng trong các ứng dụng điều hướng thời gian thực (Real-time Navigation). Nguyên nhân cốt lõi nằm ở quá trình tiền xử lý: các cạnh tắt (shortcuts) được tính toán và lưu trữ cố định dựa trên trọng số tĩnh ban đầu (ví dụ: chiều dài đoạn đường).

Cấu trúc phân cấp của CH có tính phụ thuộc toàn cục rất cao. Một cạnh tắt ở cấp độ cao có thể là sự tổng hợp của hàng chục cạnh nhỏ ở cấp độ thấp. Khi tình trạng giao thông thay đổi (ví dụ: xảy ra ùn tắc trên một đoạn đường nhỏ khiến thời gian di chuyển tăng đột biến), trọng số của cạnh đó bị thay đổi. Điều này dẫn đến hiệu ứng domino, làm sai lệch giá trị của tất cả các cạnh tắt nằm ở cấp độ cao hơn có bắc cầu qua đoạn đường bị kẹt đó. Để đảm bảo tính chính xác, giải thuật CH không có cách nào khác ngoài việc phải cập nhật lại một lượng khổng lồ các đỉnh hoặc phải khởi động lại toàn bộ quy trình co đỉnh từ đầu, tiêu tốn hàng giờ đồng hồ trên các hệ thống lớn.

Chính sự thiếu linh hoạt trong việc tùy chỉnh trọng số động này đã tạo ra khoảng trống kỹ thuật, thúc đẩy sự ra đời của các giải thuật dựa trên cấu trúc phân hoạch đồ thị như Customizable Route Planning (CRP) nhằm giải quyết triệt để vấn đề phản hồi thời gian thực.

3.9 Kiến trúc định tuyến Customizable Route Planning (CRP)

Như đã phân tích ở Mục 3.7, thuật toán Dijkstra truyền thống gặp giới hạn lớn về mặt hiệu suất khi triển khai trên các mạng lưới đường bộ quy mô thực tế. Mặc dù các kỹ thuật tăng tốc độ tìm đường như Contraction Hierarchies hay Transit Node Routing đã giải quyết được vấn đề tốc độ bằng cách tiền xử lý cấu trúc đồ thị, chúng lại bộc lộ một nhược điểm chí mạng: sự phụ thuộc chặt chẽ vào trọng số cạnh. Khi trọng số thay đổi (do kẹt xe, tai nạn, hay thay đổi vận tốc tối đa), các giải thuật này buộc phải tính toán lại toàn bộ cấu trúc tiền xử lý từ đầu, tốn hàng giờ đồng hồ và hoàn toàn không khả thi cho các hệ thống giao thông thời gian thực.

Để khắc phục triệt để nghịch lý giữa tốc độ truy vấn và khả năng cập nhật,

kiến trúc Customizable Route Planning (CRP) đã được nhóm nghiên cứu của Microsoft đề xuất [11]. Điểm đột phá của CRP nằm ở việc tách biệt hoàn toàn cấu trúc hình học của mạng lưới khỏi các hàm chi phí. Quy trình hoạt động của CRP được chia thành ba giai đoạn hoàn toàn độc lập: Tiền xử lý cấu trúc, Tùy chỉnh trọng số và Truy vấn.

3.9.1 Giai đoạn 1: Tiền xử lý cấu trúc

Đây là giai đoạn phức tạp nhất nhưng chỉ cần thực hiện duy nhất một lần khi hệ thống nạp dữ liệu bản đồ. Giai đoạn này hoàn toàn không phụ thuộc vào trọng số giao thông, mà chỉ dựa trên cấu trúc topo của đồ thị gốc $\mathcal{G} = (\mathcal{V}, \mathcal{E}, len)$. Mục tiêu là xây dựng một cấu trúc phân hoạch đa tầng lồng nhau để nén không gian đồ thị.

- **Định nghĩa Phân hoạch:** Một phân hoạch $\mathcal{P} = \{C_1, C_2, \dots, C_k\}$ của tập đỉnh $\mathcal{V}$ là một tập hợp các ô rời rạc sao cho $\bigcup_{i=1}^k C_i = \mathcal{V}$. Để kiểm soát khối lượng tính toán, mỗi ô ở cấp độ l được ràng buộc số lượng đỉnh tối đa không vượt quá một hằng số U^l cho trước.
- **Tính chất lồng nhau:** Một phân hoạch đa tầng $\mathcal{MLP} = \{\mathcal{P}^1, \dots, \mathcal{P}^L\}$ gồm L cấp độ phải thỏa mãn điều kiện:

$$\forall u, v \in \mathcal{V}, l \in \{1, \dots, L-1\} \text{ nếu } cell_l(u) = cell_l(v) \text{ thì } cell_{l+1}(u) = cell_{l+1}(v)$$

Điều này có nghĩa là một ô ở cấp độ cao luôn là hợp của các ô ở cấp độ thấp hơn.

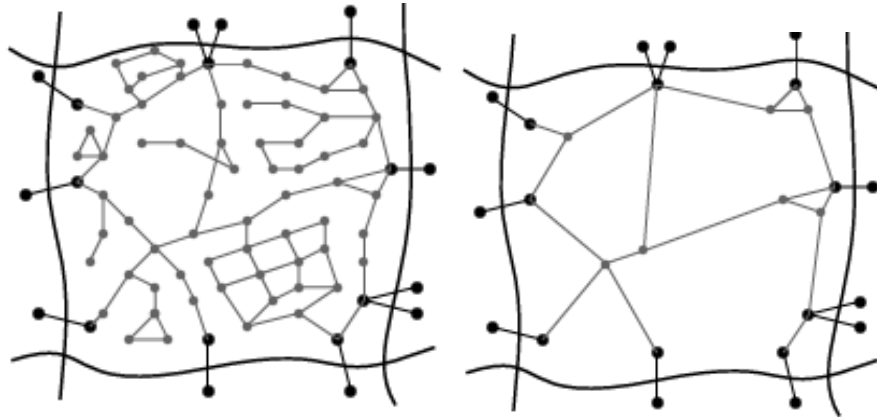
- **Đỉnh biên:** Một đỉnh $u \in C_i^l$ được gọi là đỉnh biên nếu tồn tại cạnh cắt (u, v) hoặc (v, u) nối với một đỉnh v thuộc ô khác ($cell_l(u) \neq cell_l(v)$). Tập các đỉnh biên của ô C_i^l ký hiệu là B_i^l .

Dựa trên các đỉnh biên này, CRP thiết lập trước các kết nối ảo (gọi là topology lớp phủ) giữa mọi cặp đỉnh biên trong cùng một ô. Đây chính là bộ khung để xây dựng các cạnh tắt sau này.

3.9.2 Giai đoạn 2: Tùy chỉnh trọng số

Khi tình trạng giao thông thực tế được đưa vào hệ thống, trọng số của các cạnh gốc trong đồ thị bắt đầu thay đổi. Giai đoạn tùy chỉnh sẽ được kích hoạt để tính toán lại trọng số cho các cạnh tắt trên đồ thị lớp phủ.

Như thể hiện trong Hình 9, quá trình tùy chỉnh thực chất là việc xây dựng một đồ thị lớp phủ đè lên đồ thị gốc nhằm trừu tượng hóa các chi tiết nội bộ. Quá trình này diễn ra theo nguyên lý từ dưới lên:



(a) Đồ thị gốc nội bộ của một ô [11] (b) Đồ thị lớp phủ [11]

Hình 9: Quá trình nén dữ liệu từ đồ thị gốc sang đồ thị lớp phủ

1. **Tại cấp độ cơ sở (cấp độ 1):** Thuật toán Dijkstra sẽ chạy nội bộ trong từng ô nhỏ để tìm khoảng cách ngắn nhất giữa tất cả các cặp đỉnh biên của ô đó. Trọng số này được gán cho các cạnh tắt, đại diện cho khoảng cách ngắn nhất xuyên qua ô:

$$dist_{\overline{CG}}(u, v) = dist_{CG}(u, v) \quad \forall u, v \in B_i^l$$

2. **Tại các cấp độ cao hơn (cấp độ $l > 1$):** Thuật toán không duyệt trên đồ thị gốc nữa mà tính toán khoảng cách nội bộ của ô lớn bằng cách kết hợp các cạnh tắt của các ô nhỏ bên trong nó.

Nhờ giới hạn kích thước của các ô và tính chất hoạt động hoàn toàn độc lập, quá trình tính toán này có thể được thực thi song song trên nhiều luồng. Khi một sự cố giao thông xảy ra, hệ thống chỉ cần tính toán lại nội bộ trong các ô bị ảnh hưởng, thay vì phải xử lý lại toàn bộ đồ thị, cho phép cập nhật dữ liệu trên quy mô lớn chỉ trong vài mili giây.

3.9.3 Giai đoạn 3: Truy vấn lộ trình

Sau khi đồ thị lớp phủ đã được cập nhật trọng số giao thông thời gian thực, giai đoạn truy vấn diễn ra để tìm đường từ điểm xuất phát v_s đến điểm đích v_t .

Thuật toán truy vấn của CRP chạy trên đồ thị đa tầng với một bộ quy tắc cắt tĩa chặt chẽ: thay vì duyệt qua mọi con đường chi tiết, thuật toán cố gắng leo lên cấp độ phân hoạch cao nhất có thể. Nếu thuật toán đang duyệt tới một đỉnh nằm ở biên của một ô, và phát hiện ra cả điểm xuất phát lẫn điểm đích đều không nằm trong ô này, nó sẽ từ chối đi vào bên trong ô. Thay vào đó, nó sử dụng cạnh tắt của ô đó để nhảy thẳng sang đỉnh biên phía bên kia.



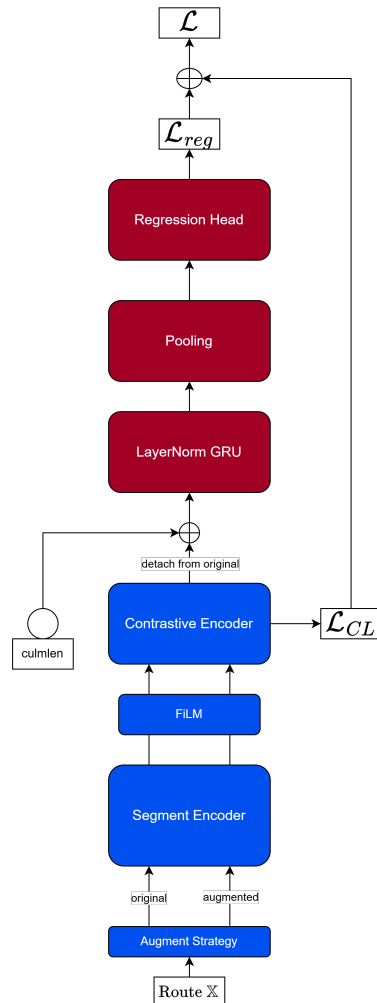
Nhờ cơ chế nhảy qua các khối không gian rộng lớn, CRP tạo ra sự cân bằng hoàn hảo: tốc độ truy vấn cực nhanh tương đương các giải thuật tĩnh, đồng thời duy trì khả năng cập nhật động theo thời gian thực, mở ra kỷ nguyên mới cho các hệ thống bản đồ số.

4

Mô hình đề xuất

Trong chương này, nghiên cứu trình bày kiến trúc mô hình đề xuất nhằm giải quyết những hạn chế còn tồn đọng của mô hình MulT-TTE. Trên thực tế, các phân đoạn đường bộ (road segments) dù có sự tương đồng về cấu trúc hình học nhưng lại chịu sự chi phối của các yếu tố ngoại cảnh khác nhau, dẫn đến sự khác biệt đáng kể về đặc trưng và lưu lượng giao thông.

Nhằm khắc phục vấn đề này, mô hình được thiết kế để khai thác sâu yếu tố ngữ cảnh không gian thông qua việc tích hợp thông tin từ các Điểm quan tâm (Points of Interest - POIs). Bên cạnh đó, nghiên cứu áp dụng phương pháp học tương phản thuộc nhánh học bán tự giám sát (Semi Self-supervised Learning) vào quá trình huấn luyện. Cơ chế này đóng vai trò tinh chỉnh khả năng biểu diễn đặc trưng (feature representation), đồng thời giúp mô hình xử lý hiệu quả tình trạng không nhất quán và nhiễu dữ liệu khi trích xuất đặc trưng mạng lưới đường bộ từ OpenStreetMap (OSM).



Hình 10: Kiến trúc tổng quan của mô hình PoI-CL-TTE. Các thành phần được tô màu xanh bao gồm chiến lược huấn luyện, bộ mã hóa đoạn đường, mô-đun FiLM và bộ mã hóa tương phản là các thành phần đóng góp chính của nghiên cứu.

Cụ thể chúng tôi đề xuất mô hình PoI-CL-TTE, có cấu trúc tổng quát được chia thành bốn mô-đun chính:

- **Mô-đun trích xuất đặc trưng đoạn đường (Segment Encoder):** mã hóa từng phân đoạn đường thành véc-tơ đặc trưng, tích hợp thông tin POI và điều chỉnh theo thời gian khởi hành.
- **Mô-đun điều chế đặc trưng theo thời gian bắt đầu (FiLM):** Tiếp nhận tập véc-tơ đặc trưng đoạn đường và thực hiện điều chế tuyến tính dựa trên véc-tơ thời gian khởi hành, tạo ra biểu diễn mang tính thời điểm.

- **Mô-đun học tương phản** (Contrastive Encoder): tính chỉnh không gian biểu diễn kết hợp hàm mất mát tương phản.
- **Mô-đun dự đoán thời gian di chuyển** (Temporal Encoder & Regression): mã hóa chuỗi phân đoạn theo trình tự của các bước đoạn đường, sau đó tổng hợp qua cơ chế Gộp chú ý (Attention pooling) và hồi quy để đưa ra dự đoán cuối cùng.

Đóng góp chính của nghiên cứu gồm:

- Đề xuất cơ chế mã hóa và tích hợp thông tin Điểm quan tâm (Points of Interest - POI) vào từng phân đoạn đường.
- Tối ưu hóa không gian biểu diễn bằng Học tương phản (Contrastive Learning)
- Phân tích ảnh hưởng và áp dụng kỹ thuật điều chế trên đặc trưng thời gian bắt đầu di chuyển.

4.1 Mô-đun trích xuất đặc trưng đoạn đường (Segment Encoder)

Mô-đun này có nhiệm vụ chuyển hóa các đặc trưng thô đầu vào thành một véc-tơ biểu diễn. Mô-đun nhận đầu vào sau bước Tăng cường (đề cập tại Mục 4.3.1) gồm 2 tập véc-tơ chứa các đặc trưng thô $\{x_t^{original}\}_{t=1}^T$ và $\{x_t^{augment}\}_{t=1}^T$. Các đặc trưng đầu vào bao gồm độ dài phân đoạn, loại đường và thông tin POI lân cận, được xử lý theo các nhánh riêng biệt trước khi hợp nhất:

- **Độ dài đoạn đường:** Đại lượng vô hướng này có biên độ biến thiên rộng. Do đó, nhóm tác giả áp dụng chuẩn hóa Standard Scaler, đưa hai giá trị này về phân phối xấp xỉ chuẩn với $\mu = 0, \sigma = 1$.
- **Loại đường:** Theo phân loại của OSM, mỗi đoạn đường được gán hai thuộc tính loại đường. Mỗi thuộc tính được mã hóa độc lập thông qua lớp nhúng (Embedding Layer), sau đó hai véc-tơ nhúng được nối ghép với nhau.
- **Tốc độ tối đa:** Thuộc tính này phản ánh trực tiếp năng lực thông hành của đoạn đường. Do tốc độ tối đa (*maxspeed*) trong thực tế chỉ nhận một tập giá trị rời rạc hữu hạn (ví dụ: 30, 40, 50, 60 km/h), hai thuộc tính này được rời rạc hóa thành các xô và mã hóa thông qua lớp nhúng (Embedding Layer), tương tự như cách xử lý loại đường.
- **Thông tin POI lân cận:** Các POI trong vùng lân cận của mỗi đoạn đường được mã hóa thành véc-tơ biểu diễn thông qua bộ mã hóa POI chuyên biệt. Chi tiết về cơ chế này được trình bày trong Mục 4.1.

Sau bước mã hóa các đặc trưng thô này, nghiên cứu thu được 2 tập véc-tơ đặc trưng đoạn đường $\{x_t^{(1)original}\}_{t=1}^T$ và $\{x_t^{(1)augment}\}_{t=1}^T$

Bộ mã hóa PoI (PoI Encoder)

Mỗi đoạn đường được đặc trưng bởi một véc-tơ hiện diện $\mathbf{p}_{t,g} \in \{0, 1\}$, trong đó phần tử thứ g biểu thị sự hiện diện PoI thuộc nhóm danh mục g nằm trong vùng lân cận của đoạn đường đó, với G là tổng số nhóm danh mục.

Mỗi nhóm danh mục g được ánh xạ thành một véc-tơ nhúng học được $\mathbf{e}_g \in \mathbb{R}^D$ thông qua lớp nhúng (Embedding Layer). Sau đó biểu diễn PoI được tổng hợp qua hai luồng song song:

$$\mathbf{r}_t^{\text{mean}} = \frac{\sum_{g=1}^G p_{t,g} \cdot \mathbf{e}_g}{\sum_{g=1}^G p_{t,g}} \quad (39)$$

$$\mathbf{r}_t^{\text{max}} = \max_{g:p_{t,g}=1} \mathbf{e}_g \quad (40)$$

Luồng Gộp trung bình phản ánh chức năng trung bình của khu vực xung quanh, ví dụ một đoạn đường có thể được biểu diễn là 50% dân cư, 50% thương mại thay vì bị chi phối bởi mật độ PoI tổng thể vốn biến thiên lớn giữa nội đô và ngoại ô. Luồng Gộp tối đa nắm bắt danh mục PoI nổi bật nhất trong vùng lân cận, ví dụ sự hiện diện của trường học hay bệnh viện có thể đặc trưng hóa rõ ràng hành vi giao thông của đoạn đường đó.

Hai luồng được nối ghép và đưa qua khối chiếu tuyến tính:

$$\mathbf{x}_t^{\text{poi}} = \text{Activation}(\mathbf{W}([\mathbf{r}_t^{\text{mean}}; \mathbf{r}_t^{\text{max}}]) + \mathbf{b}) \quad (41)$$

4.2 Mô-đun điều chế đặc trưng theo thời gian bắt đầu (FiLM)

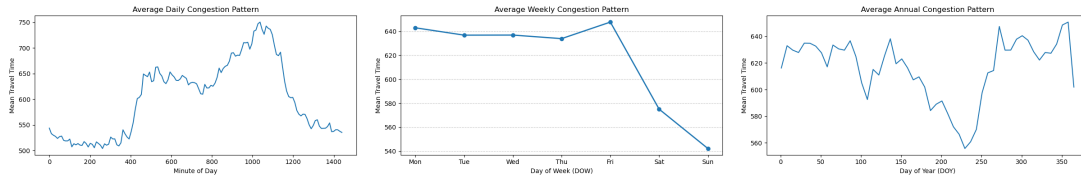
Mô-đun này có nhiệm vụ tinh chỉnh 2 tập véc-tơ $\{x_t^{(1)original}\}_{t=1}^T$ và $\{x_t^{(1)augment}\}_{t=1}^T$, thu được từ Segment Encoder (xem Mục 4.1), qua phép biến đổi tuyến tính so với thời gian bắt đầu nhằm ánh xạ ảnh hưởng của ngữ cảnh thời gian.

4.2.1 Mã hóa thời gian bắt đầu

Thời gian bắt đầu hành trình là một biến số quan trọng, có tác động mang tính hệ thống đến tổng thời gian di chuyển thực tế. Qua phân tích thực nghiệm trên tập dữ liệu (Hình 11), nghiên cứu nhận diện ba đặc tính chu kỳ lồng ghép quyết định trạng thái lưu thông của mạng lưới đường bộ:

- **Chu kỳ theo ngày:** Phản ánh biến động giao thông mạnh mẽ nhất gắn liền với nhịp sống đô thị. Thời gian di chuyển trung bình đạt mức thấp nhất vào rạng sáng và tịnh tiến đến đỉnh điểm vào khung giờ cao điểm chiều (khoảng 17:30) — thời điểm diễn ra sự bùng nổ lưu lượng do nhu cầu tan sở tại các đô thị lớn. Đáng chú ý, sự tương đồng về đặc trưng giữa thời điểm cuối ngày (23:59) và đầu ngày kế tiếp (00:00) khẳng định tính liên tục và chu kỳ chặt chẽ của biến số này.
- **Chu kỳ theo tuần:** Thể hiện sự phân hóa rõ rệt trong hành vi di chuyển giữa ngày làm việc và ngày nghỉ. Trong khi các ngày từ Thứ Hai đến Thứ Sáu duy trì mức độ ùn tắc ổn định ở ngưỡng cao, thì vào Thứ Bảy và Chủ Nhật, áp lực lên mạng lưới hạ tầng giảm mạnh, dẫn đến thời gian di chuyển trung bình thấp hơn đáng kể.
- **Chu kỳ theo năm:** Minh họa các biến động có quy luật dài hạn theo mùa. Những thay đổi này thường liên quan mật thiết đến lịch trình học tập (các kỳ nghỉ hè/đông), các dịp lễ tết và sự chuyển biến của điều kiện thời tiết, gây ảnh hưởng gián tiếp đến tổng cung và cầu trong hệ thống giao thông.

Ba chu kỳ này có biên độ và pha khác nhau, không thể được nắm bắt bởi một hàm mã hóa đơn lẻ. Do đó, nghiên cứu mã hóa thời gian khởi hành thành véc-tơ $\mathbf{c}$ bằng cách khai thác đồng thời ba chu kỳ độc lập thông qua ba module Mã hóa thời gian theo chu kỳ riêng biệt.



Hình 11: Biến động thời gian di chuyển trung bình theo giờ trong ngày (trái), ngày trong tuần (giữa), và ngày trong năm (phải) trên tập dữ liệu huấn luyện.

Bộ mã hóa thời gian theo chu kỳ (Cyclical Time Encoder)

Mô-đun Bộ mã hóa thời gian theo chu kỳ nhận đầu vào là một giá trị vô hướng x biểu diễn vị trí trong chu kỳ và ánh xạ thành véc-tơ $\mathbf{h} \in \mathbb{R}^{d_{\text{time}}}$ theo công thức:

$$\mathbf{c}_a = \left[\sin\left(\frac{2\pi x}{P_a} \cdot \boldsymbol{\omega}\right); \cos\left(\frac{2\pi x}{P_a} \cdot \boldsymbol{\omega}\right) \right] \in \mathbb{R}^{d_{\text{time}}} \quad (42)$$

trong đó $\boldsymbol{\omega} \in \mathbb{R}^{d_{\text{time}}/2}$ là tập tần số học được, khởi tạo theo thang logarithm:

$$\omega_k = \exp\left(\frac{k}{d_{\text{time}}/2 - 1} \cdot \ln\left(\frac{d_{\text{time}}}{2}\right)\right), \quad k = 0, \dots, \frac{d_{\text{time}}}{2} - 1 \quad (43)$$

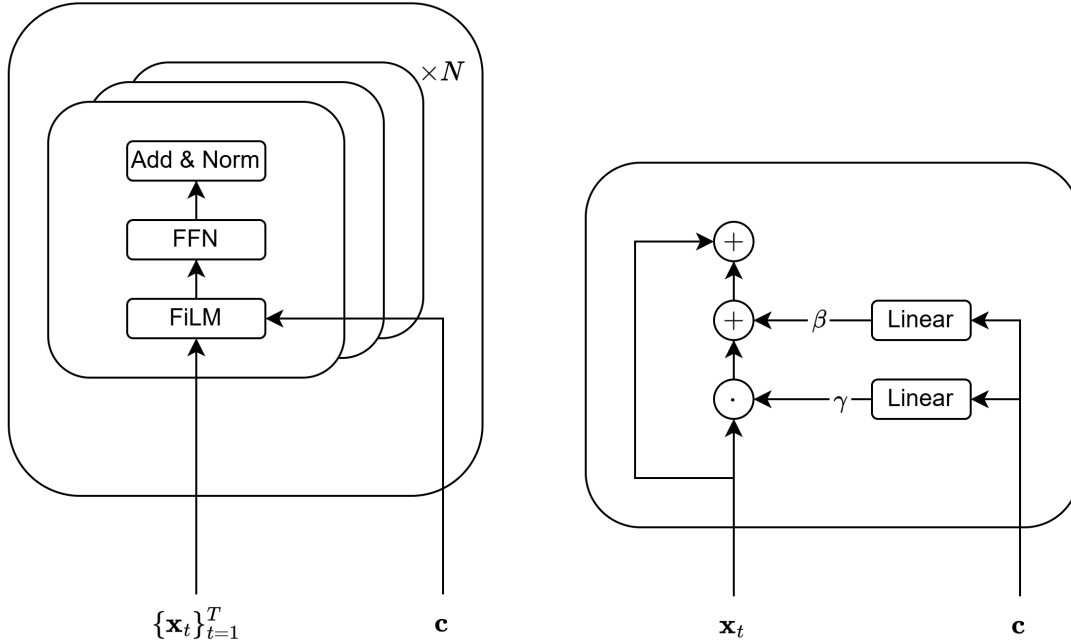
Các tần số thấp nắm bắt xu hướng tổng quát trong chu kỳ, như sáng so với tối, ngày thường so với cuối tuần, trong khi các tần số cao phân biệt các mốc thời gian chi tiết hơn.

Ba véc-tơ đầu ra sau đó được nối ghép thành véc-tơ thời gian tổng hợp:

$$\mathbf{c} = [\mathbf{c}_{\text{day}}; \mathbf{c}_{\text{week}}; \mathbf{c}_{\text{year}}] \in \mathbb{R}^{d_{st}} \quad (44)$$

Véc-tơ $\mathbf{c}$ này được sử dụng làm tín hiệu điều kiện cho cơ chế FiLM (Mục 4.2.2).

4.2.2 Bộ mã hóa FiLM



Hình 12: Kiến trúc Bộ mã hóa FiLM (trái) và kiến trúc FiLM (phải) [10]

Sau Bộ mã hóa đoạn đường, 2 tập véc-tơ $\{\mathbf{x}_t^{(1)original}\}_{t=1}^T$ và $\{\mathbf{x}_t^{(1)augment}\}_{t=1}^T$ được điều kiện hóa theo thông tin thời gian khởi hành thông qua cơ chế Điều chế

tuyến tính theo đặc trưng (Feature-wise Linear Modulation). Động lực cốt lõi của cơ chế này xuất phát từ thực tế rằng cùng một đoạn đường có thể mang đặc tính giao thông hoàn toàn khác nhau tùy theo thời điểm trong ngày hay ngày trong tuần. Ví dụ một đoạn đường trước cổng trường sẽ tắc nghẽn vào giờ tan học nhưng thông thoáng vào buổi tối. Do đó, thay vì học một biểu diễn tĩnh cho mỗi đoạn đường, mô hình cần khả năng điều chỉnh biểu diễn đó theo ngữ cảnh thời gian.

Cụ thể, véc-tơ thời gian $\mathbf{c}$ được sử dụng để tạo ra 2 tham số điều chế γ và β , từ đó tỉ lệ và tịnh tiến từng chiều của $\mathbf{f}_t$:

$$\mathbf{x}_t^{(2)(original||augment)} = \gamma(\mathbf{c}) \odot \mathbf{x}_t^{(1)(original||augment)} + \beta(\mathbf{c}) \quad (45)$$

Về mặt bản chất, 2 tham số này mô phỏng hai hiệu ứng vật lý thực tế của thời gian lên trạng thái giao thông:

- γ : Tham số này đóng vai trò khuếch đại hoặc triệt tiêu tác động của một yếu tố không gian theo thời điểm. Chẳng hạn, đặc trưng "gần trường học" có loại đường "dân cư" sẽ được γ phóng to tác động vào giờ tan tầm, nhưng bị thu nhỏ về gần 0 vào ban đêm khi yếu tố này không còn gây kẹt xe.
- β : Tham số này đại diện cho tình trạng giao thông tổng thể của mạng lưới tại một khung giờ. Ví dụ, vào giờ cao điểm, β sẽ cộng thêm một mức "ùn tắc nền" chung cho mọi đoạn đường, độc lập với đặc điểm cấu trúc cục bộ của từng đoạn.

Sau đó, nghiên cứu sử dụng kết nối tắt để duy trì độ ổn định gradient trong quá trình huấn luyện, đồng thời cho phép thông tin từ các lớp trước được bảo toàn qua từng phân đoạn.

$$\mathbf{x}_t^{(3)(original||augment)} = \mathbf{x}_t^{(1)(original||augment)} + \mathbf{x}_t^{(2)(original||augment)} \quad (46)$$

Toàn bộ cơ chế này được áp dụng xen kẽ với các khối biến đổi tuyến tính, tạo ra 2 tập véc-tơ biểu diễn $\{x_t^{(4)original}\}_{t=1}^T$ và $\{x_t^{(4)augment}\}_{t=1}^T$ mang đầy đủ ngữ cảnh không gian lẫn thời gian.

4.3 Mô-đun học tương phản (Contrastive Encoder)

Mô-đun này tiếp nhận hai tập véc-tơ biểu diễn $\{x_t^{(4)original}\}_{t=1}^T$ và $\{x_t^{(4)augment}\}_{t=1}^T$ từ Bộ mã hóa FiLM và thực hiện tinh chỉnh không gian biểu diễn thông qua học tương phản [31]. Đầu ra của mô-đun là chuỗi biểu diễn ngữ cảnh hóa $\{\mathbf{h}_t^{original}\}_{t=1}^T$, được sử dụng trực tiếp cho bài toán dự đoán thời gian di chuyển (xem Mục 4.4).

4.3.1 Tăng cường dữ liệu

Trong quá trình huấn luyện, mỗi hành trình gốc $\{\mathbf{x}_t^{original}\}_{t=1}^T$ được kết hợp với một phiên bản tăng cường $\{\mathbf{x}_t^{augment}\}_{t=1}^T$ nhằm tạo ra hai khung nhìn khác nhau của cùng một tuyến đường.

Mục tiêu kỹ thuật. Việc xây dựng hai khung nhìn của cùng một hành trình giúp Bộ mã hóa Transformer học được các biểu diễn bất biến trước những biến đổi nhỏ của dữ liệu đầu vào. Thay vì phụ thuộc vào cách phân đoạn cụ thể của chuỗi đường, mô hình được định hướng học các đặc trưng cấu trúc mang tính ngữ nghĩa cao hơn của toàn bộ hành trình. Điều này góp phần cải thiện khả năng tổng quát hóa khi mô hình được áp dụng trên các tuyến đường hoặc thành phố chưa xuất hiện trong tập huấn luyện.

Mục tiêu về độ bền vững dữ liệu. Chiến lược tăng cường còn được xây dựng dựa trên đặc điểm của dữ liệu OpenStreetMap (OSM). Do OSM là hệ thống bản đồ cộng đồng, dữ liệu thường tồn tại các sai khác trong quá trình biểu diễn topology, phân đoạn đường hoặc gán thuộc tính. Cùng một tuyến đường có thể được biểu diễn thành số lượng đoạn khác nhau giữa các khu vực hoặc các phiên bản dữ liệu khác nhau. Ngoài ra, thông tin PoI và các thuộc tính hình học cũng có thể chứa nhiều hoặc thiếu nhất quán. Việc huấn luyện mô hình nhận diện các phiên bản nhiễu của cùng một hành trình là tương đồng giúp tăng khả năng bền vững của biểu diễn học được trước các sai khác đặc trưng của dữ liệu OSM.

Chiến lược tăng cường. Để mô phỏng các biến thiên thường gặp trong dữ liệu bản đồ và quá trình map matching, luận văn áp dụng hai kỹ thuật tăng cường chính:

1. **Ngẫu nhiên gộp các đoạn đường kề nhau:** Mô phỏng sự khác biệt trong quá trình phân đoạn mạng lưới đường giữa các thành phố hoặc giữa các phiên bản dữ liệu OSM. Hai đoạn đường liền kề được gộp lại với một xác suất nhỏ nếu chúng có cùng loại đường và cùng nhóm tốc độ giới hạn. Độ dài của đoạn mới được tính bằng tổng độ dài của hai đoạn ban đầu, trong khi các đặc trưng PoI được hợp nhất theo phép OR nhị phân.
2. **Ngẫu nhiên tách các đoạn đường:** Mô phỏng hiện tượng một đoạn đường được chia thành nhiều đoạn nhỏ hơn do sự khác biệt trong quá trình map matching hoặc biểu diễn tô-pô. Một đoạn đường được chọn ngẫu nhiên để tách thành hai đoạn con với tổng chiều dài được bảo toàn. Các đặc trưng ngữ cảnh như loại đường, tốc độ giới hạn và PoI được sao chép cho cả hai đoạn nhằm duy trì tính nhất quán ngữ nghĩa.

4.3.2 Học tương phản

Hai tập véc-tơ đặc trưng $\{x_t^{(4)original}\}_{t=1}^T$ và $\{x_t^{(4)augment}\}_{t=1}^T$ được bổ sung mã hóa vị trí trước khi đưa vào Bộ mã hóa Transformer nhằm cho phép mỗi đoạn đường tổng hợp thông tin ngữ cảnh từ toàn bộ các đoạn đường còn lại trong hành trình. Đầu ra của Bộ mã hóa Transformer, $\{\mathbf{h}_t^{original||augment}\}_{t=1}^T$, được thực hiện phép gộp trung bình để thu được véc-tơ biểu diễn toàn cục cho toàn bộ chuyến đi. Ngoài ra, đầu ra của chuỗi đoạn đường gốc $\{\mathbf{h}_t^{original||augment}\}_{t=1}^T$ được truyền cho Mô-đun dự đoán thời gian di chuyển.

Để học biểu diễn có khả năng khái quát hóa giữa các thành phố và giảm phụ thuộc vào sự khác biệt tô-pô giữa các phiên bản OpenStreetMap (OSM), luận văn áp dụng phương pháp học tương phản dựa trên hàm mất mát InfoNCE. Với mỗi hành trình, một cặp mẫu dương được tạo ra thông qua các phép biến đổi dữ liệu như gộp hoặc tách ngẫu nhiên các đoạn đường. Sau khi đi qua Bộ ánh xạ (Các lớp tuyến tính chồng nhau) và chuẩn hóa ℓ_2 , độ tương đồng cosine giữa hai biểu diễn được tính như sau:

$$s_{ij} = \frac{z_i^\top z_j}{\tau} \quad (47)$$

Do dữ liệu hành trình thường chứa nhiều tuyến đường có mức độ chồng lấp cao, việc xem toàn bộ các mẫu còn lại trong bó là mẫu âm có thể tạo ra hiện tượng mẫu âm giả. Điều này đặc biệt phổ biến trong bài toán dự đoán thời gian di chuyển khi nhiều hành trình có thể chia sẻ chung các tuyến đường chính hoặc có thời gian di chuyển gần tương đương mặc dù không hoàn toàn trùng khớp về tô-pô.

Để giảm ảnh hưởng của hiện tượng này, luận văn áp dụng cơ chế gán trọng số cho các mẫu âm dựa trên mức độ chênh lệch thời gian di chuyển thực tế giữa hai hành trình. Các hành trình có thời gian di chuyển gần nhau sẽ được giảm mức độ đẩy tách trong không gian biểu diễn, trong khi các hành trình có thời gian di chuyển khác biệt lớn vẫn được xem là mẫu âm mạnh.

Trọng số giữa hai hành trình được xác định:

$$w_{ij} = 2\sigma(\tau|y_i - y_j|) - 1 \quad (48)$$

trong đó:

- y_i, y_j là thời gian di chuyển thực tế của hai hành trình,
- τ là hệ số điều chỉnh độ nhạy của hàm trọng số.
- σ là hàm sigmoid có công thức $\frac{1}{1+e^{-x}}$

Hàm trọng số được thiết kế sao cho $w_{ij} \in [0, 1)$ và tăng đơn điệu theo độ chênh lệch thời gian di chuyển giữa hai hành trình.

Khi hai hành trình có thời gian di chuyển gần tương đương ($|y_i - y_j| \rightarrow 0$), ta có $\sigma(0) = 0.5$ dẫn tới $w_{ij} = 2 \times 0.5 - 1 = 0$. Điều này đồng nghĩa với việc cặp mẫu âm tương ứng gần như không tạo ra lực đẩy trong không gian biểu diễn. Về trực giác, mô hình được phép duy trì mức độ tương đồng nhất định giữa các hành trình có đặc tính di chuyển gần giống nhau thay vì cưỡng ép chúng trở nên hoàn toàn tách biệt.

Ngược lại, khi độ chênh lệch thời gian di chuyển tăng lên ($|y_i - y_j| \rightarrow \infty$) thì $\sigma(\tau|y_i - y_j|) \rightarrow 1$ dẫn đến $w_{ij} \rightarrow 1$. Trong trường hợp này, cặp hành trình được xem như mẫu âm mạnh và đóng góp đầy đủ vào cơ chế đẩy tách của hàm InfoNCE.

Khi đó, hàm mất mát InfoNCE được điều chỉnh thành:

$$\mathcal{L}_{InfoNCE} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(s_{ii})}{\sum_{j \neq i} w_{ij} \exp(s_{ij})} \quad (49)$$

Sau khi áp dụng trọng số, thành phần mẫu âm trong hàm InfoNCE được điều chỉnh theo:

$$\mathcal{L}_{InfoNCE} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(s_{ii})}{\sum_{j \neq i} w_{ij} \exp(s_{ij})} \quad (50)$$

Cơ chế này cho phép mô hình học được không gian biểu diễn phản ánh tốt hơn mối quan hệ ngữ nghĩa giữa các hành trình, đồng thời tăng khả năng tổng quát hóa khi chuyển giao giữa các thành phố hoặc các nguồn dữ liệu OSM có mức độ không đồng nhất cao.

Ngoài ra, nhằm hạn chế hiện tượng suy biến biểu diễn, luận văn bổ sung thêm hai thành phần điều chuẩn gồm

Variance regularization. Để tránh hiện tượng các véc-tơ biểu diễn suy biến về cùng một giá trị hằng, mô hình duy trì độ lệch chuẩn của từng chiều đặc trưng lớn hơn một ngưỡng tối thiểu. Thành phần mất mát phương sai được định nghĩa:

$$\mathcal{L}_{var} = \frac{1}{D} \sum_{d=1}^D \max \left(0, 1 - \sqrt{\text{Var}(z_d)} + \epsilon \right) \quad (51)$$

trong đó:

- D là số chiều của véc-tơ biểu diễn,

- z_d là chiều đặc trưng thứ d ,
- ϵ là hằng số nhỏ nhằm đảm bảo ổn định số học.

Covariance regularization. Để giảm tương quan tuyến tính giữa các chiều đặc trưng và tăng tính đa dạng thông tin trong không gian biểu diễn, mô hình áp dụng điều chuẩn hiệp phương sai. Với ma trận hiệp phương sai:

$$C = \frac{(\mathbf{Z} - \bar{\mathbf{Z}})^T (\mathbf{Z} - \bar{\mathbf{Z}})}{N - 1} \quad (52)$$

trong đó:

- $\mathbf{Z} \in \mathbb{R}^{N \times D}$ là ma trận biểu diễn,
- N là số lượng mẫu trong batch,
- $\bar{\mathbf{Z}}$ là véc-tơ trung bình theo từng chiều đặc trưng.

Hàm mất mát hiệp phương sai được xác định trên các phần tử ngoài đường chéo của ma trận C :

$$\mathcal{L}_{cov} = \frac{1}{D(D-1)} \sum_{i \neq j} C_{ij}^2 \quad (53)$$

Mục tiêu của thành phần này là giảm phụ thuộc tuyến tính giữa các chiều đặc trưng, từ đó hạn chế hiện tượng các chiều biểu diễn học thông tin dư thừa.

Tổng hàm mất mát học tương phản được biểu diễn:

$$\mathcal{L}_{CL} = \mathcal{L}_{InfoNCE} + \lambda_{var} \mathcal{L}_{var} + \lambda_{cov} \mathcal{L}_{cov} \quad (54)$$

trong đó λ_{var} và λ_{cov} là các hệ số điều chỉnh mức độ ảnh hưởng của từng thành phần điều chuẩn.

4.4 Mô-đun dự đoán thời gian di chuyển

Mô-đun này tiếp nhận chuỗi biểu diễn đoạn đường $\{\mathbf{h}_t^{original}\}_{t=1}^T$ từ Mô-đun học tương phản, sau đó thực hiện 3 bước tuần tự:

1. **Tích hợp độ dài tích lũy:** đặc trưng đầu vào độ dài tích lũy của các phân đoạn sẽ được tích hợp lại vào chuỗi biểu diễn đoạn đường $\{\mathbf{h}_t^{original}\}_{t=1}^T$:

$$\mathbf{h}_t^{(1)} = \text{concatenate}(\mathbf{h}_t^{original}, \text{culmlen}_t) \quad (55)$$

với culmlen là độ dài tích lũy của 1 đoạn đường trong hành trình

2. **Mã hóa thời gian:** chuỗi $\{\mathbf{h}_t^{(1)}\}_{t=1}^T$ được đưa qua một mạng GRU để xử lý hiệu quả chuỗi.
3. **Tổng hợp và hồi quy:** đầu ra GRU được tổng hợp toàn cục theo cơ chế chú ý và đưa qua đầu hồi quy để dự đoán thời gian di chuyển $\hat{y}$.

4.4.1 Mã hóa chuỗi theo bước đoạn đường

Chuỗi biểu diễn được xử lý tuần tự qua Khối lặp tuần hoàn có cổng (Gated Recurrent Unit - GRU). Thay vì sử dụng GRU tiêu chuẩn, nghiên cứu áp dụng biến thể LayerNorm GRU trong đó chuẩn hóa lớp (Layer Normalization) được áp dụng lên các phép chiếu tuyến tính bên trong mỗi ô GRU trước khi tính cổng. Cụ thể, tại mỗi bước đoạn đường t , các phép biến đổi từ đầu vào và trạng thái ẩn được chuẩn hóa độc lập trước khi cộng lại:

$$\text{preact}_t = \text{LN}(\mathbf{W}_i \mathbf{s}_t) + \text{LN}(\mathbf{W}_h \mathbf{h}_{t-1}) \quad (56)$$

Động lực chính của thiết kế này xuất phát từ đặc thù của dữ liệu đầu vào: các hành trình trong cùng một batch có độ dài biến thiên lớn và phân phối đặc trưng không đồng đều do sự khác biệt về loại đường và mật độ POI. Trong điều kiện đó, GRU tiêu chuẩn dễ gặp hiện tượng bùng nổ hoặc triệt tiêu gradient qua nhiều bước đoạn đường. Chuẩn hóa lớp ổn định phân phối của các kích hoạt bên trong ô GRU tại mỗi bước, giúp gradient lan truyền ổn định hơn xuyên suốt chiều dài chuỗi mà không cần cắt gradient hay điều chỉnh tốc độ học đặc biệt.

4.4.2 Gộp chú ý (Attention Pooling)

Sau khi xử lý chuỗi, chuỗi trạng thái ẩn $\{\mathbf{h}_t\}_{t=1}^T$ cần được tổng hợp thành một véc-tơ cố định. Thay vì lấy trực tiếp trạng thái ẩn cuối cùng, vốn chỉ phản ánh các phân đoạn cuối hành trình, nghiên cứu sử dụng cơ chế chú ý với một véc-tơ truy vấn học được $\mathbf{q} \in \mathbb{R}^D$:

$$\mathbf{z}_{\text{seq}} = \text{Attention}(\mathbf{q}, \{\mathbf{h}_t\}, \{\mathbf{h}_t\}) \quad (57)$$

Véc-tơ truy vấn $\mathbf{q}$ không mang thông tin vị trí hay nội dung cụ thể, mà học cách tổng hợp thông tin từ toàn bộ chuỗi theo cách tối ưu nhất cho tác vụ dự đoán thời gian di chuyển.

4.4.3 Hồi quy dự đoán

Véc-tơ tổng hợp $\mathbf{z}_{\text{seq}}$ được nối ghép với véc-tơ thời gian khởi hành $\mathbf{c}$ trước khi đưa qua lớp hồi quy:

$$\hat{y} = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 [\text{LN}(\mathbf{z}_{\text{seq}}) \parallel \mathbf{c}] + \mathbf{b}_1) + b_2 \quad (58)$$

Việc tái tích hợp $\mathbf{c}$ ở bước này là có chủ đích: mục tiêu tương phản có xu hướng kéo gần các hành trình có travel time tương đồng bất kể thời điểm khởi hành, do đó $\mathbf{z}_{\text{seq}}$ có thể không bảo toàn đầy đủ tín hiệu phân biệt theo thời gian. Việc nối ghép trực tiếp $\mathbf{c}$ vào đầu vào hồi quy đảm bảo thông tin thời gian khởi hành luôn hiện diện ở bước dự đoán cuối cùng.

4.5 Giải thuật tìm đường Multilevel Dijkstra

Để thực hiện truy vấn lộ trình trên cấu trúc phân hoạch đa tầng do kiến trúc CRP tạo ra (đã trình bày chi tiết tại Mục 3.9), nghiên cứu sử dụng giải thuật Multilevel Dijkstra (MLD) [11]. Đây là một biến thể tối ưu của thuật toán Dijkstra truyền thống, hoạt động như giai đoạn truy vấn cuối cùng trong toàn bộ hệ thống định tuyến. Bằng cách nhận thức cấu trúc phân hoạch không gian, MLD sử dụng các cạnh tắt từ đồ thị lớp phủ để bỏ qua các vùng không gian không cần thiết, giúp tăng tốc độ tính toán lên nhiều lần. Mã giả của giải thuật được trình bày chi tiết trong Thuật toán 2.

Algorithm 2 Multilevel Dijkstra (MLD)

Require: Đồ thị mạng lưới $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \text{len})$, đỉnh nguồn s , đỉnh đích t , dữ liệu tùy chỉnh.

Ensure: Khoảng cách ngắn nhất $\text{dist}(s, t)$.

- 1: Khởi tạo khoảng cách: $d[v] \leftarrow \infty$ với mọi $v \in \mathcal{V}$; $d[s] \leftarrow 0$
 - 2: Khởi tạo hàng đợi ưu tiên: $Q.\text{insert}(s, 0)$
 - 3: **while** Q không rỗng **do**
 - 4: $u \leftarrow Q.\text{deleteMin}()$
 - 5: **if** $u = t$ **then**
 - 6: **break** {Dừng tìm kiếm khi đã duyệt đến đích}
 - 7: **end if**
 - 8: $l \leftarrow \text{level}(u) = \min(\text{uncommonLevel}(s, u), \text{uncommonLevel}(t, u))$
 - 9: **for** mỗi cạnh $(u, v) \in \bar{\mathcal{E}}_{\text{cell}}^l$ (các cạnh của u ở cấp độ l) **do**
 - 10: **if** $d[u] + \text{len}(u, v) < d[v]$ **then**
 - 11: $d[v] \leftarrow d[u] + \text{len}(u, v)$
 - 12: $Q.\text{update}(v, d[v])$ {Nối lỏng cạnh}
 - 13: **end if**
 - 14: **end for**
 - 15: **end while**
-

Mẫu chốt tối ưu của giải thuật MLD nằm ở dòng số 8, nơi xác định tập cạnh nào được phép nối lỏng thông qua việc tính toán cấp độ hoạt động của đỉnh ($level(u)$). Cơ chế này được điều phối chặt chẽ qua hai công thức toán học sau:

1. Hàm đánh giá mức độ khác biệt không gian ($uncommonLevel$):

Hàm này xác định cấp độ phân hoạch cao nhất mà tại đó hai đỉnh u và v không nằm trong cùng một ô. Cụ thể:

$$uncommonLevel(u, v) = \begin{cases} 0 & \text{nếu } cell_1(u) = cell_1(v) \\ \max\{l \in \{1..L\} \mid cell_l(u) \neq cell_l(v)\} & \text{trường hợp khác} \end{cases} \quad (59)$$

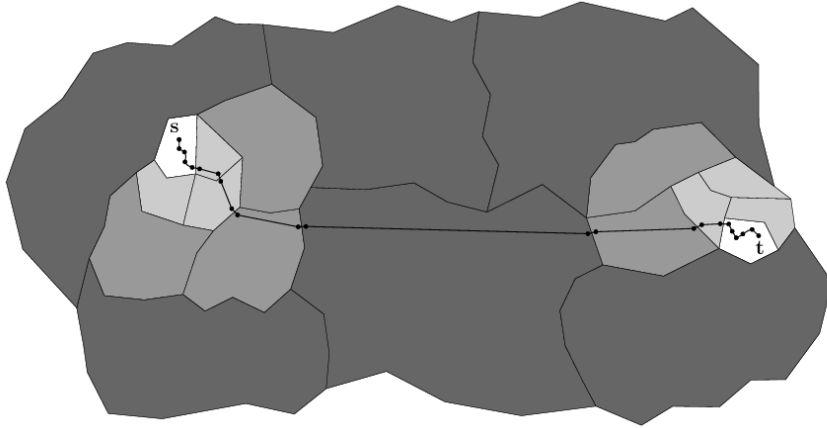
Nếu hai đỉnh nằm trong cùng ô nhỏ nhất (cấp độ 1), hàm sẽ trả về 0. Ngược lại, hàm sẽ tìm ra cấp độ lớn nhất chia tách hai đỉnh này.

2. Hàm xác định cấp độ hoạt động của đỉnh ($level$):

Dựa trên hàm $uncommonLevel$, cấp độ hoạt động của đỉnh u hiện tại được đánh giá bằng mức độ khác biệt không gian từ u đến điểm xuất phát s và điểm đích t :

$$level(u) = \min(uncommonLevel(s, u), uncommonLevel(t, u)) \quad (60)$$

Quy tắc cắt tỉa (Pruning rule): Quy tắc này đảm bảo rằng khi đỉnh u nằm ở lân cận của s hoặc t ($l = 0$), thuật toán sẽ sử dụng đồ thị gốc để tìm đường chi tiết. Ngược lại, khi u ở xa cả s và t , l sẽ mang giá trị lớn, thuật toán chỉ nối lỏng các cạnh tất trên đồ thị lớp phủ ở cấp độ cao tương ứng. Điều này cho phép thuật toán bỏ qua toàn bộ các đỉnh nội bộ của các ô trung gian mà vẫn đảm bảo tính chính xác của quãng đường.



Hình 13: Minh họa giải thuật MLD: Quá trình duyệt trên đồ thị tìm kiếm $\mathcal{G}^*$ [11]

Như được minh họa trực quan trong Hình 13, nhờ áp dụng quy tắc nối lỏng theo $level(u)$, quá trình duyệt không diễn ra trên toàn bộ đồ thị mạng lưới $\mathcal{G}$, mà

trên một đồ thị tìm kiếm rút gọn $\mathcal{G}^* = (\mathcal{V}^*, \mathcal{E}^*)$. Tập $\mathcal{V}^*$ và $\mathcal{E}^*$ chỉ bao gồm các đỉnh, cạnh gốc ở gần s, t và các cạnh tắt thừa thớt ở những phân vùng ở xa. Kích thước của đồ thị tìm kiếm bị giới hạn nghiêm ngặt bởi tham số phân hoạch và cấu trúc lớp phủ, giúp thời gian truy vấn rút ngắn xuống mức phần nghìn giây.

Độ phức tạp thuật toán: Thuật toán Dijkstra nguyên thủy với hàng đợi ưu tiên nhị phân có độ phức tạp là $\mathcal{O}((|\mathcal{V}| + |\mathcal{E}|) \log |\mathcal{V}|)$. Đối với MLD, nhờ áp dụng quy tắc nối lỏng theo $level(u)$, quá trình duyệt không diễn ra trên toàn bộ $\mathcal{G}$, mà trên một Đồ thị tìm kiếm rút gọn $\mathcal{G}^* = (\mathcal{V}^*, \mathcal{E}^*)$, như được minh họa trực quan trong Hình 13. Tập $\mathcal{V}^*$ và $\mathcal{E}^*$ chỉ bao gồm các đỉnh, cạnh gốc ở gần s, t và các cạnh tắt thừa thớt ở những phân vùng ở xa. Nhờ đó, độ phức tạp thực tế giảm mạnh xuống còn $\mathcal{O}((|\mathcal{V}^*| + |\mathcal{E}^*|) \log |\mathcal{V}^*|)$, với $|\mathcal{V}^*| \ll |\mathcal{V}|$ và $|\mathcal{E}^*| \ll |\mathcal{E}|$. Kích thước của $\mathcal{G}^*$ bị giới hạn nghiêm ngặt bởi tham số phân hoạch U^l và cấu trúc lớp phủ, giúp thời gian truy vấn rút ngắn xuống mức phần nghìn giây.

Ưu điểm của giải thuật MLD

- **Tách biệt cấu trúc và trọng số:** MLD phân tách hoàn toàn bước phân hoạch địa lý và việc tính toán trọng số. Khi tình trạng giao thông thay đổi (kẹt xe, cấm đường), hệ thống chỉ mất vài giây để cập nhật lại các cạnh tắt mà không cần tính toán lại toàn bộ cấu trúc mạng lưới.
- **Tối ưu hóa không gian tìm kiếm:** Nhờ kết hợp các ô đa tầng và đồ thị lớp phủ, MLD giảm thiểu tối đa số lượng đỉnh và cạnh cần duyệt so với thuật toán Dijkstra thông thường, mang lại hiệu năng truy vấn siêu tốc trên các bản đồ quy mô lớn.
- **Tối ưu dung lượng lưu trữ:** Việc trừu tượng hóa các ô và chỉ giữ lại cấu trúc lớp phủ giúp nén đáng kể kích thước dữ liệu bản đồ. Điều này giúp thuật toán tiêu thụ ít bộ nhớ hơn, cực kỳ lý tưởng để triển khai các ứng dụng dẫn đường ngoại tuyến.

Tính ưu việt của giải thuật Multilevel Dijkstra (MLD) tích hợp kiến trúc CRP còn được thể hiện rõ ràng khi so sánh độ phức tạp lý thuyết và đặc tính vận hành với các thuật toán định tuyến phổ biến khác. Bảng 1 trình bày sự khác biệt cốt lõi giữa Dijkstra cơ bản, A^* , Contraction Hierarchies (CH) và giải thuật MLD đề xuất.

Bảng 1: So sánh đặc tính và độ phức tạp của các giải thuật định tuyến

Thuật toán	Giai đoạn Tiền xử lý (Preprocessing)	Cập nhật trọng số giao thông	Độ phức tạp truy vấn (Query)
Dijkstra cơ bản	Không yêu cầu.	Không yêu cầu.	$\mathcal{O}((\mathcal{V} + \mathcal{E}) \log \mathcal{V})$.
A*	Không yêu cầu.	Không yêu cầu.	Xấu nhất: $\mathcal{O}((\mathcal{V} + \mathcal{E}) \log \mathcal{V})$.
Contraction Hierarchies (CH)	Phụ thuộc vào trọng số.	Phải tính toán lại toàn bộ cấu trúc phân cấp từ đầu.	$\mathcal{O}(V_{CH} \log V_{CH})$ với tập đỉnh duyệt $V_{CH} \ll \mathcal{V} $.
Multilevel Dijkstra (MLD)	Không phụ thuộc vào trọng số	Chỉ cập nhật các cạnh tắt bị ảnh hưởng.	$\mathcal{O}((\mathcal{V}^* + \mathcal{E}^*) \log \mathcal{V}^*)$ với tập đỉnh rút gọn $ \mathcal{V}^* \ll \mathcal{V} $.

Qua Bảng 1, có thể thấy Dijkstra cơ bản tuy không mất thời gian tiền xử lý nhưng lại tạo ra nút thắt cổ chai ở giai đoạn truy vấn. Ngược lại, các thuật toán như Contraction Hierarchies (CH) giải quyết được tốc độ truy vấn nhưng lại bộc lộ điểm yếu chí mạng khi áp dụng cho giao thông thực tế: mỗi khi trọng số thay đổi (do dự báo ETA từ mô hình PoI-CL-TTE), CH phải mất hàng giờ để tiền xử lý lại từ đầu. MLD khắc phục được toàn bộ các nhược điểm này bằng cách đóng băng cấu trúc topo và chỉ cập nhật trọng số cạnh tắt (shortcuts) trong các ô (cells) bị ảnh hưởng, mang lại sự cân bằng hoàn hảo giữa tốc độ truy vấn và khả năng cập nhật linh hoạt.

5

Chuẩn bị dữ liệu

5.1 Thu thập dữ liệu di chuyển tại khu vực TP. Hồ Chí Minh

5.1.1 Cơ sở lựa chọn phương pháp thu thập dữ liệu

Đối với các bài toán ước lượng thời gian di chuyển ứng dụng học sâu, chất lượng và quy mô của bộ dữ liệu huấn luyện đóng vai trò nền tảng quyết định hiệu năng của phương pháp. Để mô hình có khả năng hội tụ tốt và dự đoán chính xác, tập dữ liệu yêu cầu độ phủ cực kỳ lớn và đa dạng. Cụ thể, dữ liệu cần phải bao quát không chỉ về mặt không gian với đầy đủ các loại hình mạng lưới đường bộ, trạng thái nút giao hay các dải khoảng cách chuyển đi khác nhau, mà còn phải phong phú về mặt thời gian, bao gồm biến động độ dài chuyến đi, thời điểm xuất phát trong ngày và các ngày trong tuần.

Tuy nhiên, việc thu thập một bộ dữ liệu thực tế đáp ứng đầy đủ các tiêu chí khắt khe này là một thách thức lớn đối với các nghiên cứu độc lập. Ban đầu, nghiên cứu cân nhắc hai phương pháp tiếp cận dữ liệu thực tế: tự tổ chức thu thập thông qua các thiết bị định vị gắn trên phương tiện và trao đổi dữ liệu quỹ đạo từ các công ty cung cấp dịch vụ gọi xe công nghệ. Dù vậy, cả hai phương án này đều nhanh chóng cho thấy tính bất khả thi. Việc tự đo đạc trực tiếp trên đường phố đòi hỏi nguồn lực khổng lồ về thời gian và chi phí, hoàn toàn không thể đáp ứng được khối lượng hàng triệu mẫu dữ liệu mà học sâu đòi hỏi. Trong khi đó, các công ty công nghệ sở hữu nguồn dữ liệu khổng lồ nhưng lại áp dụng chính sách bảo mật dữ liệu người dùng vô cùng nghiêm ngặt, khiến việc tiếp cận các tập dữ liệu định tuyến thực tế này để phục vụ nghiên cứu mở là điều gần như không thể.

Đứng trước rào cản về dữ liệu thực tế, các phương pháp tạo lập dữ liệu nhân tạo cũng được đưa ra xem xét. Một giải pháp phổ biến là sử dụng các phần mềm mô phỏng giao thông vi mô như SUMO (Simulation of Urban MObility). Dù chi phí thấp, nhưng dữ liệu sinh ra từ các môi trường giả lập thường mang tính lý tưởng hóa, rất khó để phản ánh chính xác những biến động phức tạp, tính ngẫu nhiên và đặc tính ùn tắc cục bộ đặc thù của giao thông đô thị đời thực.

Do đó, sau khi cân nhắc toàn diện, nghiên cứu quyết định lựa chọn phương pháp thu thập dữ liệu thông qua các nền tảng API định tuyến thương mại, cụ thể là Tomtom API. Mặc dù dữ liệu trả về từ API về bản chất là kết quả ước lượng từ mô hình nội bộ của nhà cung cấp thay vì dữ liệu quỹ đạo xe chạy thực

tế thô, nhưng cách tiếp cận này lại mang đến độ tin cậy rất cao. Các hệ thống như Tomtom hay Google Maps được xây dựng trên nền tảng tổng hợp dữ liệu giao thông theo thời gian thực từ nguồn dữ liệu đám đông khổng lồ (crowdsourced floating car data) kết hợp với các mô hình dự báo tiên tiến. Do vậy, dữ liệu truy xuất từ API phản ánh rất sát trạng thái giao thông thực tế, giải quyết trọn vẹn bài toán thiếu hụt khối lượng dữ liệu lớn trong khi vẫn đảm bảo được tính đa dạng, phức tạp và tính đại diện cho môi trường giao thông đô thị thực tế.

5.1.2 Quy trình thu thập dữ liệu

Trong nghiên cứu này, chúng tôi xây dựng một công cụ thu thập dữ liệu bằng Python nhằm truy vấn thông tin di chuyển từ Tomtom API như minh họa trong Hình 14. Tomtom API cung cấp dữ liệu giao thông theo thời gian thực bao gồm thời gian di chuyển thực tế, tốc độ di chuyển trung bình và các tuyến đường được đề xuất (chi tiết về các trường dữ liệu thu thập được trình bày cụ thể tại Mục 5.1.3). Nhờ dữ liệu được cập nhật liên tục từ cộng đồng người dùng, Tomtom là nguồn dữ liệu phù hợp cho các bài toán ước lượng thời gian di chuyển.

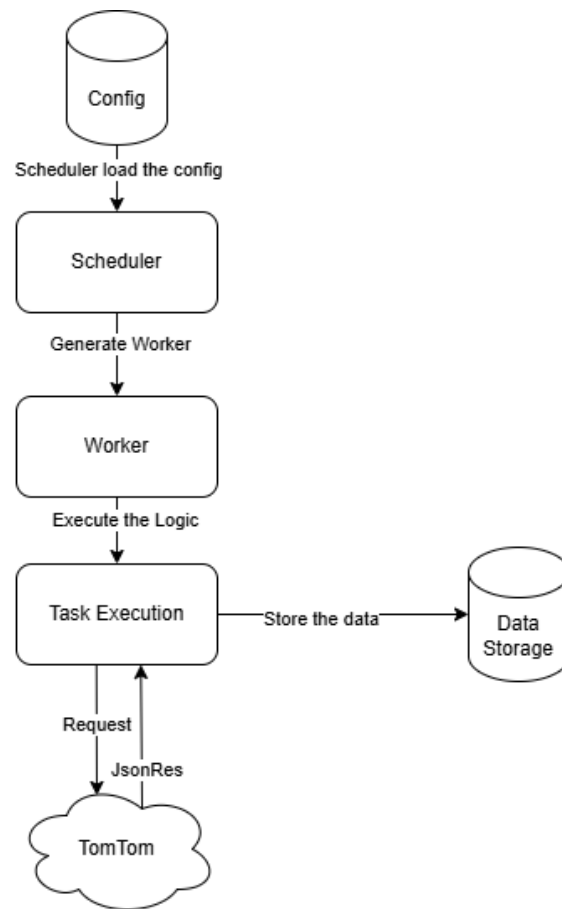
Để đảm bảo mức độ chi tiết và chất lượng của dữ liệu, bộ lập lịch thu thập được cấu hình chạy hằng ngày từ 5:00 đến 22:00, với mỗi yêu cầu gửi cách nhau 3 giây. Khu vực thu thập được cố định trong phạm vi vùng bao giới hạn của TP. Hồ Chí Minh, với tọa độ:

[10.7301793, 106.6164137, 10.8524436, 106.7132954]

như Hình 15 minh họa.

Việc lựa chọn vùng không gian này không phải ngẫu nhiên mà dựa trên sự cân nhắc kỹ lưỡng giữa tính đại diện của dữ liệu và giới hạn tài nguyên hệ thống:

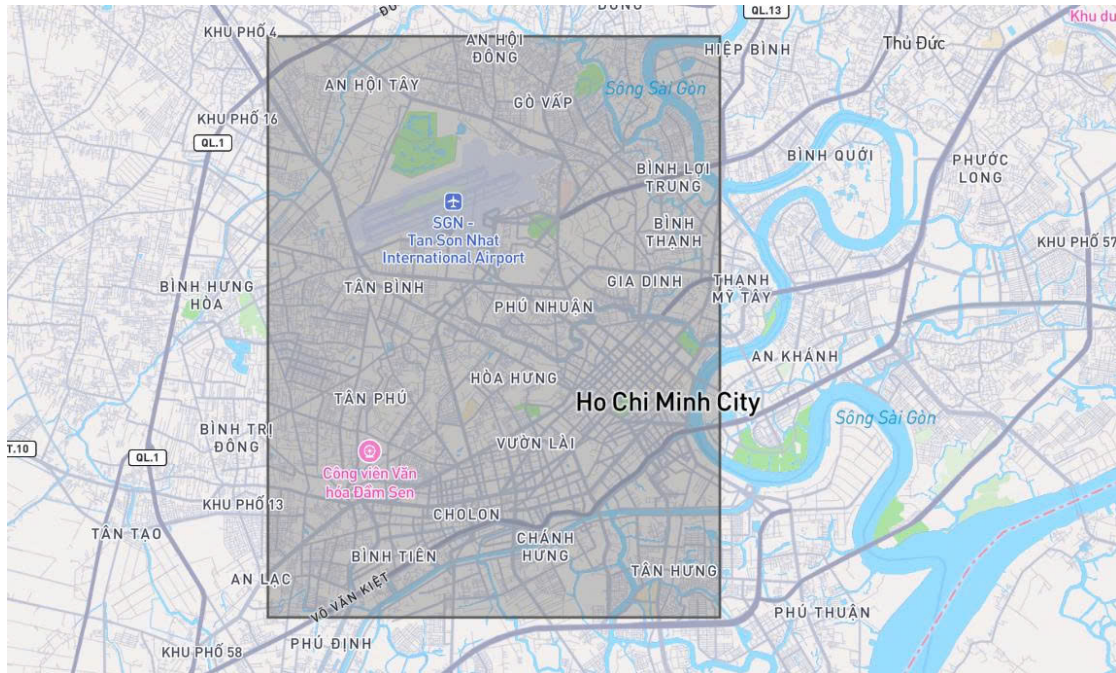
- **Tính tập trung vào vùng lõi đô thị:** Vùng bao này được thiết kế để bao phủ trọn vẹn khu vực trung tâm TP.HCM, nơi có mật độ giao thông cao nhất và diễn biến phức tạp nhất. Chiến lược này giúp loại bỏ nhiều từ các khu vực ngoại ô thưa thớt (như Củ Chi, Bình Dương) nơi tần suất các chuyến đi thấp và ít xảy ra ùn tắc, đảm bảo mô hình tập trung học các mẫu dữ liệu giàu thông tin.
- **Tối ưu hóa tài nguyên tính toán:** Kích thước vùng bao được xác định là ngưỡng tối đa mà môi trường thực nghiệm hiện tại có thể xử lý ổn định. Việc tải và xử lý đồ thị mạng lưới đường bộ cho một khu vực lớn hơn sẽ dẫn đến quá tải bộ nhớ khi thực nghiệm mô hình, trong khi việc thu hẹp phạm vi sẽ làm giảm tính bao quát của dữ liệu. Do đó, kích thước này là điểm cân bằng tối ưu giữa độ phủ dữ liệu và khả năng vận hành của hệ thống.



Hình 14: Thiết kế công cụ thu thập dữ liệu

Về đặc điểm địa lý, vùng khảo sát này có diện tích **143.29 km^2** , bao phủ qua **17 quận/huyện** trọng điểm (bao gồm Quận 1, Quận 3, Bình Thạnh, Phú Nhuận, Tân Bình,...). Mạng lưới giao thông bên trong vùng bao này cực kỳ dày đặc với tổng chiều dài đường ước tính khoảng **1,863 km**, đảm bảo cung cấp đầy đủ ngữ cảnh không gian cho bài toán ước lượng thời gian di chuyển.

Chúng tôi áp dụng chiến lược thu thập dựa trên các điểm quan tâm (Points of Interest – PoI). Các PoI ban đầu bao gồm trường đại học, bệnh viện, chợ, trung tâm thương mại, điểm du lịch và sân bay. Từ danh sách này, chúng tôi chọn ra 100 PoI có mức độ thu hút giao thông lớn để làm điểm đầu và điểm cuối cho quá trình truy vấn. Các PoI được kết nối theo chiến lược ghép cặp toàn vẹn (all-to-all) nhằm bao phủ đa dạng các luồng di chuyển trong thành phố. Cụ thể, trong danh sách 100 điểm được chọn, mỗi PoI sẽ lần lượt đóng vai trò là điểm xuất phát để kết nối tới 99 PoI còn lại làm điểm đích. Chiến lược này giúp tạo ra một tập hợp phong phú các cặp Xuất phát - Đích đến, đại diện cho các nhu cầu di chuyển thực



Hình 15: Phạm vi khu vực TP. Hồ Chí Minh được sử dụng để thu thập dữ liệu

tế. Đối với mỗi cặp điểm, Tomtom API sẽ trả về tuyến đường tối ưu dựa trên tình trạng giao thông tại thời điểm truy vấn, cùng với thời gian di chuyển tương ứng và các đặc trưng liên quan phục vụ cho bài toán dự báo. Việc sử dụng chiến lược dựa trên PoI mang lại hai lợi ích chính: (1) giúp quan sát rõ biến động giao thông tại các khu vực thường xuyên xảy ra ùn tắc, từ đó tăng giá trị của dữ liệu thu thập; (2) hạn chế nhiều đến từ các tuyến đường ít được sử dụng, đảm bảo dữ liệu thu được mang tính đại diện cao cho hoạt động giao thông đô thị.

Dữ liệu được thu thập liên tục trong 3 tuần, từ ngày 01/11 đến 22/11, trong khung giờ từ 5:00 đến 22:00 mỗi ngày, với tần suất một yêu cầu mỗi 3 giây. Công cụ thu thập chạy với ba luồng song song, tương đương với ba yêu cầu mỗi 3 giây (khoảng một yêu cầu mỗi giây).

Khối lượng dữ liệu thu được được tính như sau:

- Thời gian thu thập mỗi ngày:

$$17 \text{ giờ/ngày} = 61,200 \text{ giây/ngày}$$

- Số yêu cầu mỗi ngày:

$$\frac{61,200}{3} \times 3 = 61,200 \text{ yêu cầu/ngày}$$

- Tổng số yêu cầu trong 22 ngày:

$$61,200 \times 22 = 1,346,400 \text{ yêu cầu}$$

Theo tính toán lý thuyết, hệ thống có thể thu được khoảng 1.34 triệu mẫu dữ liệu (trong đó, mỗi mẫu đại diện cho một lộ trình hoàn chỉnh từ điểm xuất phát đến điểm đích, đi kèm với nhãn thời gian di chuyển thực tế). Tuy nhiên, ta cần loại trừ đi các yêu cầu trả về không hợp lệ, chẳng hạn như Tomtom không tìm thấy lộ trình phù hợp tại thời điểm truy vấn. Sau khi loại bỏ các mẫu này, số lượng dữ liệu hợp lệ còn khoảng 1 triệu mẫu, đáp ứng đầy đủ quy mô và mức độ chi tiết cần thiết để phục vụ huấn luyện và đánh giá mô hình ước lượng thời gian di chuyển.

5.1.3 Dữ liệu thô thu thập

trip_id	timestamp	distance	duration	geometry	distance_gap	time_gap	n_points
20160	1759005837	5880	1003	LINestring(106.6388848 10.8 0;17;96;133;192;275;324;340;394;473;517 0;4;21;29;47;59;66;68;7			87
20161	1759005843	7925	1407	LINestring(106.65372250050 0;88;186;241;253;282;300;404;576;582;60 0;15;27;34;36;40;42;56;			101
20162	1759005847	7084	1270	LINestring(106.6698845 10.7 0;58;143;210;244;269;279;333;379;476;54 0;11;23;33;43;49;51;60;			92
20163	1759005853	9723	1627	LINestring(106.62532849209 0;115;159;274;324;355;408;418;448;461;5 0;15;21;35;41;45;51;52;			150
20164	1759005861	6760	1200	LINestring(106.65510500133 0;62;87;89;96;107;420;495;588;602;635;64 0;11;18;18;20;22;65;76;			92
20165	1759005864	6836	1289	LINestring(106.68638742997 0;60;89;117;241;249;298;355;449;473;520 0;14;21;27;77;78;84;91;			91
20166	1759005868	8138	1407	LINestring(106.69781440872 0;59;165;355;470;537;667;734;788;795;81 0;9;28;62;83;94;114;127			103
20167	1759005871	8803	1298	LINestring(106.69341304263 0;84;121;251;504;669;796;864;895;963;10 0;10;18;65;157;178;195;			92
20168	1759005874	6707	1155	LINestring(106.68647627729 0;57;81;176;213;285;296;455;488;574;612 0;16;21;33;39;66;70;111			119
20169	1759005877	7696	1371	LINestring(106.66095802714 0;105;117;131;377;556;574;599;710;729;8 0;20;22;24;50;71;74;78;			98
20170	1759005880	7432	1276	LINestring(106.65557671543 0;31;75;148;221;330;382;513;528;569;605 0;6;13;22;30;42;48;62;6			105
20171	1759005884	5221	717	LINestring(106.70547021808 0;62;109;117;136;150;321;331;435;720;79 0;11;18;19;22;24;52;53;			47
20172	1759005887	20374	1897	LINestring(106.64761447778 0;80;126;165;180;206;269;334;375;416;46 0;16;23;29;31;34;42;52;			121
20173	1759005896	4027	907	LINestring(106.68542257437 0;25;150;162;290;354;376;513;635;646;79 0;11;102;105;127;138;1			68
20174	1759005899	9317	1462	LINestring(106.64577863981 0;22;53;90;99;169;267;280;306;327;383;45 0;4;8;13;14;23;35;37;41			111
20175	1759005903	16519	2321	LINestring(106.7080164 10.7 0;39;94;106;509;523;907;1216;1468;1485; 0;7;27;32;74;76;108;137			140
20176	1759005907	6660	957	LINestring(106.70472884368 0;62;72;176;447;517;738;746;887;947;101 0;10;11;22;53;62;104;10			45
20177	1759005910	3677	617	LINestring(106.65958072625 0;57;134;286;304;326;394;436;458;589;65 0;10;24;53;57;60;68;74;			64
20178	1759005914	2923	529	LINestring(106.66658625370 0;61;177;199;316;380;419;433;512;557;60 0;18;39;43;64;82;92;95;			59
20179	1759005916	13406	1823	LINestring(106.71253769585 0;99;230;634;910;920;1262;1421;1435;146 0;13;25;79;108;109;149;			94
20180	1759005920	8873	1354	LINestring(106.66045682339 0;69;216;330;356;405;467;526;583;618;69 0;7;24;37;41;53;98;108;			112
20181	1759005923	3219	528	LINestring(106.6811112 10.7 0;187;395;416;505;680;810;851;972;1077; 0;23;46;49;71;90;103;10			31
20182	1759005926	3541	588	LINestring(106.63982823061 0;54;85;101;203;310;398;468;476;487;548 0;6;11;13;26;42;51;59;6			59
20183	1759005930	14443	2010	LINestring(106.71110764924 0;80;133;181;315;417;519;591;602;802;81 0;18;29;38;56;68;80;100			154
20184	1759005934	7634	1208	LINestring(106.66168599207 0;71;140;211;220;313;422;519;630;733;83 0;15;26;50;52;67;103;11			94
20185	1759005937	12505	1852	LINestring(106.6909976 10.8 0;17;56;94;143;176;202;247;275;290;357;4 0;2;7;12;19;24;28;34;38			167
20186	1759005943	4867	821	LINestring(106.63208283197 0;81;88;100;318;348;363;378;409;449;517 0;10;11;13;34;40;43;46;			65

Hình 16: Mẫu dữ liệu thô thu thập được lưu trữ dưới dạng bảng (CSV)

Như được minh họa trong Hình 16, sau quá trình thu thập, dữ liệu thô được tổ chức và lưu trữ dưới dạng các file CSV. Mỗi dòng dữ liệu đại diện cho một mẫu lộ trình cụ thể, được mô tả chi tiết thông qua các trường thông tin (cột) sau:

- **trip_id**: Là định danh duy nhất (Unique ID) do công cụ thu thập tự gán cho mỗi chuyến đi. Trường này đóng vai trò khóa chính để quản lý, tham chiếu và phân biệt các bản ghi dữ liệu khác nhau trong toàn bộ tập dữ liệu.

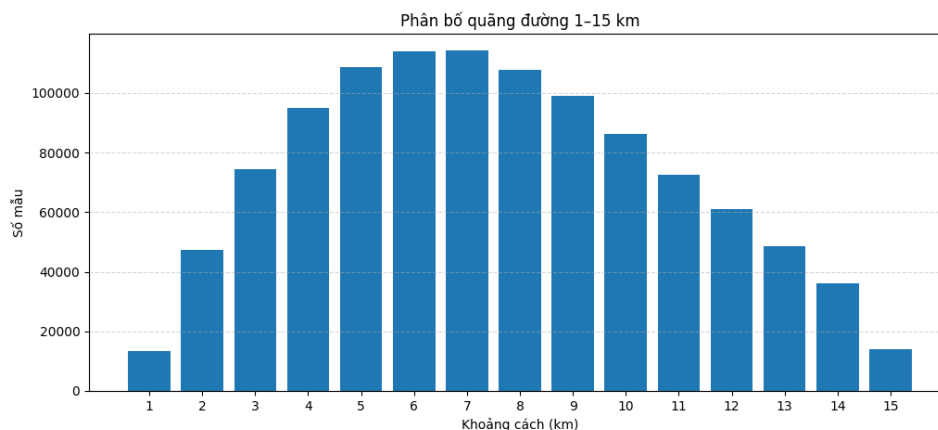
- **timestamp:** Thời điểm phương tiện bắt đầu xuất phát, được lưu trữ dưới định dạng UNIX timestamp. Đây là mốc thời gian thực giúp mô hình nắm bắt được yếu tố thời điểm (giờ cao điểm, ngày thường/cuối tuần).
- **distance:** Tổng chiều dài của lộ trình được Tomtom đề xuất, tính theo đơn vị mét (m).
- **duration:** Thời gian di chuyển thực tế của phương tiện trên lộ trình đó, tính theo đơn vị giây (s). Đây chính là **nhân thời gian (ground truth)** phản ánh tình trạng giao thông tại thời điểm truy vấn mà mô hình cần học để dự đoán.
- **geometry:** Dữ liệu hình học dạng **LINESTRING**, chứa chuỗi các tọa độ GPS (kinh độ, vĩ độ) tạo nên hình dáng thực tế của tuyến đường. Trường này cung cấp thông tin không gian chi tiết về cấu trúc tuyến đường - nguồn đặc trưng quan trọng nhất cho các mô hình ước lượng.
- **distance_gap:** Danh sách các khoảng cách tích lũy (cumulative distance) giữa các điểm nút dọc theo tuyến đường (đơn vị: mét). Giá trị bắt đầu từ 0 và tăng dần đến tổng chiều dài quãng đường, cho phép phân tích chi tiết cấu trúc độ dài của từng phân đoạn nhỏ trong lộ trình.
- **time_gap:** Danh sách thời gian tích lũy tương ứng với **distance_gap** (đơn vị: giây). Trường này cho biết thời gian ước lượng để phương tiện đi đến từng điểm nút cụ thể dọc theo tuyến.
- **n_points:** Số lượng điểm GPS có trong trường **geometry**, phản ánh độ chi tiết và độ phức tạp hình học của lộ trình thu thập được.

5.1.4 Tiền xử lý dữ liệu

Trước khi đưa vào giai đoạn khảo sát và huấn luyện mô hình, tập dữ liệu thô được làm sạch nhằm loại bỏ các trường hợp bất thường có thể gây nhiễu cho mô hình. Chúng tôi loại bỏ các mẫu có quãng đường di chuyển quá ngắn (dưới 1 km), thời gian di chuyển quá dài (trên 3600 giây - 1 giờ), hoặc vận tốc trung bình bất thường — cụ thể là dưới 15 km/h hoặc vượt quá 100 km/h. Các điều kiện tiền xử lý này được áp dụng nhằm loại bỏ những điểm ngoại lai có thể gây nhiễu cho mô hình, chẳng hạn như các chuyến đi có quãng đường quá ngắn, thời gian quá dài hoặc vận tốc trung bình bất hợp lý. Sau bước lọc, chỉ những mẫu dữ liệu nằm trong phạm vi cho phép mới được giữ lại để phục vụ cho quá trình huấn luyện mô hình.

5.1.5 Khảo sát chất lượng dữ liệu

Chúng tôi tiến hành khảo sát các đặc trưng thống kê cơ bản của tập dữ liệu nhằm đánh giá mức độ ổn định, độ phủ không gian và sự phân bố của các mẫu. Hai biểu đồ dưới đây mô tả phân bố quãng đường di chuyển và mật độ mẫu theo từng giờ trong ngày, giúp kiểm tra xem dữ liệu thu thập có đủ đa dạng và có xuất hiện các điểm bất thường hay không.



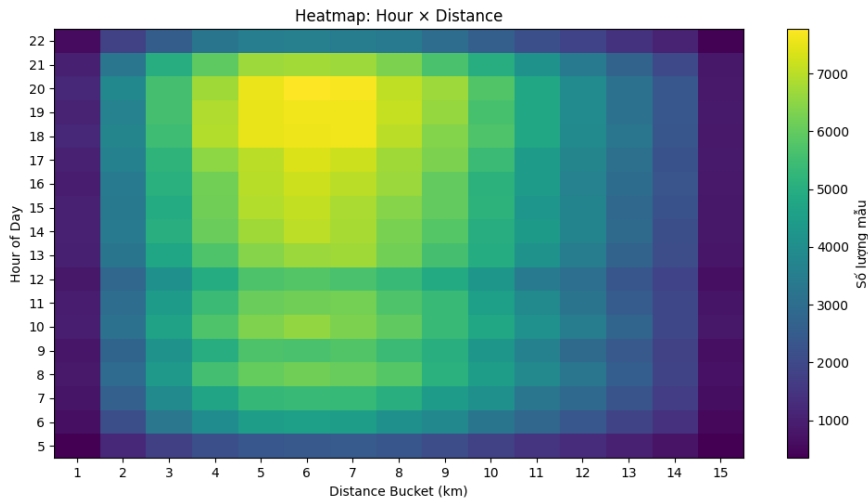
Hình 17: Phân bố quãng đường di chuyển

Dựa vào hình 17, có thể rút ra một số quan sát quan trọng:

- **Dải khoảng cách tập trung rõ rệt:** Phần lớn các mẫu nằm trong khoảng 4–9 km, với đỉnh phân bố ở khoảng 6–8 km. Đây là phạm vi di chuyển phổ biến trong nội đô TP. Hồ Chí Minh.
- **Ít mẫu ở hai cực:** Các chuyến đi ngắn dưới 2 km và dài trên 15 km chiếm tỉ lệ nhỏ. Điều này giúp mô hình tập trung học trên các mẫu tiêu biểu, tránh bị ảnh hưởng bởi các chuyến đi quá đặc thù hoặc ngoại lệ.
- **Phân bố đối xứng tương đối:** Lượng mẫu giảm dần về hai phía cho thấy dữ liệu không bị lệch mạnh về cực ngắn hoặc cực dài.

Biểu đồ nhiệt ở hình 18 tiếp tục khẳng định tính ổn định của tập dữ liệu:

- **Mật độ mẫu ổn định trong khung giờ 5–22h:** Đây là khoảng thời gian mà hệ thống thu thập dữ liệu đều đặn, thể hiện rõ qua vùng màu sáng. Điều này phù hợp với thực tế giao thông đô thị—hoạt động chủ yếu vào ban ngày.
- **Khoảng cách phổ biến trùng khớp với đồ thị phân bố:** Các ô sáng tập trung ở khoảng cách 4–9 km, cho thấy hai phân tích độc lập đưa ra kết quả nhất quán.



Hình 18: Mật độ dữ liệu theo Giờ và Quãng đường

- **Không xuất hiện nhiều điểm bất thường:** Không có các cụm dữ liệu rời rạc bất ngờ, không có dải giờ hoặc khoảng cách bị trống. Điều này cho thấy quá trình thu thập ổn định, ít nhiễu.

Tổng kết

Từ những quan sát trên, có thể kết luận rằng tập dữ liệu có chất lượng tốt, phân bố hợp lý và đủ ổn định để sử dụng trong các bước tiền xử lý tiếp theo cũng như huấn luyện mô hình dự đoán thời gian di chuyển.

5.1.6 Xây dựng tập dataset

Dữ liệu thô thu thập từ Tomtom, mặc dù đã qua bước lọc nhiễu, vẫn tồn tại dưới dạng các tọa độ GPS rời rạc. Để chuyển đổi các tọa độ này thành thông tin có ý nghĩa trên mạng lưới giao thông, chúng tôi thực hiện kỹ thuật gán ghép bản đồ sử dụng thuật toán Fast Map Matching (FMM) được đề xuất bởi Yang và Gidófalvi [8]. Thuật toán này tích hợp mô hình Markov ẩn (HMM) với kỹ thuật tính toán trước, cho phép xác định chính xác lộ trình thực tế của phương tiện trên bản đồ số (OpenStreetMap).

Sau khi quá trình map matching hoàn tất, mỗi chuyến đi được ánh xạ thành công sẽ được trích xuất và lưu trữ lại với các trường như sau:

- **Chuỗi Segment ID:** Danh sách các định danh của từng đoạn đường mà xe đã đi qua. Đây là thông tin không gian cốt lõi giúp mô hình nhận biết

cấu trúc hình học của tuyến đường. Các ID này có thể sử dụng để truy vấn thêm các đặc trưng khác từ dữ liệu của OSM.

- **Thông tin thời gian xuất phát:** Bao gồm *Day of week* (thứ trong tuần), *Day of year* (ngày trong năm) và *Minute of day* (phút trong ngày). Các đặc trưng này đóng vai trò giúp mô hình học được tính chu kỳ và các quy luật biến động của giao thông theo thời gian.
- **Thời gian di chuyển:** Tổng thời gian thực tế để phương tiện đi hết lộ trình đã gắn ghép, đơn vị là giây. Đây chính là nhãn mục tiêu dùng để huấn luyện và đánh giá độ chính xác của mô hình.

5.2 Trích xuất đặc trưng

Mỗi phân đoạn đường trong mạng lưới giao thông được đặc trưng hóa thông qua ba nhóm thông tin:

1. Đặc trưng hình học mô tả cấu trúc không gian.
2. Đặc trưng thuộc tính phản ánh năng lực vật lý của hạ tầng.
3. Đặc trưng ngữ cảnh không gian và thời gian từ môi trường xung quanh.

Quá trình trích xuất ba nhóm đặc trưng này được trình bày lần lượt trong các mục dưới đây.

5.2.1 Đặc trưng hình học

Hai đặc trưng hình học cơ bản được trích xuất trực tiếp từ đồ thị đường bộ:

- **Độ dài phân đoạn** là chiều dài thực tế của đoạn đường tính bằng mét, phản ánh quy mô không gian của phân đoạn và đóng vai trò nền tảng để mô hình ngầm học thời gian di chuyển lý tưởng khi kết hợp với tốc độ tối đa.
- **Độ dài tích lũy** là tổng chiều dài từ điểm xuất phát của hành trình đến đầu phân đoạn hiện tại, phản ánh vị trí tương đối của đoạn đường trong toàn bộ lộ trình.

Lưu ý về việc không sử dụng tọa độ GPS. Trong nghiên cứu này, tọa độ GPS (kinh độ/vĩ độ) được chủ động loại khỏi tập đặc trưng đầu vào vì ba lý do:

- **Thiếu tính tổng quát:** Tọa độ GPS mang tính định danh tuyệt đối, tại đó mỗi phân đoạn đường có một bộ kinh độ/vĩ độ duy nhất, khiến chúng hoạt động như một khóa định danh thay vì một đặc trưng mang ý nghĩa cấu trúc. Điều này xung đột trực tiếp với mục tiêu của học tương phản (Mục 4.3): khi hai hành trình đi qua cùng một đoạn đường, mô hình sẽ dễ dàng nhận ra sự tương đồng chỉ dựa trên tọa độ trùng khớp mà không cần học bất kỳ đặc trưng ngữ nghĩa nào, làm mất đi tác dụng của cơ chế học tương phản.
- **Nguy cơ học lệch (shortcut learning):** Khi tọa độ được cung cấp trực tiếp, mô hình có xu hướng học ánh xạ từ vị trí địa lý đến thời gian di chuyển thay vì học các đặc trưng mang tính cấu trúc và có khả năng tổng quát hóa như loại đường, số làn hay ngữ cảnh PoI. Mô hình học lệch theo tọa độ sẽ suy giảm hiệu năng đáng kể khi áp dụng sang khu vực địa lý mới hoặc khi mạng lưới đường bộ thay đổi. **Nhiều đo lường trong môi trường đô thị:** Dữ liệu GPS thực tế thường chứa sai số định vị. Việc đưa tọa độ nhiễu vào mô hình mà không có bước xử lý chuyên biệt có thể làm giảm chất lượng biểu diễn và gây bất ổn định trong quá trình huấn luyện.

Thay vào đó, mô hình tập trung vào các đặc trưng mang tính tương đối và cấu trúc (như độ dài, loại đường, PoI, và thông tin thời gian bắt đầu), nhằm học được các biểu diễn có khả năng tổng quát hóa tốt hơn và ít phụ thuộc vào vị trí tuyệt đối.

5.2.2 Đặc trưng thuộc tính đường

Để mô tả năng lực lưu thông vật lý của hạ tầng giao thông, nghiên cứu trích xuất hai thuộc tính cốt lõi từ dữ liệu OSM: loại đường, tốc độ tối đa.

Loại đường. OSM phân loại đường bộ theo hệ thống phân cấp chức năng, từ đường cao tốc đến đường dân sinh.

Tốc độ tối đa. Tốc độ tối đa là chỉ số đại diện cho cấp bậc và tiêu chuẩn kỹ thuật của tuyến đường. Khi kết hợp với chiều dài phân đoạn, đặc trưng này cho phép mô hình ngầm học được thời gian di chuyển lý tưởng trong điều kiện không tắc nghẽn, làm nền tảng trước khi tổng hợp các yếu tố ngữ cảnh khác. Tốc độ được ánh xạ vào các ngưỡng cố định $B_{\text{speed}} = \{0, 10, 20, \dots, 90, 120\}$ (km/h). Đối với các phân đoạn có nhiều giá trị tốc độ đồng thời, giá trị nhỏ nhất được ưu tiên để phản ánh đúng năng lực thông hành tại điểm thắt nút.

Xử lý nhiễu dữ liệu OSM. Do OSM là nền tảng đóng góp cộng đồng, dữ liệu thuộc tính thường tồn tại sai số như giá trị xung đột hoặc thiếu sót. Quy trình

rời rạc hóa trên đóng vai trò chuẩn hóa và tăng cường tính bền vững của mô hình trước các dạng nhiễu đặc trưng này.

5.2.3 Đặc trưng ngữ cảnh không gian từ PoI

Điều kiện lưu thông của một đoạn đường chịu ảnh hưởng của môi trường chức năng xung quanh. Các địa điểm như trường học, trung tâm thương mại hay bệnh viện tạo ra các luồng phương tiện đặc trưng theo không gian và thời gian. Do đó, nghiên cứu tích hợp dữ liệu PoI từ OpenStreetMap vào từng phân đoạn đường.

Cơ chế gán PoI. Do tọa độ của PoI thường nằm cạnh đường thay vì trùng khít trên trục đường, nghiên cứu áp dụng kỹ thuật Spatial Join với vùng đệm bán kính $R = 50$ mét. Một PoI được coi là thuộc về phân đoạn đường s nếu vị trí của nó nằm trong vùng đệm của s . Ngưỡng 50 mét được lựa chọn để bao phủ các cơ sở hạ tầng ven đường nhưng không gây nhiễu từ các tuyến đường song song lân cận.

Phân nhóm danh mục PoI. Dữ liệu PoI thô được phân loại vào G nhóm chức năng nhằm phản ánh đặc tính sinh/hấp dẫn chuyển đi của khu vực lân cận:

- **Dịch vụ ăn uống và bán lẻ:** nhà hàng, cafe, siêu thị, trung tâm, thương mại. Dịch vụ ảnh hưởng đến biến động giao thông ngoài giờ hành chính.
- **Giáo dục và văn phòng:** trường học, đại học, tòa nhà văn phòng. Các điểm này tạo áp lực giao thông tập trung vào giờ cao điểm.
- **Vận chuyển và du lịch:** trạm xe buýt, metro, khách sạn, điểm tham quan; phản ánh đặc trưng kết nối đa phương thức và thu hút khách du lịch.
- **Y tế và chức năng đặc thù:** bệnh viện, cơ sở y tế và các khu vực chức năng có lưu lượng đặc trưng.

Mã hóa nhị phân. Khác với phương pháp đếm số lượng truyền thống vốn dễ bị nhiễu do mật độ PoI không đồng nhất giữa nội đô và ngoại ô, nghiên cứu chỉ tập trung vào *sự hiện diện* của từng nhóm danh mục. Với mỗi phân đoạn t , đặc trưng PoI được biểu diễn bằng véc-tơ nhị phân $\mathbf{p}_t \in \{0, 1\}^G$:

$$p_{t,g} = \begin{cases} 1, & \exists g \text{ trong phạm vi đoạn đường } t \\ 0, & \text{nếu ngược lại} \end{cases} \quad (61)$$

Cách mã hóa này giúp mô hình tập trung vào chức năng đô thị của khu vực thay vì bị chi phối bởi mức độ phủ dữ liệu OSM vốn không đồng đều theo địa bàn.

5.3 Đặc trưng đầu vào

Mỗi hành trình được biểu diễn dưới dạng chuỗi các phân đoạn đường $\{x_t\}_{t=1}^T$, trong đó mỗi phân đoạn x_t được đặc trưng bởi ba nhóm đặc trưng:

- **Đặc trưng hình học:** mô tả cấu trúc không gian của phân đoạn đường:
 - *Độ dài phân đoạn:* chiều dài thực tế của đoạn đường.
 - *Độ dài tích lũy:* tổng chiều dài từ điểm xuất phát đến đầu phân đoạn hiện tại, phản ánh vị trí tương đối của đoạn đường trong hành trình.
- **Đặc trưng thuộc tính** mô tả các tính chất vật lý và pháp lý của phân đoạn đường được trích xuất từ OpenStreetMap:
 - *Tốc độ tối đa:* giới hạn tốc độ cho phép trên đoạn đường.
 - *Loại đường:* phân loại đường theo hệ thống phân cấp của OSM (ví dụ: đường cao tốc, đường phố chính, đường dân sinh).
- **Đặc trưng ngữ nghĩa** mã hóa ngữ cảnh không gian và thời gian của hành trình:
 - *Sự hiện diện PoI lân cận:* véc-tơ nhị phân $\mathbf{p}_s \in \{0, 1\}^G$ biểu diễn sự hiện diện của từng nhóm danh mục PoI trong vùng lân cận của phân đoạn, trong đó G là tổng số nhóm danh mục (xem Mục 3.3), xác định ngữ cảnh không gian của chuyến đi.
 - *Thời gian bắt đầu hành trình:* bộ ba giá trị (phút trong ngày, ngày trong tuần, ngày trong năm) xác định ngữ cảnh thời gian của chuyến đi (xem Mục 4.2.1).

6

Luồng huấn luyện

Quá trình huấn luyện mô hình PoI-CL-TTE được thiết kế dưới dạng học đa nhiệm (multi-task learning), kết hợp giữa việc tối ưu hóa sai số dự đoán thời gian di chuyển và việc tinh chỉnh không gian biểu diễn thông qua học tương phản (Contrastive Learning) và học tái tạo cấu trúc (Reconstruction Learning).

Toàn bộ quy trình được thực hiện thông qua một luồng xử lý khép kín từ tiền xử lý, lan truyền tiến đến tối ưu hóa tham số.

6.1 Hàm mất mát biểu diễn

Hàm mất mát biểu diễn $\mathcal{L}_{repr}$ (chi tiết tại Mục 4.3) được thiết kế nhằm tinh chỉnh và làm giàu không gian biểu diễn của các đặc trưng đầu vào sau khi được mã hóa thông qua tín hiệu học bổ trợ từ học tương phản.

6.2 Hàm mất mát hồi quy

Hàm mất mát hồi quy $\mathcal{L}_{eta}$ đo lường sai số giữa thời gian di chuyển dự đoán $\hat{y}$ và giá trị thực y . Nghiên cứu sử dụng hàm Smooth L1 (được đề cập tại Mục 2.13), được định nghĩa như sau:

$$\mathcal{L}_{eta} = \mathcal{L}_{SmoothL1}(\hat{y}, y) = \begin{cases} \frac{1}{2\beta}(\hat{y} - y)^2 & \text{nếu } |\hat{y} - y| < \beta \\ |\hat{y} - y| - \frac{\beta}{2} & \text{nếu } |\hat{y} - y| \geq \beta \end{cases} \quad (62)$$

trong đó β là siêu tham số.

Hàm Smooth L1 được lựa chọn vì hai lý do chính.

1. Trong vùng sai số nhỏ theo β ($|\hat{y} - y| < \beta$), cho phép tối ưu hóa mượt mà và gradient ổn định.
2. Trong vùng sai số lớn, hàm chuyển sang dạng tuyến tính, qua đó giảm thiểu ảnh hưởng của mẫu ngoại lai, vốn thường xuất hiện trong dữ liệu thời gian di chuyển do các sự kiện bất thường không thể lường trước, như tai nạn giao thông.

6.3 Cơ chế tối ưu hóa đa mục tiêu

Một thách thức lớn trong huấn luyện đa nhiệm là sự chênh lệch về biên độ giữa các hàm mất mát. Hàm mất mát hồi quy $\mathcal{L}_{eta}$ và hàm mất mát biểu diễn $\mathcal{L}_{repr}$ có bậc độ lớn khác nhau, khiến một trong hai có thể chiếm ưu thế và làm lệch hướng gradient.

Nghiên cứu đề xuất lớp *LossBalancer* kết hợp EMA và kẹp giá trị để cân bằng động hai thành phần loss. Tổng hàm mất mát:

$$\mathcal{L}_{total} = \beta \cdot \mathcal{L}_{eta} + (1 - \beta) \cdot \mathcal{L}_{repr} \cdot scale_s \quad (63)$$

Cập nhật EMA. Tại mỗi bước huấn luyện s , hai bộ tích lũy EMA được cập nhật:

$$EMA_s(\mathcal{L}) = \alpha \cdot EMA_{s-1}(\mathcal{L}) + (1 - \alpha) \cdot \mathcal{L}_s \quad (64)$$

với hệ số suy giảm $\alpha = 0.9$, tương ứng của số hiệu dụng $\frac{1}{1-\alpha} = 10$ bước. Điều kiện khởi tạo được đặt bằng giá trị thực tế của bước đầu tiên:

$$EMA_1(\mathcal{L}) = \mathcal{L}_1 \quad (65)$$

Cách khởi tạo này tránh hiện tượng khởi đầu lạnh vốn xuất hiện khi khởi tạo EMA tại 0, đảm bảo $scale_s$ có giá trị hợp lệ ngay từ bước huấn luyện đầu tiên.

Tính toán scale và cơ chế kẹp giá trị. Hệ số cân bằng $scale_s$ được tính từ tỷ số hai EMA và sau đó được giới hạn vào khoảng $[\gamma_{min}, \gamma_{max}]$ để tránh phân kỳ:

$$scale_s = \text{clamp}\left(\frac{EMA_s(\mathcal{L}_{eta})}{EMA_s(\mathcal{L}_{repr})}, \gamma_{min}, \gamma_{max}\right) \quad (66)$$

với $[\gamma_{min}, \gamma_{max}] = [0.1, 10.0]$ giới hạn mức độ điều chỉnh tối đa. Ràng buộc kẹp giá trị đặc biệt quan trọng ở giai đoạn đầu huấn luyện, khi $\mathcal{L}_{repr}$ chưa ổn định và tỷ số thô có thể dao động lớn.

Cơ chế này đảm bảo $\mathcal{L}_{repr}$ luôn được đưa về cùng bậc độ lớn với $\mathcal{L}_\eta$ tại mọi giai đoạn huấn luyện, giúp siêu tham số β thực sự điều tiết tỷ trọng đóng góp của từng nhiệm vụ thay vì bị nhiễu bởi biên độ thô của loss.

6.4 Chiến lược điều chỉnh tốc độ học

Nhằm giúp mô hình hội tụ ổn định, đặc biệt là các mô-đun nhạy cảm như Transformer Encoder và LayerNorm GRU, nghiên cứu áp dụng chiến lược khởi động kết hợp suy giảm Cosine:

- **Giai đoạn khởi động (Warmup):** Trong 2% tổng số bước huấn luyện đầu tiên, tốc độ học tăng dần từ 0 đến giá trị mục tiêu. Điều này giúp tránh hiện tượng gradient bùng nổ khi các trọng số khởi tạo ngẫu nhiên chưa thích nghi với dữ liệu.
- **Giai đoạn suy giảm Cosine (Cosine Decay):** Sau giai đoạn khởi động, tốc độ học giảm dần theo hàm cosine về gần 0 [32]. Cơ chế này cho phép mô hình thực hiện các bước nhảy lớn ở giai đoạn đầu để thoát khỏi các cực tiểu cục bộ và tinh chỉnh nhẹ nhàng ở giai đoạn cuối để đạt được độ hội tụ tối ưu.

6.5 Các kỹ thuật ổn định gradient và hiệu năng

Để xử lý các thách thức về bộ nhớ và độ ổn định khi huấn luyện chuỗi có độ dài biến thiên, các kỹ thuật sau được tích hợp trong luồng huấn luyện:

- **Huấn luyện độ chính xác hỗn hợp (Mixed Precision Training):** Để tối ưu hóa hiệu năng tính toán và tiết kiệm tài nguyên bộ nhớ GPU, nghiên cứu áp dụng kỹ thuật huấn luyện với độ chính xác hỗn hợp [33]. Thay vì thực hiện tất cả các phép toán trên số thực dấu phẩy động 32-bit (FP32), mô hình sử dụng đồng thời FP32 và FP16 (16-bit):
 - **Cơ chế hoạt động:** Các phép toán nặng về tính toán như nhân ma trận và tích chập được thực hiện ở định dạng FP16, giúp tăng tốc độ xử lý. Trong khi đó, các phép toán tích lũy được giữ ở định dạng FP32 để đảm bảo độ chính xác số học.
 - **Thang đo Gradient (Gradient Scaling):** Một vấn đề phổ biến khi dùng FP16 là hiện tượng "triệt tiêu số" (underflow) do phạm vi biểu diễn của FP16 hẹp hơn FP32, khiến các gradient quá nhỏ bị làm tròn về 0. Để khắc phục, nghiên cứu sử dụng Gradient Scaler. Kỹ thuật này nhân hàm mất mát với một hệ số thang đo (S) trước khi lan truyền ngược để đưa gradient vào vùng biểu diễn an toàn của FP16, sau đó giải tỷ lệ (unscale) trước khi cập nhật trọng số, đảm bảo tính ổn định và chính xác cho quá trình hội tụ của mô hình.
- **Gradient Clipping:** Các gradient được cắt ở ngưỡng 1.0 trước khi cập nhật trọng số. Bước này đặc biệt quan trọng đối với mạng lặp như GRU để ngăn chặn hiện tượng bùng nổ gradient qua nhiều bước thời gian.

6.6 Thuật toán huấn luyện tổng quát

Quy trình thực thi cho mỗi epoch được thực hiện qua các bước:

1. Pha huấn luyện (Train Phase):

- Kích hoạt chế độ tính toán gradient và lấy mẫu từ `DataLoader`.
- Lan truyền tiến qua mô hình để tính $\hat{y}$ và $\mathcal{L}_{repr}$.
- Tính tổng loss $\mathcal{L}_{total}$, thực hiện lan truyền ngược.
- Cập nhật trọng số theo từng batch.

2. Pha kiểm định (Validation Phase):

- Chuyển mô hình sang trạng thái `eval` và tắt tính toán gradient.
- Dự đoán trên tập kiểm định và tính toán các chỉ số đánh giá (MAE, RMSE, MAPE).
- So sánh chỉ số MAE hiện tại với giá trị tốt nhất. Nếu đạt kết quả tốt hơn, tiến hành sao lưu toàn bộ trạng thái hệ thống bao gồm trọng số mô hình, trạng thái bộ tối ưu và các tham số điều phối.

6.7 Các công thức đánh giá (Evaluation Metrics)

Để đánh giá hiệu quả của mô hình dự đoán thời gian đến (ETA) được đề xuất, chúng tôi sử dụng ba thước đo đánh giá phổ biến và được công nhận rộng rãi, bao gồm: *Root Mean Squared Error* (RMSE), *Mean Absolute Error* (MAE) và *Mean Absolute Percentage Error* (MAPE).

6.7.1 Root Mean Squared Error (RMSE)

RMSE là một thước đo thường được sử dụng để đánh giá độ chính xác của các mô hình hồi quy. Chỉ số này phản ánh độ lớn trung bình của sai số dự đoán và có cùng đơn vị với biến được dự đoán, trong trường hợp này là thời gian đến ước lượng (ETA). RMSE được tính bằng căn bậc hai của trung bình bình phương sai lệch giữa giá trị ETA dự đoán và giá trị ETA thực tế:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (67)$$

trong đó n là số lượng mẫu, y_i là giá trị ETA thực tế và $\hat{y}_i$ là giá trị ETA do mô hình dự đoán.

6.7.2 Mean Absolute Error (MAE)

MAE là một thước đo phổ biến khác, phản ánh độ sai lệch tuyệt đối trung bình giữa giá trị ETA dự đoán và giá trị ETA thực tế. Không giống như RMSE, MAE ít nhạy cảm hơn với các giá trị ngoại lai và cung cấp khả năng diễn giải trực quan hơn về mức sai số trung bình của mô hình. MAE được xác định bằng trung bình cộng của độ sai lệch tuyệt đối giữa giá trị dự đoán và giá trị thực tế:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (68)$$

6.7.3 Mean Absolute Percentage Error (MAPE)

Trong số các thước đo trên, luận văn sử dụng *Mean Absolute Percentage Error* (MAPE) làm thước đo đánh giá chính. MAPE được sử dụng rộng rãi trong các bài toán hồi quy do khả năng phản ánh sai số dự đoán dưới dạng phần trăm, giúp việc diễn giải hiệu năng mô hình trở nên trực quan và dễ hiểu hơn.

MAPE được xác định như sau:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100 \quad (69)$$

trong đó n là số lượng mẫu, y_i là giá trị ETA thực tế và $\hat{y}_i$ là giá trị ETA do mô hình dự đoán.

MAPE biểu diễn sai số phần trăm tuyệt đối trung bình giữa giá trị ETA dự đoán và giá trị ETA thực tế. Giá trị MAPE càng nhỏ cho thấy hiệu năng mô hình càng tốt, phản ánh khả năng dự đoán chính xác với sai số tương đối thấp.

7

Kết quả thực nghiệm

7.1 Môi trường thực nghiệm

Các thí nghiệm được thực hiện trên nền tảng Google Colab sử dụng GPU NVIDIA Tesla T4 (16GB VRAM). Môi trường phần mềm bao gồm:

- Python 3.10
- Pytorch 2.9.0
- CUDA

7.2 Tập dữ liệu kiểm nghiệm

Để đánh giá tính hiệu quả và khả năng tổng quát hóa của phương pháp, nhóm nghiên cứu chuẩn bị hai bộ dữ liệu quỹ đạo riêng biệt:

1. **Tập dữ liệu Porto (Porto Taxi Dataset):** Bộ dữ liệu chuẩn được công bố trên Kaggle [34], sử dụng làm thước đo cơ sở.
2. **Tập dữ liệu TP.HCM:** Bộ dữ liệu giao thông thực tế tại Thành phố Hồ Chí Minh do nhóm tự thu thập và xử lý.

1. Tập dữ liệu Porto

Sau khi lọc nhiễu và khớp bản đồ (map matching), bộ dữ liệu Porto còn lại **1.126.097** bản ghi, được chia theo tỷ lệ 8:1:1:

- **Tập huấn luyện (Training Set):** 900.877 bản ghi (80%).
- **Tập xác thực (Validation Set):** 112.610 bản ghi (10%).
- **Tập kiểm thử (Test Set):** 112.610 bản ghi (10%).

2. Tập dữ liệu TP.HCM

Bộ dữ liệu TP.HCM bao gồm tổng cộng **1.049.242** bản ghi hợp lệ. Để đảm bảo tính nhất quán trong quy trình huấn luyện, nhóm cũng thực hiện phân chia tương tự theo tỷ lệ xấp xỉ 8:1:1 như sau:

- **Tập huấn luyện (Training Set):** 839.393 bản ghi.
- **Tập xác thực (Validation Set):** 104.924 bản ghi.
- **Tập kiểm thử (Test Set):** 104.925 bản ghi.

Mục tiêu dự đoán chính của các mô hình ETA trong nghiên cứu là tổng thời gian di chuyển của một quỹ đạo. Giá trị này được lưu trong trường thông tin “time” (đơn vị: giây). Các mô hình được huấn luyện nhằm tối thiểu hóa sai số giữa giá trị dự đoán và thời gian thực tế này.

7.3 Các mô hình thử nghiệm

Nghiên cứu hiện thực đánh giá mô hình PoI-CL-TTE (xem Mục 4) với 1 mô hình cơ bản và 2 mô hình học sâu:

- **Linear Regression:** Mô hình cơ sở tuyến tính sử dụng tổng độ dài hành trình làm đặc trưng đầu vào duy nhất để dự đoán thời gian di chuyển thông qua một lớp tuyến tính $\hat{y} = w \cdot L + b$, trong đó L là tổng độ dài hành trình. Mô hình này không có khả năng nắm bắt các yếu tố không gian, thời gian hay cấu trúc tuyến đường, do đó phản ánh mức hiệu suất tối thiểu mà các mô hình học sâu cần vượt qua.
- **MulT-TTE:** Mô hình học sâu dựa trên kiến trúc Transformer đa luồng, xử lý đồng thời nhiều phương thức đặc trưng của hành trình bao gồm đặc trưng đoạn đường, thông tin không gian và thời gian khởi hành.
- **DeepTTE:** Mô hình học sâu kết hợp mạng tích chập (CNN) và mạng hồi quy (RNN) để mã hóa chuỗi GPS của hành trình. DeepTTE xử lý trực tiếp tọa độ GPS thô theo từng bước thời gian, không dựa trên phân đoạn đường bộ, từ đó phản ánh một hướng tiếp cận khác biệt so với các mô hình dựa trên bản đồ đường.

7.4 Kết quả thực nghiệm

Nghiên cứu này sử dụng 3 phương thức đánh giá: MAE (mean absolute error), RMSE (root mean square error) và MAPE để đánh giá hiệu suất của các mô hình.

Trong mô-đun học ngữ nghĩa của MulT-TTE (BERT MLM co-occurrence learning) và PoI-CL-TTE (Transformer Encoder Contrastive learning), chúng tôi đặt kích thước biểu diễn nhúng cũng như kích thước ẩn (hidden size) là 128 với 4 lớp Transformer Encoder cho MulT-TTE và 2 lớp Transformer Encoder cho PoI-CL-TTE, với 8 đầu chú ý (attention heads).

PoI-CL-TTE được huấn luyện trên tập huấn luyện với tối đa 20 epoch để đạt được epoch tối ưu nhất trên tập đánh giá.

Nghiên cứu sử dụng bộ tối ưu Adam với tốc độ học ban đầu là 5×10^{-3} trên tất cả các mô hình.

Bảng 2: So sánh hiệu năng giữa mô hình đề xuất và các baseline trên tập kiểm thử của Porto và TP.HCM.

Mô hình	Porto			TP.HCM		
	RMSE (s)	MAE (s)	MAPE (%)	RMSE (s)	MAE (s)	MAPE (%)
Linear Regression	213.3467	145.9934	25.1531	261.8113	198.5848	14.18
DeepTTE	156.0548	103.5346	16.2868	105.0757	79.7528	6.30
MulT-TTE	136.1171	92.3132	16.7291	76.6016	53.9334	4.40
PoI-CL-TTE	128.7875	85.3743	14.8879	37.7744	25.5981	1.91

Kết quả tại Bảng 2 cho thấy hiệu năng của các mô hình trên cả hai tập dữ liệu. Trên tập Porto, mô hình PoI-CL-TTE đạt RMSE = 133.89 giây, MAE = 86.23 giây và MAPE = 14.75%, cải thiện lần lượt khoảng 2.8%, 6.7% và 11.5% so với MulT-TTE. Trên tập TP.HCM, mức cải thiện rõ rệt hơn nhiều với RMSE = 39.99 giây, MAE = 26.22 giây và MAPE = 1.96%, giảm lần lượt 47.8%, 51.4% và 55.5% so với MulT-TTE.

Mô hình Linear Regression cho thấy hiệu suất thấp nhất trên cả hai tập dữ liệu, với MAPE lên đến 25.15% trên Porto và 14.18% trên TP.HCM. Điều này hoàn toàn có thể dự đoán được, vì mô hình chỉ sử dụng tổng độ dài hành trình làm đặc trưng duy nhất, bỏ qua hoàn toàn các yếu tố ảnh hưởng trực tiếp đến thời gian di chuyển thực tế như loại đường, mật độ giao thông, thời điểm khởi hành và cấu trúc tuyến đường. Trên thực tế, hai hành trình có cùng độ dài có thể có thời gian di chuyển chênh lệch rất lớn tùy thuộc vào các yếu tố này, khiến độ dài hành trình đơn thuần trở thành một đặc trưng không đủ mạnh để dự đoán chính xác. Kết quả này xác nhận rằng bài toán ước tính thời gian di chuyển đòi hỏi khả năng mô hình hóa các quan hệ phi tuyến phức tạp mà các mô hình học sâu mới có thể nắm bắt được.

Mô hình DeepTTE xử lý chuỗi GPS thô với tiền xử lý tối thiểu nên bị giới hạn về khả năng biểu diễn đặc trưng không gian. MulT-TTE cải thiện đáng kể so với DeepTTE nhờ học biểu diễn đa tầng kết hợp đặc trưng đoạn đường và thông tin ngữ cảnh, tuy nhiên cơ chế học tương phản dựa trên đồng xuất hiện MLM (MLM co-occurrence) chưa khai thác triệt để quan hệ ngữ nghĩa giữa các quỹ đạo có cùng thời gian di chuyển thực tế. Mô hình PoI-CL-TTE vượt trội hơn MulT-TTE nhờ ba cải tiến cốt lõi:

1. Thay thế MLM co-occurrence bằng học tương phản (contrastive learning), cho phép mô hình kéo gần các quỹ đạo có cấu trúc và đặc tính phân đoạn

và đẩy xa các quỹ đạo có cấu trúc khác biệt trong không gian biểu diễn, từ đó tạo ra biểu diễn có tính phân biệt cao hơn.

2. Tích hợp mô-đun điều chế theo thời gian bắt đầu (temporal modulation), cho phép mô hình thích nghi động với các điều kiện giao thông đặc trưng theo từng khung giờ thay vì chỉ học tương tác đơn giản qua lớp MLP.
3. Mã hóa thời gian bắt đầu theo chu kỳ giúp nắm bắt tính tuần hoàn của mẫu giao thông theo phút trong ngày, ngày trong tuần và ngày trong năm.

Ngoài ra, PoI-CL-TTE chỉ sử dụng 1,300,000 tham số (trong đó 899,000 tham số có thể huấn luyện), tương đương **25%** số tham số của MulT-TTE (5,240,000 tham số). Việc giảm đáng kể số lượng tham số kéo theo yêu cầu bộ nhớ thấp hơn trong cả quá trình huấn luyện lẫn suy luận (inference), cho phép mô hình có thể được triển khai trên các hệ thống phần cứng hạn chế hơn mà không đòi hỏi GPU hiệu năng cao. Trong bối cảnh triển khai thực tế, đây là một lợi thế quan trọng khi cơ sở hạ tầng tính toán thường là yếu tố giới hạn.

7.5 So sánh giải thuật định tuyến

7.5.1 Môi trường thực nghiệm và thiết lập

Để đảm bảo tính công bằng và khách quan trong việc đánh giá hiệu năng của hai giải thuật định tuyến Contraction Hierarchies (CH) và Multilevel Dijkstra (MLD), toàn bộ quá trình tiền xử lý và đo lường thời gian truy vấn đều được thực hiện trên cùng một máy tính cục bộ thông qua môi trường ảo hóa Docker. Cấu hình phần cứng cụ thể bao gồm: bộ vi xử lý AMD Ryzen 7 5800H (16 luồng tính toán), bộ nhớ trong (RAM) 16GB, và bộ xử lý đồ họa NVIDIA GeForce RTX 3050. Quá trình đánh giá được thiết lập lặp lại 3 lần cho mỗi kịch bản để lấy giá trị trung bình, qua đó loại trừ các độ trễ ngẫu nhiên từ hệ điều hành và môi trường mạng ảo của Docker.

7.5.2 Đánh giá thời gian và không gian tiền xử lý

Bảng 3: So sánh thời gian và không gian tiền xử lý giữa giải thuật CH và MLD

Khu vực	Tiến trình	Thời gian xử lý (giây)				TB RAM (GB)	Speedup
		Lần 1	Lần 2	Lần 3	Trung bình		
Việt Nam	CH (Contract)	1430.31	1293.45	1382.09	1368.62	4.10	1.0x
	MLD (Partition)	279.33	376.31	296.44	317.36	4.28	-
	MLD (Customize)	20.91	23.80	20.10	21.60	2.60	63.36x
Porto (Bồ Đào Nha)	CH (Contract)	297.70	293.69	409.46	333.62	1.16	1.0x
	MLD (Partition)	113.85	106.32	69.49	96.55	1.22	-
	MLD (Customize)	4.51	7.67	4.61	5.60	0.84	59.58x

Nhận xét: Kết quả từ Bảng 3 cho thấy sự khác biệt rõ rệt về chi phí tính toán khi hệ thống cần cập nhật dữ liệu. Đối với giải thuật CH, quá trình co đỉnh là một khối thống nhất và đòi hỏi sự tính toán lại toàn cục. Do đó, mỗi khi có thay đổi về giao thông, CH tiêu tốn khoảng thời gian rất lớn, trung bình hơn 22 phút cho mạng lưới Việt Nam, và sử dụng đến 4.10 GB RAM. Ngược lại, MLD tách biệt hoàn toàn cấu trúc hình học (Partition) và trọng số (Customize). Khi biến động giao thông xảy ra, MLD chỉ cần chạy lại tiến trình Customize. Thực nghiệm cho thấy tiến trình này chỉ tốn 21.60 giây tại Việt Nam và 5.60 giây tại Bồ Đào Nha, đạt tốc độ gia tốc (Speedup) lên tới khoảng 60 lần so với tiến trình Contract của CH. Đồng thời, lượng RAM tiêu thụ ở bước này cũng được tối ưu đáng kể, chứng minh sức mạnh của MLD trong điều kiện mạng lưới thay đổi liên tục.

7.5.3 Đánh giá thời gian truy vấn và Kết luận chung

Bảng 4: Thời gian truy vấn trung bình các giải thuật (mili giây).

Khu vực	Giải thuật	Thời gian truy vấn (ms)				Speedup (so với MLD)
		Lần 1	Lần 2	Lần 3	Trung bình	
Việt Nam (TP.HCM)	Dijkstra	3542.87	2947.68	2701.05	3063.87	0.037x
	CH	65.32	23.04	68.97	52.44	2.17x
	MLD	232.22	43.02	66.77	114.00	1.0x
Porto (Bồ Đào Nha)	Dijkstra	747.69	598.69	478.57	608.32	0.031x
	CH	20.68	18.62	20.94	20.08	0.95x
	MLD	19.48	17.88	19.97	19.11	1.0x

Nhận xét: Kết quả thực nghiệm tại Bảng 4 cung cấp bức tranh đa chiều về hiệu năng của ba giải thuật trên các quy mô đồ thị khác nhau.

Thứ nhất, số liệu chứng minh sự hạn chế của giải thuật Dijkstra truyền thống khi áp dụng vào hệ thống giao thông thực tế. Trên quy mô mạng lưới lớn như TP.HCM, Dijkstra tốn trung bình tới hơn 3 giây (3063.87 ms) cho một truy vấn, chỉ đạt mức hiệu năng bằng 0.037 lần so với MLD. Thời gian chờ đợi này là quá lớn, hoàn toàn không đáp ứng được yêu cầu phản hồi thời gian thực của một hệ thống web tương tác.

Thứ hai, khi loại bỏ Dijkstra, đối với mạng lưới giao thông quy mô siêu lớn như Việt Nam (hơn 25 triệu đỉnh), giải thuật CH thể hiện sức mạnh vượt trội khi thời gian truy vấn nhanh gấp 2.17 lần so với MLD. Điều này phản ánh đúng lý thuyết: cấu trúc phân cấp toàn cục của CH cho phép thu hẹp không gian duyệt hiệu quả hơn cơ chế nhảy qua các ô cục bộ của MLD trên các khoảng cách xa.

Thứ ba, trên các mạng lưới có quy mô nhỏ hơn như Bồ Đào Nha, thời gian truy vấn của cả hai giải thuật MLD và CH đều hội tụ về ngưỡng xấp xỉ 20 mili giây. Sự chênh lệch bị xóa nhòa ở đây không phải do bản chất thuật toán, mà do thời gian xử lý nội tại lúc này đã giảm xuống dưới 1 mili giây. Phần lớn độ trễ

ghi nhận được đến từ quá trình truyền tải mạng, môi trường ảo hóa Docker và khâu phân tích cú pháp dữ liệu. Đặc biệt, dữ liệu thực nghiệm cũng ghi nhận độ trễ khởi động trong lần chạy đầu tiên của MLD (lên tới 232.22 ms tại Việt Nam), trước khi tốc độ tăng vọt và ổn định ở các lần tiếp theo nhờ vào cơ chế bộ nhớ đệm cache.

Kết luận: Tổng hợp từ hai khía cạnh tiền xử lý và truy vấn, ta có thể thấy rõ sự đánh đổi bản chất giữa hai giải thuật. CH đánh đổi chi phí cập nhật dữ liệu cực lớn và chậm chạp để đổi lấy tốc độ truy vấn tức thời, biến nó thành giải pháp hoàn hảo cho các hệ thống bản đồ tĩnh. Tuy nhiên, kiến trúc của đồ án hướng tới một hệ thống dẫn đường thông minh có tích hợp trực tiếp mô hình dự báo thời gian di chuyển, đòi hỏi khả năng cập nhật trạng thái ùn tắc giao thông một cách liên tục. Giải thuật MLD, dù tốn thêm vài chục mili giây cho mỗi truy vấn, lại mang đến khả năng tùy chỉnh trọng số linh hoạt với tốc độ nhanh gấp 60 lần. Với sự cân bằng xuất sắc giữa thời gian phản hồi hệ thống và chi phí duy trì dữ liệu động, MLD được lựa chọn làm nền tảng định tuyến cốt lõi cho ứng dụng dẫn đường của đồ án.

8

Triển khai ứng dụng

8.1 Phân tích yêu cầu

8.1.1 Tổng quan về ứng dụng

Ứng dụng được xây dựng như một hệ thống hỗ trợ dự báo thời gian di chuyển và tìm đường trực quan, cho phép người dùng tương tác trực tiếp trên bản đồ số. Mục tiêu chính của ứng dụng là tích hợp và so sánh hiệu năng của các mô hình học sâu khác nhau trong việc giải quyết bài toán ước lượng thời gian đi lại.

Hệ thống cung cấp các chức năng chính bao gồm:

- **Tương tác bản đồ:** Người dùng có thể chọn điểm xuất phát và điểm kết thúc bất kỳ trên giao diện bản đồ tại khu vực TP. Hồ Chí Minh hoặc Porto.
- **Dự báo đa mô hình:** Hệ thống thực hiện dự báo đồng thời bằng các giải thuật tiên tiến như PoI-CL-TTE, Deep-TTE và MulT-TTE để đưa ra cái nhìn đa chiều về kết quả.
- **Đánh giá đối chiếu:** Kết quả từ các mô hình AI được đối chiếu trực tiếp với dữ liệu thực tế từ TomTom API, vốn được coi là mốc chuẩn để đánh giá độ chính xác.
- **Phân tích sai số:** Ứng dụng tự động tính toán và hiển thị các chỉ số sai số như MAE (Mean Absolute Error) và MAPE (Mean Absolute Percentage Error), giúp người dùng dễ dàng nhận thấy sự khác biệt về hiệu quả giữa các thuật toán.

8.1.2 Yêu cầu chức năng và phi chức năng

Yêu cầu chức năng

Yêu cầu chức năng mô tả các tác vụ cụ thể mà hệ thống phải thực hiện để hỗ trợ quá trình nghiên cứu và tương tác của người dùng:

- **Quản lý điểm tọa độ:** Hệ thống cho phép người dùng xác định điểm bắt đầu và điểm kết thúc thông qua tương tác chuột trực tiếp trên bản đồ hoặc tự động cập nhật tọa độ khi người dùng thay đổi vị trí điểm đánh dấu.

- **Tùy chọn khu vực dữ liệu:** Hỗ trợ chuyển đổi linh hoạt giữa các bộ dữ liệu khác nhau, đồng thời tự động điều chỉnh tâm bản đồ và nạp các đặc trưng mạng lưới đường bộ tương ứng.
- **Ước lượng thời gian di chuyển:** Thực hiện dự báo thời gian di chuyển dựa trên danh sách các giải thuật được chọn. Hệ thống phải xử lý luồng dữ liệu từ tìm đường thô, khớp nối bản đồ đến rút trích đặc trưng trước khi đưa vào mô hình học sâu.
- **Đánh giá chuẩn:** Tích hợp dịch vụ bên thứ ba (TomTom API) để lấy thời gian di chuyển thực tế làm cơ sở đối chiếu, từ đó tính toán độ lệch tuyệt đối và sai số phần trăm cho từng kết quả dự báo.
- **So sánh song song:** Cho phép hiển thị kết quả của mô hình chính đồng thời với các mô hình phụ trợ khác và giải thuật cơ sở (OSRM) để cung cấp cái nhìn tổng quan về hiệu quả của từng phương pháp nghiên cứu.

Yêu cầu phi chức năng

Các yêu cầu phi chức năng đảm bảo hệ thống vận hành ổn định, chính xác và có khả năng mở rộng trong môi trường thực nghiệm:

- **Hiệu năng xử lý:** Nhờ vào cơ chế xử lý bất đồng bộ, hệ thống phải đảm bảo thời gian phản hồi cho một yêu cầu dự báo phức tạp (bao gồm gọi nhiều API và chạy suy luận mô hình) diễn ra trong thời gian ngắn, tránh tình trạng treo giao diện người dùng.
- **Tối ưu hóa tài nguyên:** Sử dụng cơ chế nạp trọng số trì hoãn (Lazy Loading), các mô hình học sâu chỉ được nạp vào RAM khi người dùng thực hiện yêu cầu lần đầu tiên, giúp tiết kiệm bộ nhớ cho máy chủ AI khi không sử dụng.
- **Tính nhất quán dữ liệu:** Hệ thống phải đảm bảo các bước tiền xử lý dữ liệu như chuẩn hóa (Scaling) và đệm dữ liệu (Padding) được thực hiện chính xác theo đúng thông số của từng tác giả mô hình gốc, nhằm đảm bảo kết quả suy luận có độ tin cậy cao nhất.
- **Khả năng bảo trì và mở rộng:** Kiến trúc đăng ký mô hình cho phép nhà phát triển dễ dàng tích hợp thêm các giải thuật mới vào hệ thống backend mà không cần thay đổi cấu trúc cốt lõi của ứng dụng.
- **Trải nghiệm người dùng (Usability):** Giao diện cần hỗ trợ các phím tắt điều hướng (WASD) và các thao tác chuột linh hoạt (chuột phải để hoàn tác) để tối ưu hóa việc lấy mẫu dữ liệu thực nghiệm trên bản đồ.

8.2 Kiến trúc và Lựa chọn công nghệ

Thành phần công nghệ

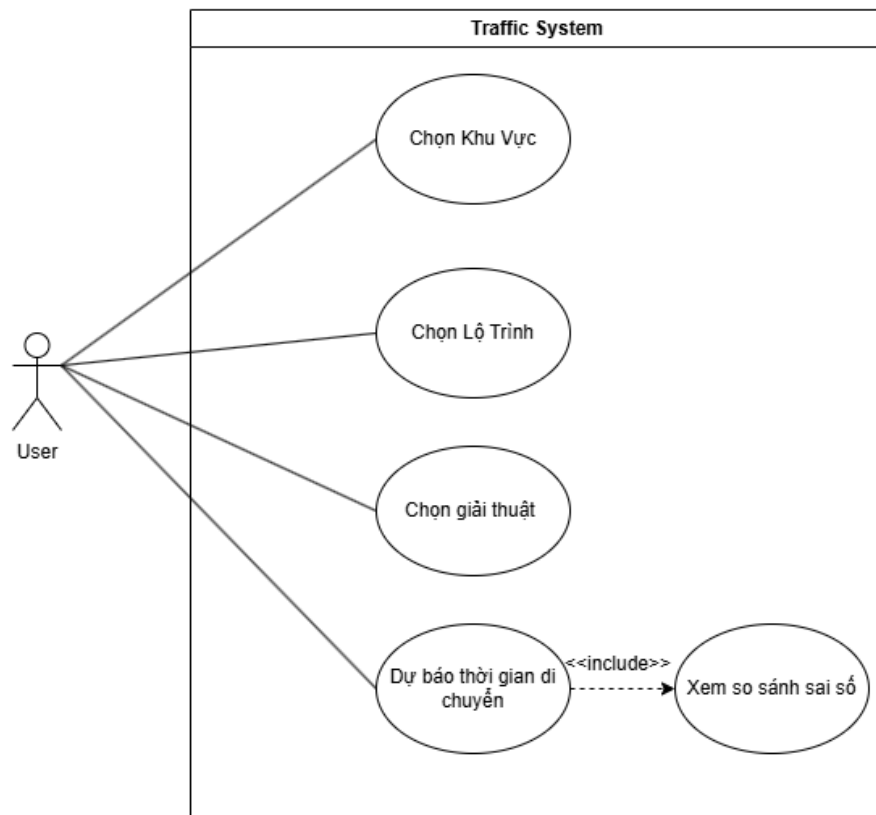
Để đảm bảo tính linh hoạt, khả năng mở rộng và hiệu năng xử lý các mô hình học sâu phức tạp, hệ thống được thiết kế theo kiến trúc Client-Server với các thành phần công nghệ sau:

- **Frontend:** Sử dụng thư viện React.js kết hợp với Leaflet API. Sự kết hợp này giúp xây dựng giao diện bản đồ mượt mà, hỗ trợ tốt việc hiển thị tuyến đường và các điểm đánh dấu theo thời gian thực.
- **Backend:** FastAPI (Python) được lựa chọn nhờ khả năng xử lý bất đồng bộ, cho phép hệ thống gọi cùng lúc nhiều API từ các mô hình AI khác nhau mà không gây nghẽn cổ chai.
- **AI Engine:** Toàn bộ logic dự báo được triển khai trên nền tảng PyTorch. Các trọng số mô hình đã được huấn luyện được nạp vào bộ nhớ RAM dưới dạng Lazy Loading để tối ưu hóa tài nguyên máy chủ.
- **Hệ thống định tuyến và Map Matching:**
 - **OSRM (Open Source Routing Machine):** Đóng vai trò tìm kiếm quãng đường ngắn nhất dựa trên dữ liệu OpenStreetMap sử dụng giải thuật MLD và CH.
 - **FMM (Fast Map Matching):** Sử dụng một server độc lập để thực hiện khớp tuyến đường thô từ OSRM vào danh sách các cạnh chính xác trong mạng lưới giao thông, phục vụ cho quá trình rút trích đặc trưng của mô hình AI.

8.3 Thiết kế hệ thống

8.3.1 Sơ đồ Use Case

Sơ đồ Use Case (Hình 19) mô tả trực quan các chức năng cốt lõi của hệ thống và sự tương tác của người dùng trong quá trình thực nghiệm.



Hình 19: Sơ đồ Use Case của hệ thống.

Chi tiết các trường hợp sử dụng (Use Case) được đặc tả trong Bảng 5, tóm tắt mục đích và điều kiện tiên quyết để thực thi các chức năng trong quy trình phần mềm.

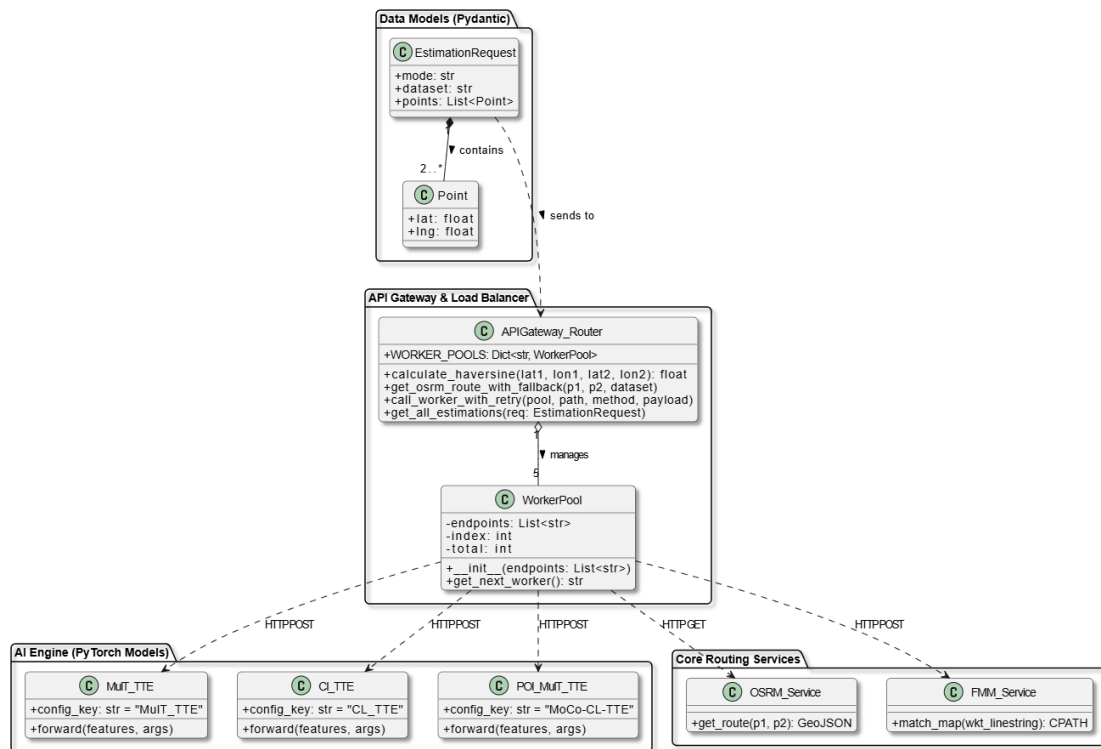
Bảng 5: Đặc tả các Use Case của hệ thống.

ID	Tên Use Case	Mô tả	Điều kiện tiên quyết
UC01	Chọn Khu Vực	Người dùng lựa chọn bản đồ thực nghiệm (TP.HCM hoặc Porto).	Ứng dụng đã khởi động thành công.
UC02	Chọn Lộ Trình	Người dùng xác định điểm đi và điểm đến thông qua tương tác trên bản đồ số.	Bản đồ khu vực đã được nạp.
UC03	Chọn Giải Thuật	Lựa chọn một trong các mô hình học sâu mục tiêu để thực hiện tính toán.	Hệ thống đã load danh sách mô hình.
UC04	Dự Báo Thời Gian	Kích hoạt luồng xử lý từ định tuyến, Map-matching đến suy luận AI.	Đã hoàn thành UC01, UC02, UC03.
UC05	Xem So Sánh Sai Số	Hiển thị kết quả AI song song với dữ liệu chuẩn (Ground Truth) và tính toán sai số.	UC04 được thực thi thành công.

8.3.2 Sơ đồ lớp (Class Diagram)

Sơ đồ lớp (Hình 20) mô tả cấu trúc tĩnh của hệ thống backend, thể hiện các lớp đối tượng, các thuộc tính, phương thức và mối quan hệ giữa chúng. Điểm nhấn của kiến trúc này là việc áp dụng mô hình phân tán **Workpool** kết hợp với thuật toán cân bằng tải **Round-Robin**.

Thay vì xử lý trực tiếp các tác vụ nặng như chạy mô hình học sâu hay khớp nối bản đồ, API Gateway đóng vai trò như một bộ điều phối trung tâm. Khi có một yêu cầu gửi đến, nó sẽ tra cứu trong danh sách WorkerPool. Thuật toán Round-Robin sẽ luân phiên điều hướng công việc đến máy trạm đang rảnh tiếp theo trong danh sách. Cơ chế này không chỉ giúp tránh tình trạng thất nút cổ chai khi có lượng lớn người dùng truy cập đồng thời mà còn tăng cường tính sẵn sàng cho hệ thống.



Hình 20: Sơ đồ lớp (Class Diagram) mô tả cấu trúc dữ liệu và cơ chế Workpool của hệ thống.

Chi tiết về các nhóm lớp thành phần, dữ liệu lưu trữ và chức năng được mô tả như sau:

- **Nhóm Data Models:** Đảm nhiệm việc định nghĩa và xác thực cấu trúc dữ liệu giao tiếp với Client.
 - Point: Lớp lưu trữ tọa độ không gian.
 - * *Dữ liệu:* lat (Vĩ độ - float), lng (Kinh độ - float).
 - EstimationRequest: Lớp cấu trúc gói tin yêu cầu từ người dùng.
 - * *Dữ liệu:* mode (Phương tiện di chuyển), dataset (Khu vực bản đồ), points (Danh sách các đối tượng Point).
- **Nhóm API Gateway & Load Balancer:** Chịu trách nhiệm tiếp nhận, phân giải và điều phối tải trọng.
 - WorkerPool: Lớp cốt lõi quản lý trạng thái của một nhóm máy trạm (Worker cluster).

- * *Dữ liệu:* `endpoints` (Danh sách địa chỉ IP/Port của các worker), `index` (Vị trí worker hiện tại đang trở tới), `total` (Tổng số lượng worker trong pool).
- * *Hàm:* `__init__()` (Khởi tạo danh sách máy trạm), `get_next_worker()` (Thực thi thuật toán Round-Robin để tính toán và trả về địa chỉ URL của worker tiếp theo).
- `APIGateway_Router`: Lớp điều khiển luồng chính (Controller).
 - * *Dữ liệu:* `WORKER_POOLS` (Từ điển ánh xạ định danh dịch vụ với đối tượng `WorkerPool` tương ứng).
 - * *Hàm:* `calculate_haversine()` (Tính khoảng cách đường chim bay để kiểm tra ngoại lệ), `get_osrm_route_with_fallback()` (Gọi API tìm đường), `call_worker_with_retry()` (Gửi yêu cầu HTTP đến Worker kèm logic thử lại khi lỗi), `get_all_estimations()` (API chính điều phối luồng thực thi song song).
- **Nhóm AI Engine:** Bao gồm các lớp định nghĩa kiến trúc mạng nơ-ron nhân tạo.
 - `MULT_TTE`, `PoI_MULT_TTE`: Các lớp đại diện cho các mô hình học sâu.
 - * *Dữ liệu:* `config_key` (Chuỗi định danh để map với cấu hình trọng số hệ thống).
 - * *Hàm:* `forward(features, args)` (Hàm lan truyền tiến thực hiện tính toán ma trận đầu vào và đưa ra kết quả dự đoán thời gian bằng giây).
- **Nhóm Core Routing Services:** Các dịch vụ hỗ trợ định tuyến và xử lý không gian.
 - `OSRM_Service`: Lớp cung cấp dịch vụ định tuyến MLD và CH.
 - * *Hàm:* `get_route(p1, p2)` (Nhận 2 điểm tọa độ, trả về hình học tuyến đường định dạng GeoJSON).
 - `FMM_Service`: Lớp cung cấp dịch vụ khớp nối bản đồ tốc độ cao.
 - * *Hàm:* `match_map(wkt_linestring)` (Nhận hình học tuyến đường thô dạng WKT, nội suy và trả về chuỗi mã cạnh chuẩn CPATH).

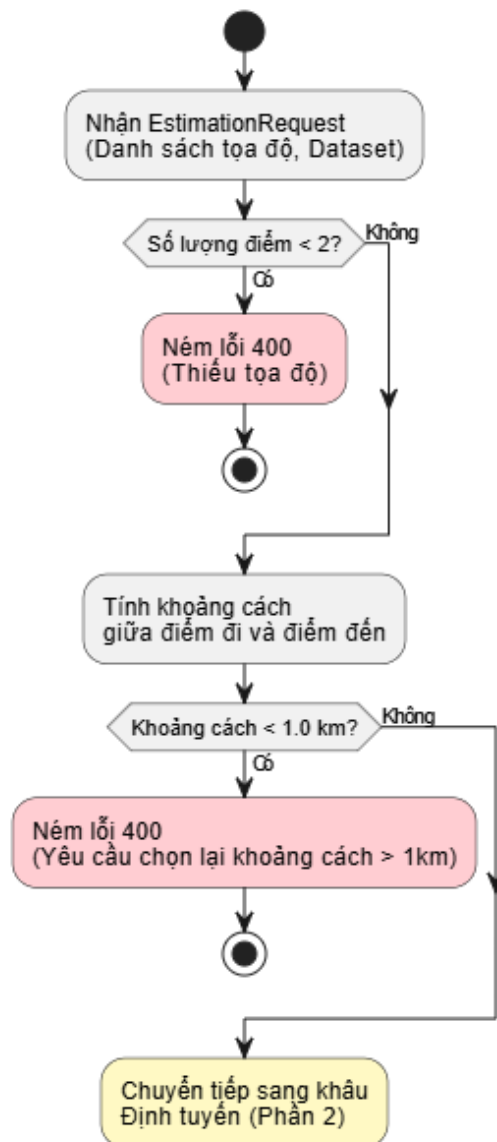
8.3.3 Sơ đồ hoạt động (Activity Diagram)

Sơ đồ hoạt động (Hình 21, Hình 22 và Hình 23) mô tả chi tiết logic điều khiển nội bộ (Control Flow) và thuật toán ra quyết định của hệ thống, đặc biệt là tại

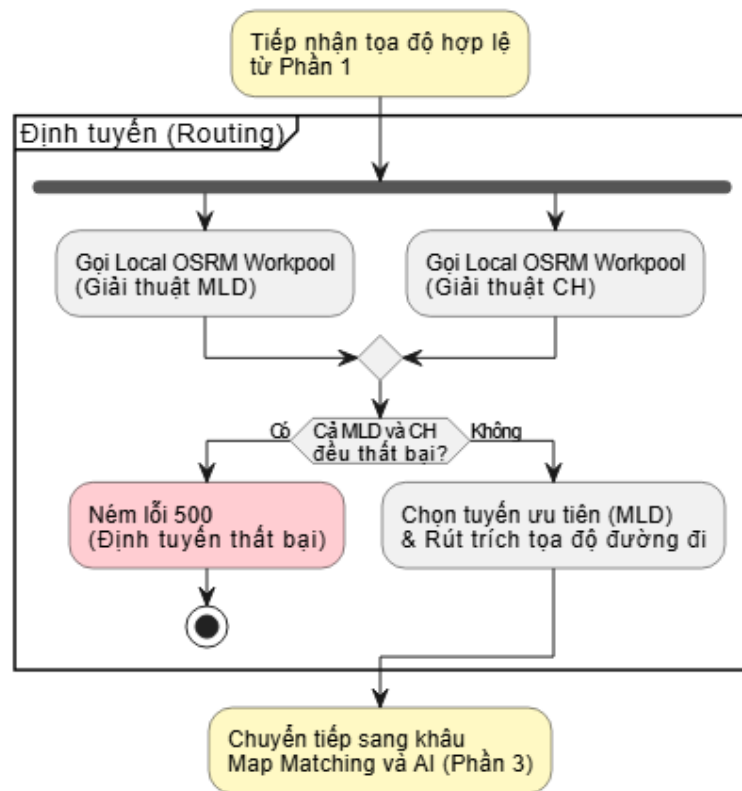
thành phần API Gateway, trong suốt quá trình xử lý một yêu cầu dự báo thời gian di chuyển.

Quá trình thực thi bắt đầu bằng bước kiểm tra tính hợp lệ của dữ liệu đầu vào thông qua việc tính toán khoảng cách đường giữa điểm xuất phát và điểm đích. Nhằm ngăn chặn các truy vấn vô nghĩa và tiết kiệm tài nguyên tính toán cho máy chủ, hệ thống sẽ lập tức ngắt luồng và trả về lỗi 400 (Bad Request) nếu cự ly di chuyển dưới 50 mét. Sau khi vượt qua bước xác thực, luồng điều khiển tiến vào phân khu định tuyến. Hệ thống sẽ gọi trực tiếp xuống các Workpool OSRM nội bộ nhằm cô lập độ trễ mạng, từ đó đo đạc chính xác thời gian truy vấn của các thuật toán.

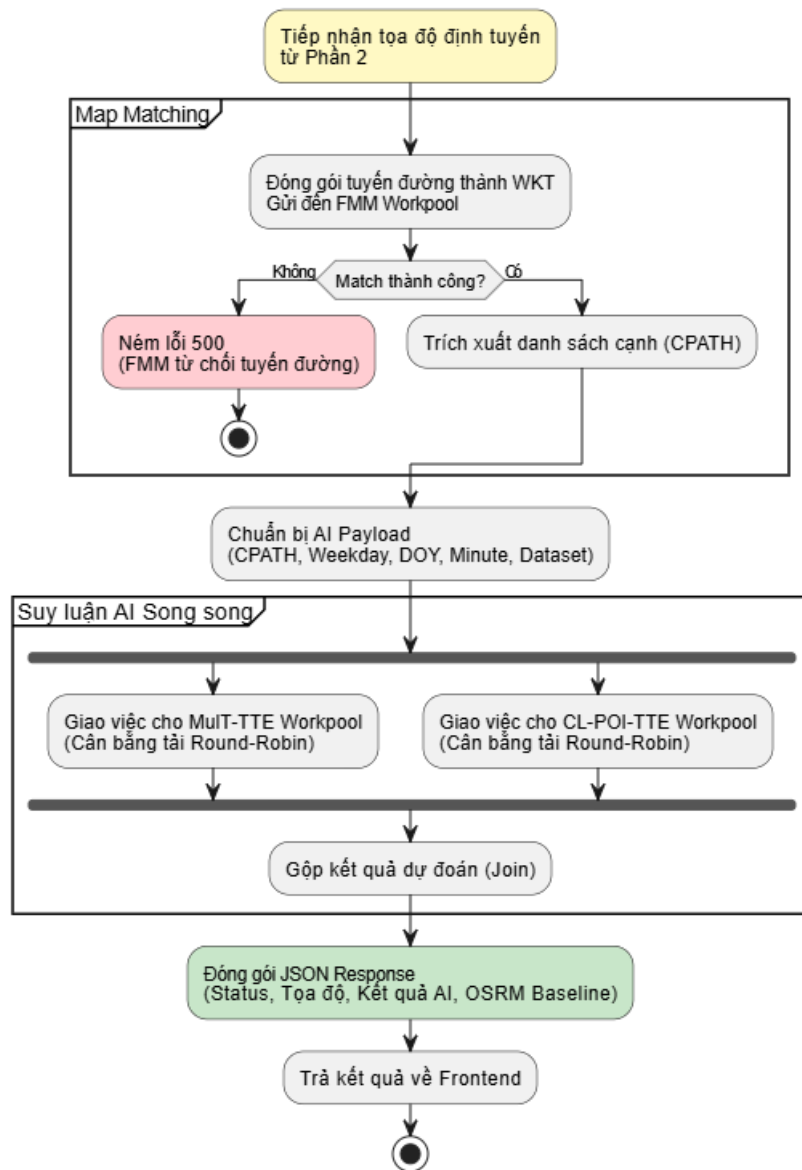
Tiếp nối quy trình, dữ liệu tuyến đường thô được chuyển đến phân khu Map Matching để nội suy danh sách mã cạnh chuẩn xác. Điểm tối ưu nhất trong luồng thuật toán nằm ở giai đoạn suy luận cuối cùng: thay vì thực thi các mô hình học sâu một cách tuần tự, hệ thống áp dụng cơ chế tách và gộp luồng (Fork/Join) của kiến trúc xử lý bất đồng bộ. Khối lượng công việc được phân phối song song xuống cả hai cụm máy trạm Mult-TTE Workpool và PoI-CL-TTE Workpool. Phương thức xử lý đồng thời này giúp hệ thống giảm thiểu triệt để độ trễ, trước khi đồng bộ và tổng hợp kết quả cuối cùng để trả về cho người dùng.



Hình 21: Sơ đồ hoạt động của API Gateway (Phần 1: Xác thực dữ liệu đầu vào).



Hình 22: Sơ đồ hoạt động của API Gateway (Phần 2: Cơ chế định tuyến OSRM).

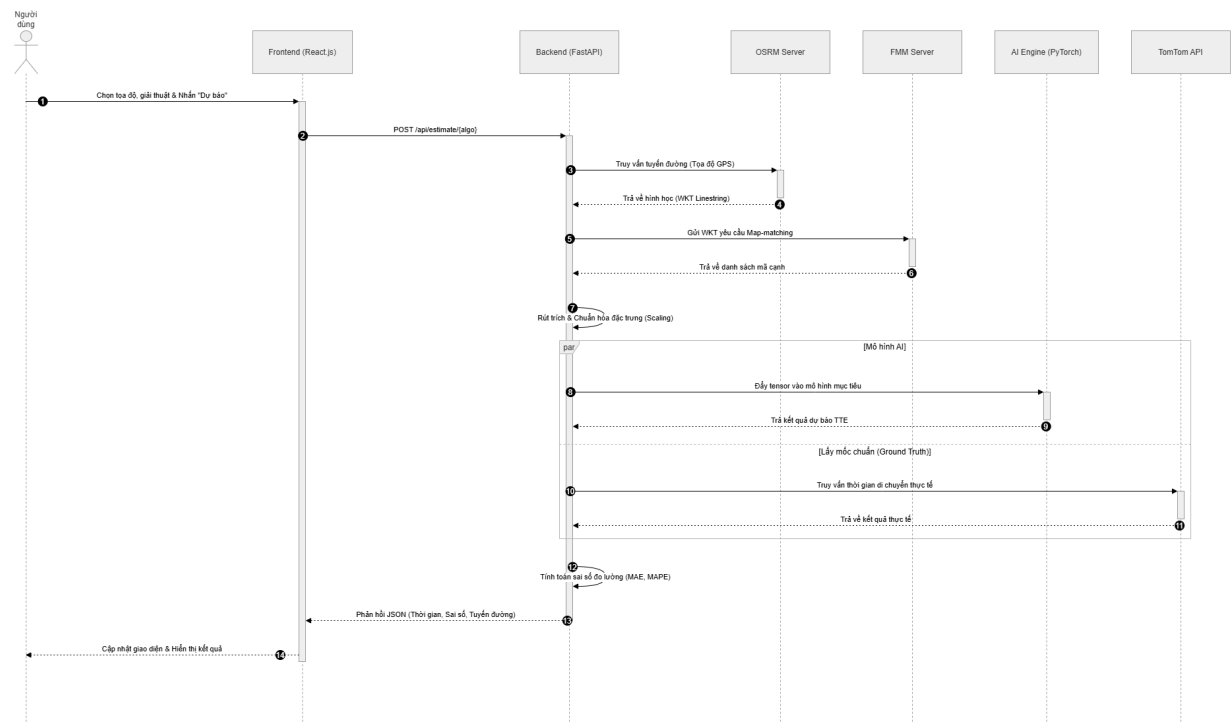


Hình 23: Sơ đồ hoạt động của API Gateway (Phần 3: Khớp nối bản đồ và Suy luận song song).

Luồng hoạt động tổng thể được phân tách thành các phân khu rõ ràng, giúp cô lập rủi ro tại từng giai đoạn tiến trình. Nếu bất kỳ một nút dịch vụ nào gặp sự cố (ví dụ: mô đun Map Matching không thể khớp tuyến đường vào mạng lưới), hệ thống sẽ chủ động ném lỗi 500 một cách có kiểm soát mà không gây treo hay sập toàn bộ ứng dụng.

8.3.4 Sơ đồ tuần tự (Sequence Diagram)

Sơ đồ tuần tự (Hình 24) thể hiện chi tiết luồng điều khiển và sự tương tác giữa các tiến trình hệ thống theo thời gian thực.



Hình 24: Sơ đồ tuần tự minh họa luồng xử lý dự báo thời gian di chuyển.

Quy trình vận hành cốt lõi được diễn ra qua các bước tuần tự như sau:

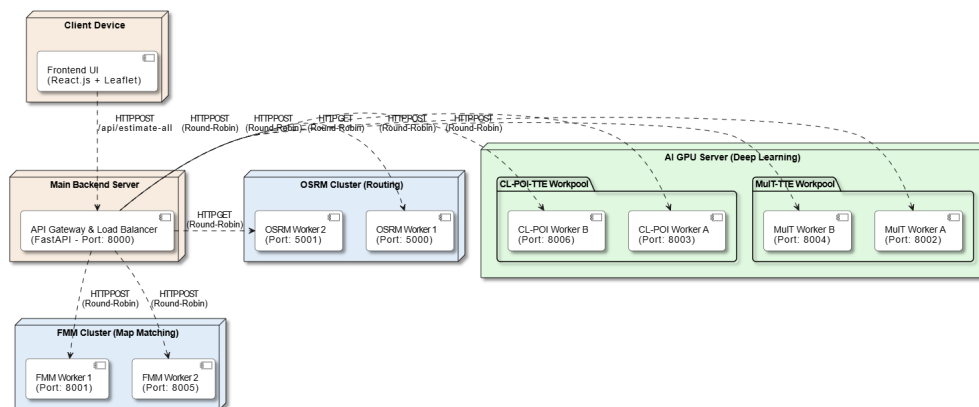
- Khởi tạo yêu cầu:** Giao diện React tiếp nhận thao tác của người dùng và gửi một HTTP POST Request chứa tọa độ GPS và thuật toán đã chọn đến FastAPI Backend.
- Định tuyến và Khớp nối bản đồ:** Backend lần lượt gọi OSRM Server để lấy hình học tuyến đường thô (Geometry dạng WKT), sau đó chuyển tiếp cho FMM Server để thực hiện Map-matching, nhận về danh sách mã cạnh chuẩn xác.
- Tiền xử lý dữ liệu:** Backend trích xuất các đặc trưng chiều dài, tọa độ, PoI tương ứng với danh sách cạnh từ RAM và thực hiện chuẩn hóa theo cấu trúc không gian chuẩn của mô hình mục tiêu.
- Xử lý song song:** Để tối ưu hóa thời gian phản hồi, hệ thống thực hiện hai tác vụ đồng thời:

- Đưa tensor dữ liệu vào PyTorch Engine để suy luận mô hình học sâu.
- Truy vấn TomTom API qua môi trường mạng để lấy thời gian di chuyển thực tế.

5. **Hoàn tất và Phản hồi:** Backend tổng hợp kết quả dự báo, tự động tính toán các thang đo sai số (MAE, MAPE) và trả gói dữ liệu (JSON) về để Frontend cập nhật lên biểu đồ hiển thị cho người dùng.

8.3.5 Sơ đồ triển khai (Deployment Diagram)

Sơ đồ triển khai (Hình 25) mô tả chi tiết cách thức phân bổ các thành phần phần mềm lên các nút mạng và giao thức giao tiếp giữa chúng. Để đáp ứng yêu cầu chịu tải cao và tối ưu hóa thời gian xử lý AI, hệ thống không sử dụng kiến trúc nguyên khối (Monolithic) mà được thiết kế theo mô hình phân tán với các cụm máy trạm độc lập.



Hình 25: Sơ đồ triển khai hệ thống dự báo thời gian di chuyển.

Hệ thống bao gồm các thành phần chính sau:

- **Client Device (Frontend Node):**
 - Nơi triển khai ứng dụng Web tương tác bằng React.js và Leaflet.
 - Chỉ giao tiếp duy nhất với hệ thống thông qua API Gateway bằng giao thức HTTP POST. Việc này đảm bảo tính bảo mật, giấu hoàn toàn cấu trúc mạng phức tạp của các máy chủ bên trong khỏi tầm nhìn của người dùng cuối.
- **Main Backend Server (API Gateway & Load Balancer):**

- Xây dựng trên nền tảng FastAPI (chạy trên Port 8000), đóng vai trò là người điều phối toàn bộ luồng dữ liệu.
- Server này không trực tiếp chạy các tác vụ tính toán nặng. Khi có yêu cầu mới, nó đóng vai trò là bộ cân bằng tải, sử dụng thuật toán *Round-Robin* để phân phối công việc luân phiên xuống các tiến trình Worker đang rảnh rỗi ở các cụm bên dưới.
- **Core Routing & Map Matching Nodes:** Cụm máy trạm phụ trách tiền xử lý dữ liệu không gian, được chia thành các Workpool:
 - **OSRM Cluster:** Gồm nhiều tiến trình OSRM Worker chạy song song để giải quyết bài toán tìm đường ngắn nhất.
 - **FMM Cluster:** Gồm các FMM Worker nhận hình học tuyến đường thô và thực hiện thuật toán Map-matching để khớp vào mạng lưới đồ thị thực tế.
- **AI GPU Server Node:** Môi trường tính toán hiệu năng cao chuyên dụng cho AI, thường được trang bị GPU:
 - **MuT-TTE & PoI-CL-TTE Workpool:** Mỗi giải thuật học sâu được cấp phát nhiều Worker chạy độc lập trên các cổng khác nhau. Việc tách biệt này giúp chặn tình trạng xung đột bộ nhớ VRAM, đồng thời cho phép mở rộng quy mô linh hoạt khi lượng người dùng tăng cao đột biến.

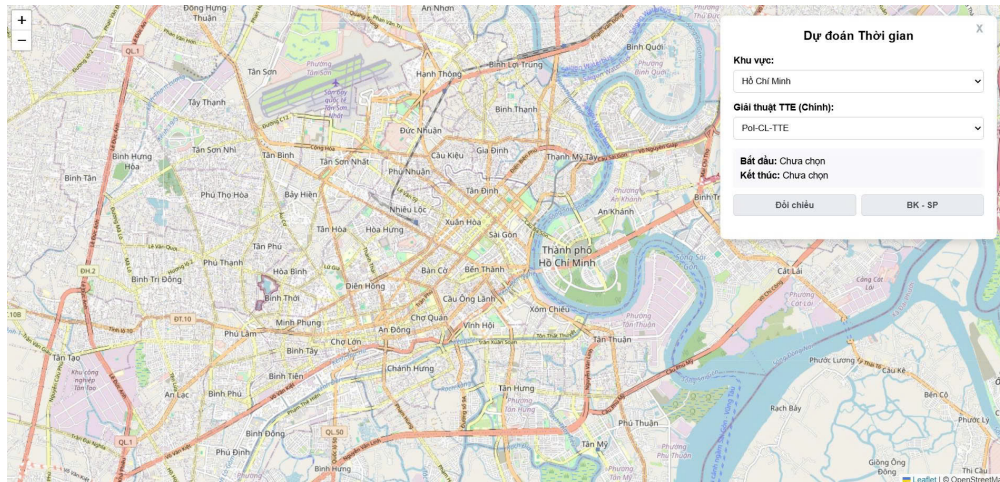
Việc chia nhỏ hệ thống thành các Workpool kết nối qua trung tâm API Gateway mang lại khả năng chịu lỗi rất cao. Trong trường hợp có một tiến trình Worker bị treo do quá tải, API Gateway sẽ tự động phát hiện và chuyển hướng yêu cầu sang các Worker khác trong cùng cụm, đảm bảo tính sẵn sàng cao cho toàn bộ ứng dụng.

8.4 Hiện thực hệ thống

Phần này trình bày kết quả hiện thực giao diện ứng dụng và các kịch bản tương tác thực tế của người dùng trên hệ thống dự báo thời gian di chuyển.

8.4.1 Giao diện tổng quan và Tương tác bản đồ

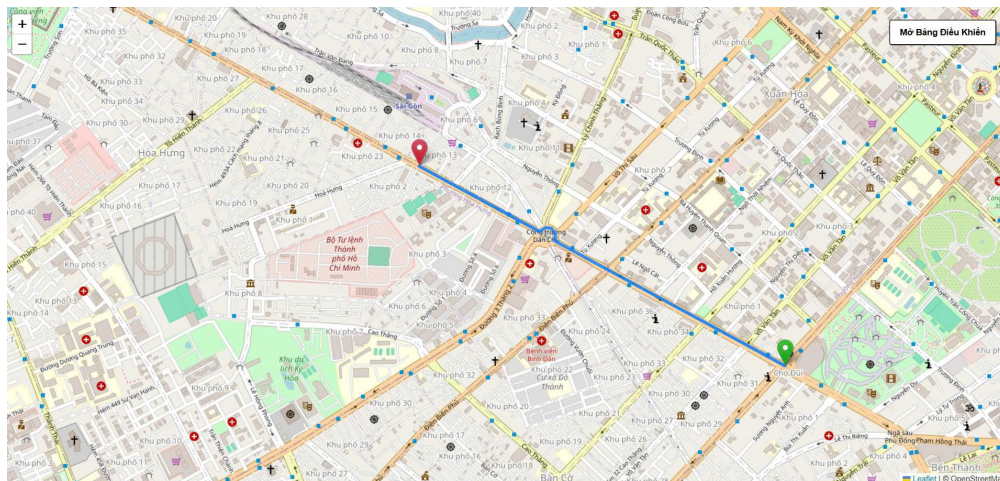
Giao diện ứng dụng được thiết kế tập trung vào trải nghiệm bản đồ số, cho phép người dùng bao quát toàn bộ khu vực thực nghiệm (Hình 26).



Hình 26: Giao diện bản đồ tổng quan của ứng dụng.

8.4.2 Xác định lộ trình thông qua GPS

Hệ thống cho phép xác định điểm xuất phát và điểm đích thông qua tương tác chuột trực tiếp. Các tọa độ GPS (Kinh độ, Vĩ độ) được cập nhật liên tục và hiển thị trên thanh công cụ để người dùng theo dõi (Hình 27).

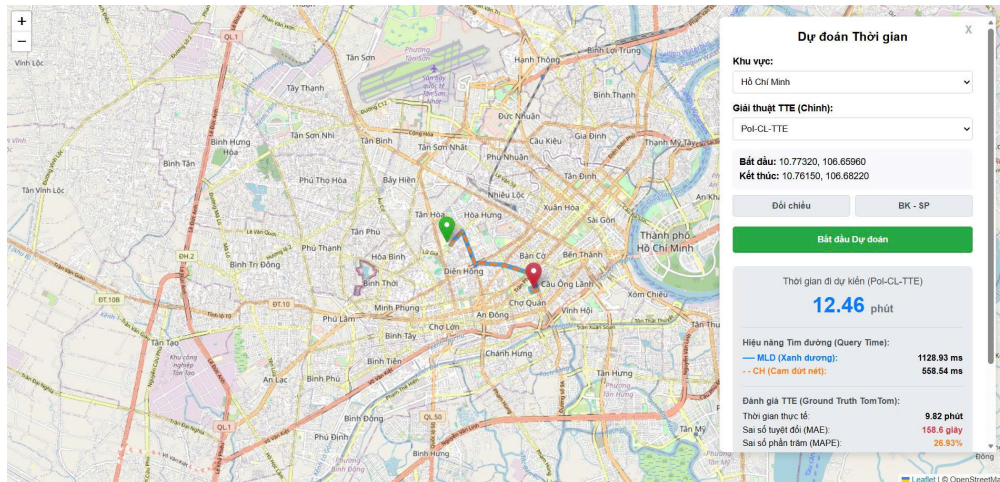


Hình 27: Giao diện chọn điểm đi và điểm đến bằng tương tác chuột.

8.4.3 Kết quả dự báo và Phân tích sai số

Trước khi thực hiện dự báo, người dùng có thể tùy chỉnh các tham số thực nghiệm tại bảng điều khiển bên trái, bao gồm việc lựa chọn các mô hình AI mục tiêu. Sau khi kích hoạt lệnh dự báo, lộ trình ngắn nhất được vẽ trực quan trên

bản đồ. Đồng thời, một bảng so sánh chi tiết sẽ xuất hiện, hiển thị thời gian dự báo của các mô hình AI đối chiếu với dữ liệu TomTom (Hình 28).



Hình 28: Hiển thị kết quả dự báo và các chỉ số sai số MAE, MAPE.

8.5 Kiểm thử hệ thống

Quy trình kiểm thử được thực hiện nhằm đảm bảo tính ổn định của các thành phần tích hợp và độ chính xác trong luồng xử lý dữ liệu từ Backend đến AI Engine.

8.5.1 Kiểm thử chức năng (Functional Testing)

Các trường hợp kiểm thử chức năng được xây dựng dựa trên các đặc tả yêu cầu đã nêu tại mục 8.1.2. Chi tiết các kịch bản thực nghiệm được trình bày trong Bảng 6.

Bảng 6: Các trường hợp kiểm thử chức năng chính.

ID	Kịch bản kiểm thử	Kết quả mong đợi
TS01	Chọn lộ trình nằm ngoài vùng dữ liệu thực nghiệm (ví dụ: Hà Nội).	Hệ thống hiển thị cảnh báo lỗi và ngăn chặn việc gửi yêu cầu dự báo.
TS02	Khoảng cách giữa điểm xuất phát và điểm kết thúc quá gần nhau (dưới 50 mét).	Hệ thống từ chối xử lý, ném lỗi 400 (Bad Request) và yêu cầu chọn lại khoảng cách lớn hơn để đảm bảo tính thực tiễn của giải thuật.
TS03	Chuyển đổi linh hoạt giữa các khu vực bản đồ (TP.HCM và Porto).	Hệ thống tự động di chuyển tâm bản đồ, nạp đúng mạng lưới và trọng số tương ứng.
TS04	Kích hoạt dự báo đồng thời với nhiều mô hình AI khác nhau.	Giao diện hiển thị bảng so sánh song song kết quả từ các mô hình và dữ liệu chuẩn.

8.5.2 Kiểm thử hiệu năng và Độ tin cậy

Để đánh giá khả năng chịu tải và tối ưu hóa tài nguyên, Backend được thiết lập với tổng cộng 8 Workers chạy song song, trong đó mỗi giải thuật lõi (FMM, MulT-TTE và PoI-CL-TTE) được phân bổ 2 Workers.

Kịch bản 1: Đánh giá hiệu năng tiêu thụ tài nguyên phần cứng

Mục tiêu của kịch bản này là đo lường thời gian xử lý và mức độ chiếm dụng tài nguyên (RAM, VRAM) của từng thành phần dịch vụ khi xử lý các yêu cầu đa dạng. Quá trình kiểm thử giả lập 3 người dùng liên tục gửi yêu cầu dự báo đồng thời lên hệ thống. Các lộ trình được chọn mang tính đại diện cao cho giao thông thực tế, bao gồm các cự ly khoảng 1km, 5km và 10km nhằm đánh giá độ ổn định của hệ thống khi khối lượng tính toán (độ dài tuyến đường) thay đổi liên tục.

Mô hình / Dịch vụ	Thời gian xử lý TB (s)	RAM CPU TB (MB)	VRAM GPU Đỉnh (MB)
OSRM Local (Định tuyến MLD)	0.045	2439.17	N/A (Chỉ dùng CPU)
OSRM Local (Định tuyến CH)	0.021	8500.50	N/A (Chỉ dùng CPU)
Fast Map Matching (FMM)	0.167	53.50	N/A (Chỉ dùng CPU)
MulT-TTE	0.467	58.60	151.50
PoI-CL-TTE	0.421	54.13	22.00

Bảng 7: Đánh giá và so sánh hiệu năng tiêu thụ tài nguyên của các mô hình

Kết luận Kịch bản 1: Qua việc kiểm thử với 3 luồng truy vấn đồng thời ở

các cự ly biến thiên (1km, 5km, 10km), Bảng 7 cho thấy hệ thống hoạt động cực kỳ ổn định và không xảy ra hiện tượng thất nút cổ chai. Bất chấp sự chênh lệch về độ dài tuyến đường, OSRM Local vẫn duy trì tốc độ phản hồi. Các dịch vụ xử lý chuỗi không gian như FMM hoạt động rất mượt với mức chiếm dụng bộ nhớ thấp (53.50 MB).

Đáng chú ý nhất là tại cụm AI GPU Server, khi phải tiếp nhận và nội suy ma trận dữ liệu song song từ 3 người dùng, các mô hình MulT-TTE và PoI-CL-TTE vẫn giữ được thời gian suy luận ở mức lý tưởng (dưới 0.5s). Việc kiểm soát VRAM cực kỳ tốt (chỉ chạm đỉnh 151.50 MB đối với MulT-TTE) chứng minh kiến trúc hệ thống hiện tại quản lý bộ nhớ rất hiệu quả. Đánh giá tổng thể, hệ thống đáp ứng xuất sắc khả năng xử lý song song đa luồng mà vẫn đảm bảo được tính toàn vẹn của dữ liệu và thời gian phản hồi theo thời gian thực.

Kịch bản 2: Kiểm thử chịu đựng (Stress Test) với lượng tải lớn

Kịch bản này giả lập tình huống hệ thống phải đón nhận lượng truy cập cao đột biến. Quá trình kiểm thử giả lập 20 và 25 người dùng liên tục gửi yêu cầu dự báo đồng thời lên API Gateway. Các lộ trình kiểm thử mang tính đại diện cao, cự ly ngẫu nhiên từ 1km đến 10km.

Số Concurrent Users	Tổng thời gian xử lý (s)	Throughput (Req/s)	Thời gian chờ TB (s)	Thời gian chờ Max (s)
20 Users	71.24	0.25	60.20	61.10
25 Users	87.77	0.22	73.07	74.29

Bảng 8: Kết quả kiểm thử chịu đựng của hệ thống Backend

Kết luận Kịch bản 2: Dữ liệu từ Bảng 8 cung cấp cái nhìn chi tiết về hiệu năng thực tế của kiến trúc Workpool dưới áp lực tải cao. Đi sâu vào các thông số hiệu năng, điểm sáng lớn nhất là sự chênh lệch cực kỳ nhỏ giữa Thời gian chờ trung bình và Thời gian chờ tối đa (60.20s so với 61.10s ở 20 Users). Sự đồng đều này là minh chứng rõ ràng cho thấy thuật toán cân bằng tải Round-Robin tại API Gateway đã phân phối khối lượng công việc rất tốt lên các Worker. Không có bất kỳ tiến trình nào bị quá tải cục bộ hay treo cứng. Thêm vào đó, khi nâng số lượng truy cập đồng thời từ 20 lên 25 người dùng, mặc dù tổng thời gian xử lý tăng lên do cơ chế hàng đợi, thông lượng (Throughput) của hệ thống chỉ giảm rất nhẹ (từ 0.25 xuống 0.22 Req/s). Điều này khẳng định năng lực duy trì tính ổn định của hệ thống: máy chủ không bị sụt giảm hiệu năng nghiêm trọng hay sập nguồn khi phải giải quyết khối lượng công việc lớn cùng lúc.

9

Tổng kết

Trong bối cảnh đô thị hóa diễn ra nhanh chóng với mật độ phương tiện gia tăng đột biến, đặc biệt tại các siêu đô thị có đặc thù giao thông hỗn hợp phức tạp như Thành phố Hồ Chí Minh, bài toán ước lượng thời gian đến đích (ETA) và tìm đường tối ưu đóng vai trò then chốt trong việc vận hành các hệ thống giao thông thông minh. Việc dự báo chính xác thời gian di chuyển không chỉ là nền tảng cốt lõi cho các ứng dụng định vị, gọi xe công nghệ và tối ưu hóa vận tải logistic mà còn góp phần quan trọng vào việc giảm thiểu ùn tắc cho xã hội. Tuy nhiên, các phương pháp học sâu hiện đại dựa trên dữ liệu quỹ đạo GPS tuy đã cải thiện đáng kể độ chính xác nhưng vẫn bộc lộ hạn chế do thường bỏ qua các thông tin ngữ cảnh không gian lân cận, đặc biệt là sự phân bố của các Điểm quan tâm (PoI).

Khắc phục những hạn chế kể trên, đề án đã nghiên cứu và đề xuất thành công mô hình học sâu đa phương thức mang tên PoI-CL-TTE. Giải pháp của nhóm tích hợp thông tin chuỗi lộ trình và ngữ cảnh khu vực thông qua cơ chế mã hóa PoI, kết hợp cùng phương pháp học tương phản có nhãn và bộ điều chế tuyến tính theo thời gian khởi hành. Cùng với đó, để giải quyết nút thắt về hiệu năng tìm đường trên các bản đồ quy mô lớn, đề án đã khảo sát và ứng dụng thành công giải thuật định tuyến Multilevel Dijkstra (MLD) với cấu trúc đồ thị phân hoạch lồng nhau.

Các kết quả thực nghiệm đã chứng minh tính hiệu quả và sự vượt trội của hướng tiếp cận này trên môi trường thực tế. Cụ thể, trên tập dữ liệu giao thông thực tế tại TP.HCM do nhóm tự thu thập và xử lý, mô hình PoI-CL-TTE đạt kết quả vô cùng khả quan với sai số MAPE chỉ 1.96%, giúp giảm tương ứng 47.8%, 51.4% và 55.5% trên các thang đo RMSE, MAE và MAPE so với mô hình cơ sở là MulT-TTE. Đối với bài toán tìm đường, giải thuật MLD cho thấy khả năng tùy chỉnh trọng số giao thông linh hoạt với tốc độ cập nhật nhanh gấp khoảng 60 lần so với giải thuật Contraction Hierarchies (CH) truyền thống, mang lại sự cân bằng hoàn hảo giữa tốc độ truy vấn và khả năng cập nhật động.

Điểm nhấn quan trọng cuối cùng của đề án là việc tích hợp các mô hình dự báo cùng giải thuật định tuyến vào hệ thống giao thông trực tuyến theo kiến trúc Client-Server. Ứng dụng không chỉ cho phép người dùng tương tác tìm đường trực quan mà còn cung cấp khả năng dự báo ETA đa mô hình, đồng thời tự động tính toán và so sánh sai số trực tiếp với dữ liệu thực tế. Những kết quả này đã khẳng định tính thực tiễn của đề tài, hoàn thành các mục tiêu đã đề ra và đặt nền móng cho các hướng phát triển tiếp theo trong tương lai.

Nguồn tham khảo

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.
- [2] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder–decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [3] A. Amidi and S. Amidi, “Cs 230 - recurrent neural networks cheatsheet,” <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>, 2019.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186.
- [5] Quantpedia, “Bert model – bidirectional encoder representations from transformers,” <https://quantpedia.com/bert-model-bidirectional-encoder-representations-from-transformers/>.
- [6] D. Wang, J. Zhang, W. Cao, J. Li, and Y. Zheng, “When will you arrive? estimating travel time based on deep neural networks,” in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*. New Orleans, Louisiana, USA: AAAI Press, 2018, pp. 2500–2507.
- [7] A. Derrow-Pinion, J. She, D. Wong, O. Lange, T. Hester, L. Perez, M. Nunkesser, S. Lee, X. Guo, B. Wiltshire, P. W. Battaglia, V. Gupta, A. Li, Z. Xu, A. Sanchez-Gonzalez, Y. Li, and P. Velickovic, “Eta prediction with graph neural networks in google maps,” *arXiv preprint arXiv:2108.11482*, vol. abs/2108.11482, 2021.
- [8] C. Yang and G. Gidófalvi, “Fast map matching, an algorithm integrating hidden markov model with precomputation,” *International Journal of Geographical Information Science*, vol. 32, no. 3, pp. 547–570, 2018.
- [9] T. Liao, L. Han, Y. Xu, T. Zhu, L. Sun, and B. Du, “Multi-faceted route representation learning for travel time estimation,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 9, pp. 11 782–11 793, 2024.

- [10] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, “Film: Visual reasoning with a general conditioning layer,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [11] D. Delling, A. V. Goldberg, T. Pajor, and R. F. Werneck, “Customizable route planning,” in *International Symposium on Experimental Algorithms*. Springer, 2011, pp. 376–387.
- [12] A. v. d. Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in neural information processing systems*, vol. 30, 2017.
- [14] R. Girshick, “Fast r-cnn,” in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1440–1448.
- [15] Z. Wang, K. Fu, and J. Ye, “Learning to estimate the travel time,” in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. New York, NY, USA: ACM, 2018, pp. 858–866.
- [16] Y. Li, K. Fu, Z. Wang, C. Shahabi, J. Ye, and Y. Liu, “Multi-task representation learning for travel time estimation,” in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. New York, NY, USA: ACM, 2018, pp. 1695–1704.
- [17] X. Fang, J. Huang, F. Wang, L. Zeng, H. Liang, and H. Wang, “Constgat: Contextual spatial-temporal graph attention network for travel time estimation at baidu maps,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. New York, NY, USA: ACM, 2020, pp. 2697–2705.
- [18] Q. Liu and H. Huang, “The kd-tree-based nearest-neighbor search algorithm in grid interpolation,” in *2012 International Conference on Image Analysis and Signal Processing*. IEEE, 2012, pp. 1–7.
- [19] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [20] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [21] R. Bellman, “On a routing problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

- [22] L. R. Ford Jr, “Network flow theory,” Rand Corp Santa Monica Ca, Tech. Rep., 1956.
- [23] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [24] A. V. Goldberg and C. Harrelson, “Computing the shortest path: A* search meets graph theory,” in *Proceedings of the sixteenth annual ACM-SIAM symposium on Discrete algorithms*, 2005, pp. 156–165.
- [25] U. Lauther, “An algorithmic approach to shortest path problems in road networks,” *Mitteilungen der Gesellschaft fur Informatik*, vol. 79, pp. 210–221, 2004.
- [26] M. Hilger, E. Köhler, R. H. Möhring, and H. Schilling, “Fast point-to-point shortest path queries with arc-flags,” in *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*, vol. 74. American Mathematical Society, 2009, pp. 41–72.
- [27] H. Bast, S. Funke, P. Sanders, and D. Schultes, “Fast routing in road networks with transit nodes,” in *Science*, vol. 316, no. 5824. American Association for the Advancement of Science, 2007, pp. 566–566.
- [28] I. Abraham, D. Delling, A. V. Goldberg, and R. F. Werneck, “Hierarchical hub labelings for shortest path queries,” in *European Symposium on Algorithms*. Springer, 2011, pp. 222–233.
- [29] D. Delling, A. V. Goldberg, T. Pajor, and R. F. Werneck, “Hub labels: Theory and practice,” *arXiv preprint arXiv:1303.0464*, 2013.
- [30] R. Geisberger, P. d. Sanders, D. Schultes, and D. Delling, “Contraction hierarchies: Faster and simpler hierarchical routing in road networks,” in *International Workshop on Experimental and Efficient Algorithms*. Springer, 2008, pp. 319–333.
- [31] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” *Proceedings of the 37th International Conference on Machine Learning (ICML)*, vol. 119, pp. 1597–1607, 2020.
- [32] I. Loshchilov and F. Hutter, “Sgdr: Stochastic gradient descent with warm restarts,” *arXiv preprint arXiv:1608.03983*, 2016.



- [33] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh *et al.*, “Mixed precision training,” *arXiv preprint arXiv:1710.03740*, 2017.
- [34] Kaggle, “Ecml/pkdd 15: Taxi trajectory prediction (i),” <https://www.kaggle.com/c/pkdd-15-predict-taxi-service-trajectory-i>, 2015, dataset accessed: 2025-12-19.